Implementing Portable and Efficient Software in an Open-Ended Language

C. H. A. Koster
Informatics department
Nijmegen University
Nijmegen
The Netherlands

H.-M. Stahl
epsilon · Gesellschaft
für Softwaretechnik und
Systementwicklung mbH
Berlin (West)

Working version, January 1990

Preface

Acknowledgement

We wish to thank all the people who collaborated in the development of CDL as a practical and useful programming tool and who have spread its use: Michael Bayer, Árpád Bedő, Bernhard Böhringer, Jean-Pierre Dehottay, and Hartmut Feuerhahn.

This book has been written over quite a number of years and at different locations, making use of at least two different formatting systems on different computer systems. Many people have contributed ideas and their work. To name only a few of the many, the authors thank here especially:

Hartmut Feuerhahn for thorough proofreading and for reformulating the chapters on static semantic checks and on the CDL2 LAB,

Ineke Kuster for converting most of the text to LaTeX and for performing a great number of changes and corrections to the text,

John van Krieken for the conversion to LaTeX and his support in "drawing" pictures,

the many students of the University of Nijmegen who deliberately tolerated to be exposed to incomplete preliminary versions of the book,

and — last but not least — Donald Knuth and the TeX-community for providing the tools to produce this final version.

Contents

Chapter 1

Introduction

In the computing community the term "implementation" stands for a major part of the software development cycle, starting after the design phase and ending with the first delivery of the product. Implementing large software systems is the honourable battlefield of the professional informatician, the serious side of programming. implementation

Implementing has the same relationship to freshman programming as war to manœuvers: professionalism replaces enthusiasm, collaboration is more important than individual accomplishment, harsh exigencies come in the place of academic leisure. Even though it is a peaceful and constructive activity, implementation projects have been known to end in disaster, in the brutal suppression of promise and in slavery.

Implementation is the technical side of software engineering. It is a human activity and therefore formal questions are often completely overshadowed by pragmatic considerations. In implementation it is not important what is possible, but what is feasible with the given human and material resources and in the time allowed.

It has long been realised that one of the important contributions to the success or failure of an implementation project comes from the choice of the programming language used and its support on the development hardware. By itself, the choice of a suitable implementation language cannot guarantee success but an unsuitable language can lead to a long and costly implementation, resulting in an unstable product, which is expensive to maintain and has a short economic life.

The implementation language is not an panacea but an important tool, that should be chosen with care.

In this book we will deal with one particular implementation language, CDL2, with the philosophy behind it, and the programming supports based on it. We will motivate its concepts and particularities both from the programming language point of view and from the viewpoint of software engineering.

This book is intended as a searching review of the basic concepts of programming. As such it is hoped to have a usefulness not limited to the implementation of portable and efficient software in CDL2.

1.1 About CDL

The acronym CDL stands for "Compiler Description Language". CDL2 is the revised version of this language. In the sequel, care will be taken to use the term CDL2 whenever some aspect is concerned only with the revised language, CDL1 when it only applies to the original language and its derivatives, and CDL if it applies to either. CDL

CDL at first and second sight is a very strange language, differing in notation concepts from the usual high-level languages. The language was explicitly designed to be the vehicle for a specific philosophy of programming. This style of programming differs so much from the usual style that it has to be taught along with the language.

open-ended

CDL is completely *open-ended* in that it lacks most of the features common to programming languages: It has no built-in operations like addition, multiplication and comparison; it has no assignation and has no mechanisms to access composed data.

Rather than containing predefined primitives, CDL starts from scratch: Its basis is a clean and powerful control structure together with mechanisms for semantic extension and abstraction.

As the acronym suggests, at the time of its design CDL [KOS71a] was intended for writing compilers, or rather for the (more or less static) description of compilers. This starting point is clearly reflected in the language: Its control structures are obviously syntactic in appearance and nature, and its syntax and semantics are firmly based on Affix grammars [KOS71b], a form of two-level grammars. With these means it is possible to adequately describe the main activities of a compiler, viz.:

- parsing
- syntax directed table maintenance
- input/output guided by parsing

It was realised in the early applications of CDL1 that in fact those activities are so general in nature ("data-directed algorithms") that their applicability far exceeds the field of compiler construction.

Accordingly, the language was revised in 1975 into a language for systems implementation, CDL2, with the following properties:

- explicit support of structured programming
- explicit support of modular programming
- technical solutions for software-engineering problems, such as portability, adaptability and efficiency
- integrated support of all phases of the implementation process.

ALGOL 60

A programming language can aid in structured programming, to different extents. ALGOL 60 was the first language to *allow* structured programming. The language in no way suggests how to program in a systematic fashion but it does contain suitable expressive means, such as identifiers without stringent restrictions on length, adequate control structures and a nice procedure mechanism so that many people have learned to express themselves clearly and elegantly within the given means.

Once a methodology of programming is well understood, it is possible to design programming languages such that they *support* it. By the careful choice of control structures and data structures and by the availability of suitable means for hierarchical decomposition, the systematic approaches can be encouraged, since they can be made just as simple to formulate as more sloppy approaches and short-circuits. CDL2 aims to support systematic programming.

Finally, once we understand well enough what is and what is not desirable behaviour in programming, we may try to *enforce* systematic programming by making it actually harder or even impossible to proceed in an unsystematic fashion, e.g. by the imposition of rigorous programming standards or by compulsory formal specification and verification. With CDL2 we have stopped short of that intention. Human beings cannot be forced to their luck.

SETL

Like Robert Dewar, one of the authors of the SETL language [SCH86] we believe in making

it easy to write good programs rather than making it hard to write bad ones. The qualification "good" and "bad" should however be applied to the understandability, maintainability, portability and efficiency of the resulting products.

1.2 History of CDL

The Compiler Description Language CDL was designed in 1970, and implemented in 1971, simultaneously with the first passes of an ALGOL 68 translator [KOS72] which served as a first test. ALGOL 68

For a compiler writer, the design and realization of an ALGOL 68 translator is the supreme test of his skill, for a number of reasons:

- its *syntactic* complexity, which invalidates all ad hoc or simple-minded parsing techniques
- its *static semantic* complexity, which causes the parsing problem to be only a small part of the total implementation effort
- the *many new concepts* introduced by ALGOL 68, such as coercion, balancing, identification of generic operators, as well as its unique run-time system, which call for wholly new implementation techniques, which appeared as yet in few text books (but see [BAU74])
- the *tantalizing nature of its formal definition* which, in spite of its extreme precision, does not lend itself readily to any automatic implementation technique.

(But, of course, ADA is an even more formidable challenge). ADA

1.2.1 CDL1

In the course of attempting to implement ALGOL 68, it was necessary to evolve parsing techniques driven by a two-level grammar, in order to stick as closely to the ALGOL 68 Report as possible. The two-level van Wijngaarden Grammar used in the ALGOL 68 Report [WIJ69] could not in any obvious fashion serve as a basis for syntax-directed parsing. Therefore as a basis for CDL *Affix Grammars* were chosen, a form of grammar, designed by L. Meertens in 1962 [MEE62], first used in linguistics [KOS65] and later defined formally [KOS71b].

Because of the power of Affix Grammars, not only the syntax, but also the static semantics including all aspects of parsing and table handling can be expressed in one same notation, a compiler compiler whose input language (viz. Affix Grammars) serves the role of an implementation language.

After some experimentation, the CDL language was frozen [KOS71a] and taught to others.

CDL1 was used for several years at the Technical University of Berlin (West) to teach compiler writing, and in the construction by students of various software systems, including an ALGOL 60 translator, as well as various compiler-like programs; it was ported in assembler language versions as well as high-level language versions onto a number of machines. It was also used in France [CUN76], in Hungary [SOM76] and in the German Democratic Republic [JAE??, OTT??], especially after being presented at the advanced course on compiler construction in 1974 in Munich [KOS74b].

The distribution policy at that time was to provide any interested party with a source text of the CDL1 compiler. It was hoped that useful software would be produced and shared among the CDL1 users. The CDL1 compiler was taken up rapidly enough, especially by academic implementation groups, and some useful work was done with it. Most recipients however added their own extensions to the CDL1 compiler, thus precluding the sharing of software.

In the course of teaching CDL1 and ALEPH, a variant in use at the Mathematical Centre in Amsterdam [GRU73, GRU82], it was realised that, by accident rather than by design, CDL1 had the property of greatly encouraging structured programming. ALEPH

1.2.2 CDL2

Based upon these experiences, CDL was revised, the major change being the addition of mechanisms for modular programming and static semantic checks. After some experimentation, in student courses and projects, with intermediate versions embodying various models of modularity (see chapter 3), the language was frozen at the end of 1975 under the name of CDL2. In 1975/76 the CDL2 compiler was implemented at the Technical University of Berlin by H.-M. Stahl, H. Feuerhahn and J.-P. Dehottay, supported by a grant from the Deutsche Forschungsgemeinschaft (DFG). It was, at first, hard to convince CDL1 users to switch over to the new style. As incentives, the CDL2 compiler was made to provide a number of features which the CDL1 compiler lacked

- global optimization
- static semantic checks
- separate compilation
- facilities for profiling and tracing.

In order to ensure the availability of the language to users on diverse machines, a number of code generators were written.

It was realised that the CDL technology should not be restricted to academic environments and starting from 1976, contacts were made with some manufacturers and software houses in Germany.

1.2.3 The CDL2-LAB

As a result of these contacts, in March 1977 a project was started at the Technical University of Berlin, supported by the German Federal Ministry for research and technology (BMFT) and by Siemens AG, to construct a programming system based on CDL2 and suitable for industrial use.

The resulting CDL2 LAB was completed in 1980, whereupon a number of project members started the software house epsilon, a spin-off from the Technical University of Berlin, which obtained the rights to all the CDL2 software.

While this project was going on, the further development and maintenance of the CDL2 compiler was undertaken by the Informatics department of the Catholic University of Nijmegen. A central step in the further development was the exemplary design of a machine-independent code generation interface by P. Holager [HOL80]. This same interface was adopted for the CDL2 LAB, thus ensuring the exchangeability of code generators. The resulting CDL2 LAB [BAY81] has been implemented on a large number of different computers. Since then, CDL2 has found many applications in high-technology software production, such as

- the COBOL-compiler developed by MBP Dortmund and commercially available on many computers, including then IBM PC [KIP82],
- the ANSWER operating system developed by the Hungarian Research Institute for Computer Science SZÁMKI [SOM79],
- the MPROLOG system, developed by the software house SzKI in Budapest
- the EUMEL-system, an extensible multi-user programming environment developed by Gesellschaft für Mathematik und Datenverarbeitung
- the software for the Mephisto Chess Computer
- several Software Engineering environments (the CDL2 LAB itself, SEGRAS [KRA85] and GRASPIN [TEU89])

as well as other commercial applications in the software industry and the printing industry.

1.2.4 Tiny CDL2

In order to allow a broader class of users to become acquainted with CDL2 and the CDL philosophy, in 1986 a simple straightforward implementation of CDL2 in PASCAL was made by Th. P. van der Weide ("TINY CDL2") which is distributed together with this book. It implements the full language, but has no optimizer and no semantic checker, and generates PASCAL programs for widely available personal computers.

It can be seen as a generator and development tool for PASCAL programs, and as a convenient introduction to the CDL2 programming methodology. Due to the straightforward translation from CDL2 to PASCAL and the inherent inefficiency of many PASCAL implementations, this implementation is not really suitable for production purposes. Complete implementations of CDL2, including all the support tools, can be obtained from

epsilon
Gesellschaft für Softwaretechnik
und Systementwicklung mbH
Kurfürstendamm 188-189,
D-1000 Berlin 15.

Chapter 2

Programming in-the-small: algorithms

In this chapter we will deal with the structure in-the-small of CDL2 programs, on the algorithmic side. Before that, we will introduce some concepts and terminology pertaining to programming languages in general, which is intended to show the motivation for the particular mechanisms present in CDL. In a number of places we give examples using PASCAL as a representative high-level language. No deep knowledge of that language is required.

2.1 Concepts and terminology

This section is concerned with elements of a philosophy of programming languages rather than with details of CDL.

A program is a piece of text, in some suitable representation. This text is not an amorphous mass of symbols but consists of a hierarchy of constructs according to the syntax of the programming language. These constructs can be broadly classified into those denoting *algorithms* and those denoting *objects* and *types*. Other constructs appearing in the syntax of the language can be considered as ancillary constructions in describing the relationship between the algorithms involved in the program.

algorithm
object
type

2.1.1 Entities

The dichotomy between algorithms on the one hand and objects and types on the other is similar to the distinction between verbs and nouns in natural languages. It may be an artifact of our way of thinking about the world or it may be its very basis. At any rate it is so helpful in thinking about algorithms and so widely spread amongst programming languages that we will turn it into dogma: the meaningful parts of any program are those that can be classified into algorithms, objects and types. In this classification it is at first sight somewhat disturbing to find objects and types (as classes of objects) to be discussed in one breath but every object is an instance of some type, and many relevant properties of individual objects are attributes of their types, and vice versa. At this point in the present discussion there is no need to distinguish between the two. It is awkward that we do not possess one term to denote both objects and types, but we will have to live with that.

Algorithms, objects and types are pieces of program text, that have two important properties:

- They either have a *name* or can be given one
- Upon execution of the program they *possess* some value internal to the computer:

- an algorithm possesses "executable code"
- an object possesses the internal representation of some value
- a type possesses a class of internal representations for its instances and a class of operations applicable to them.

entity

Algorithms, objects and types together we will call *entities*. The fact that entities can have a name proves one of the keys to successful programming. The most convenient way to denote an entity is by its name. Indirectly this allows us to manipulate values during execution of the program.

2.1.2 Composed and elementary entities

An entity occurring at a specific place in a program text can be either *elementary* or *composed*.

We call an entity elementary if according to the syntax and semantics of the language and at the level of abstraction provided by the context, it cannot sensibly be decomposed into other entities.

Thus the identifier *pi* bears in PASCAL no relationship to the identifiers p and i: it is the name of an elementary object, viz. a named constant. The expression

$$\textbf{if } a > b \textbf{ then } a \textbf{ else } b$$

on the other hand is a composed algorithm for computing the maximum of a and b.

2.1.3 Abstract and concrete entities

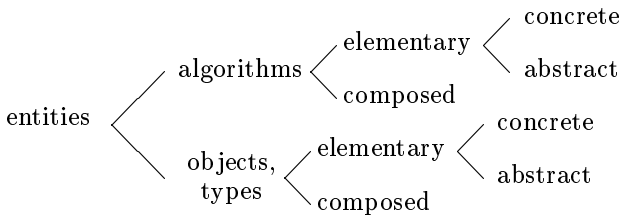


Figure 2.1: Taxonomy of constructs

For any programming language, we will call those elementary entities that are part of the language the *concrete* entities of that language. The programmer is generally free to add more entities to the language (by declaring them) which will then be called *abstract* entities: abstract entities are those that the programmer has to define himself, concrete entities are those defined for him and available for his use. These relations are shown in figure 2.1.

2.1.4 Kernel of the language

The kernel of a programming language comprises

- its *elementary concrete algorithms* (such as: assignation, jump, selection, subscription, access and coercions, sine function and print command)
- its *elementary concrete types* (highly dependent on the language, these may be denoted by constructions like "**int**" or "1..32767" or "`BIN FIXED (31)`")
- its *elementary objects* (in this category falls e.g. a predefined constant "*pi*").

2.1.5 Construction mechanisms

We use the term *construction mechanisms* in the language that serve to build composed constructs out of simpler ones, viz.

- for algorithms: *control structures* (the canonical Dijkstra collection (**while**, **if** and **;**) or e.g. LISP conditionals) as well as the *expression* or *procedure call* notation.
- for objects and types: *data structures*, e.g. (in ALGOL 68) "**ref m**" or (in PASCAL) "**record** ... **end**" or "**array**[1..*n*] **of** *integer*" as well as the associated *value constructors* (e.g. in ALGOL 68: row displays and structure displays).

2.1.6 Abstraction mechanisms

Abstraction mechanisms serve to build new elementary constructs out of constructs in the language. They comprise:

- declarations for algorithms (in the form of procedures, functions, subroutines, operators or macros)
- declarations for objects (such as variables, constants and arrays)
- declarations for types (abstract data types)

2.1.7 Extension mechanisms

Extension mechanisms serve to borrow new elementary constructs from outside the language. Classically they comprise libraries and macros, or even assembly code patches. Notice that they generally serve for *semantic* extension only. The notion of syntactic extension, popular in the early sixties, was less fruitful and seems to have been largely abandoned.

2.1.8 Mechanisms in CDL

The language CDL is practically unique in having an empty kernel — it has no concrete algorithms, types or objects at all. These have to be supplied by means of the extension mechanisms. In this sense CDL is an extreme example of an *open-ended language*.

It has construction mechanisms only for algorithms, not for objects and types. Indeed the language is typeless. It has however elaborate abstraction mechanisms to compensate for the absent aspects.

CDL was designed deliberately to explore an extreme position. One consequence is the fact that, in learning CDL, the familiar concepts of programming languages and of systematic programming have to be carefully reconsidered. The extreme position taken has proved to be fruitful in simultaneously achieving portability and efficiency [STA80].

In practice, a more or less standard library of predefined abstract entities is used, so that programming in an open-ended language is not much more strenuous than programming in a conventional high-level language.

2.2 Names in CDL

In CDL practically *all entities have names*. This sounds like a triviality, but is an essential and striking aspect of the language, and one of the barriers to overcome in learning it. name

There are no anonymous intermediate operands like in the expression

$$(a + b) * (a - b)$$

and no composed operands such as $r[i]$. The language is not expression oriented in the classical sense but statement oriented; composed algorithms can however be considered as "control expressions".

It is indeed possible to compose quite lengthy algorithms by the aid of control structures, but there is a strong invitation to use calls of separately declared algorithms rather than nesting the control structures deeply. As a consequence, CDL programs tend to consist of many small procedure declarations.

Secondly *there are no anomalous names.* In most languages, there are special kinds of names for operations, types and for some algorithms like the ternary "replace element" operator "[ ] :=" in $a[i] := x$.

In CDL all names have the form of identifiers. There is no limitation on identifier length, apart from those physical limitations imposed by the implementation. Layout carries no meaning and can be used freely to enhance readability, but the implementation attempts to conserve spaces within names.

As an example, the names `print it`, `prin tit` and `printit` are all equivalent, but the compiler will remember the positions of the spaces in the first version it meets, and use the same layout on output.

Experience shows that programmers, if they cannot avoid giving a name, are tempted to give a sensible one. In learning CDL, mediocre programmers initially find the constant necessity for choosing names a burden. They go through a period of giving incredibly long and overly concrete names and only with time acquire the knack of giving short, expressive names. In fact, they discover the power of *verbal abstraction.* They learn to distinguish what an algorithm does from how it does this.

2.3 Control structures in CDL

Apart from their unusually short notation, the control structures are rather classical. They are: sequencing, conditional choice, grouping and repetition. It will become apparent that the notation
PROLOG for the control structures is quite similar to that in PROLOG. This is no accident, since both languages have as their origin a syntactic formalism. Indeed, it might be said that CDL2 is a deterministic (and therefore highly efficient) variant of PROLOG.

2.3.1 Sequencing

sequencing The sequential execution of algorithm calls is indicated by writing a comma between the calls, which can best be read: "and then". As an example, the text

```
read a number, print it
```

strongly suggests that first a number is read, and then it is printed. In the same way,

```
read a number, read another number, add them, print sum
```

indicates four algorithms to be executed in strict order. (Of course these are abstract algorithms, they are much too specialized to be included in any concrete programming language).

2.3.2 The conditional

The conditional choice between two alternatives is expressed by writing a semicolon between them. This separator can best be pronounced "or else". As an example

```
is minus, read number, invert it, print it;
read number, print it
```

might be part of some desk calculator or interpreter. The idea is that `is minus` is the name of an algorithm that may either succeed (if it recognizes a minus sign in the input) or fail (if it doesn't). If `is minus` succeeds, the first alternative is taken, and otherwise the second is taken.

2.3.3 Classification of algorithms

In CDL, all algorithms are classified into two categories, depending on their influence on the control of execution:

- those that can either succeed or fail viz. the *predicates* and *tests*. They can be seen as Boolean procedures whose value is used to control the execution. — predicate, test
- those that cannot fail viz. the *actions* and *functions*. They can be seen as Boolean procedures that always return the value true. — action, function

We will meet with tests and functions later and for the time being will only deal with actions and predicates without parameters. We will, in our examples, distinguish actions from predicates by their names: names of predicates will start with `is`.

In the previous example then, `is minus` is a predicate whereas all others are (as their names suggest) actions. They all have an effect on either the input (reading or recognizing) or output (printing).

2.3.4 Some syntax and semantics

The body of a procedure in CDL has the form of a *group*. Informally and incompletely, its syntax and semantics can be described as follows: — group

A group consists of one or more *alternatives* separated by semicolons. An alternative consists of zero or more *members* separated by commas. An alternative containing zero members is termed an *empty alternative*. Only the last alternative of a group may be empty. — alternative, member

A member can be the (possibly parametrized) call of an algorithm. The *last* member of an alternative can also be an *enclosed group*, i.e. a group enclosed between brackets, or it can be one of the control operators (`+`, `-`, `?` and `*`).

A group is executed by executing its alternatives in textual order until either an alternative succeeds or the last alternative has been executed. The group succeeds if one of its alternatives succeeds, and fails otherwise.

An alternative is executed by executing its members in textual order until either a member fails, in which case the alternative fails, or all its members have been executed successfully, upon which the alternative succeeds. An empty alternative always succeeds.

The execution of a member consisting of a (parametrized) algorithm call is the execution of that algorithm (with those parameters). The execution of a member consisting of an enclosed group is the execution of that group.

Example

As an example, we discuss the body of some interpreter for monadic expressions of the form

$$[+|-]\ number$$

which might be programmed

```
is minus, read number, invert it, print it;
is plus, read number, print it;
is number, print it;
report error
```

Notice that `is number` is a predicate: it tries to recognize a number. On the other hand `read number` is an action (which is programmed somewhere else, probably in terms of `is number`): it will rather die than admit to not finding a number. We will program it later.

Notice also that this group as a whole cannot fail: if neither a leading plus sign nor a leading minus sign nor a number is present, it will report an error, and thus succeed. As such, it will be the body of some action.

2.3.5 Enclosed group

control tree

The fact that the last member of an alternative may be an enclosed group allows the nesting of alternatives in a hierarchical fashion (the *control tree*), e.g.:

```
read number,
   (is plus, read number, add them, print result;
   is minus, read number, subtract them, print result;
   report missing operator)
```

This suggests an interpreter for dyadic expressions of the form

$$number\ \{+|-\}\ number$$

whose control tree is shown in figure 2.2.

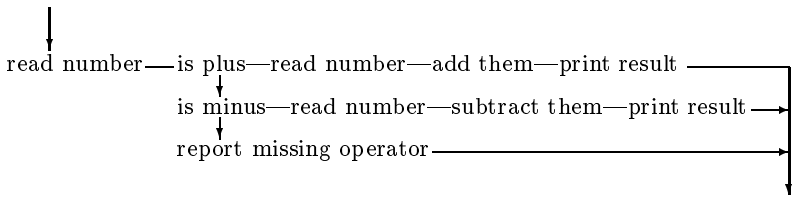


Figure 2.2: Control tree for dyadic expression

After the action `read number` is called, there are three possibilities: either a plus is present, a minus is present, or an error is reported.

Notice that only the *last* member of an alternative can be split in this way, it is not applicable to preceding members, which have to be calls of algorithms. Thus, the sequence "`) ,`" cannot occur in any CDL program.

Since the grouping structure can be used again within the enclosed group, the control tree can be nested to an arbitrary depth. Due to the restriction that splitting can occur only at the end of an alternative, the resulting control tree is still very well comprehensible, especially if suitable layout conventions are observed. The resulting control structure is simple and powerful, and supports the strategy:

> Within an alternative, as soon as you feel there is more than one possibility, write an left bracket "(" indented on a new line, then program the alternatives one by one before closing the group.

If the term "structured programming" has any meaning, it applies to the resulting programming style.

2.3.6 Repeat operator

The last member of an alternative may be a repetition, of the form

```
* [label]
```

The optional label is a name. When the label is missing, the execution of the repeat-operator consists of (a repetition of) the execution of the directly surrounding group, e.g.:

```
read number,
   (is plus, read number, add them, *;
    is minus, read number, subtract them, *;
    print result)
```

which presumably prints the result of a chain formula

$$number\ \{\{+|-\}\ number\}^*$$

Its control tree is depicted in figure 2.3.

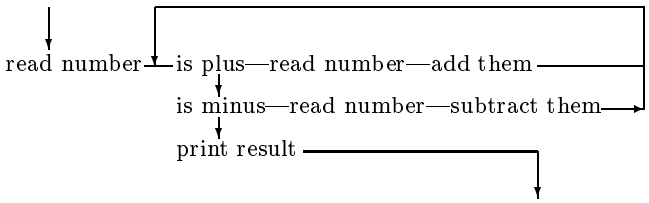


Figure 2.3: Control tree with repetition

When a label is given explicitly, the surrounding group with that label is executed. The body of a procedure is implicitly labelled with the name of that procedure. An enclosed group can be labelled by writing a label followed by a colon just after its open bracket, e.g.:

```
read number,
   (tail:
      is plus, read number, add them, *tail;
      is minus, read number, subtract them, *tail;
      print result)
```

As a final example, we will program a desk calculator (of appreciable complexity but doubtful economic value) in the form of a possible body for a procedure of that name.

It accepts sequences of chain formulas, printing their values, each on a new line, as long as it is not supplied with an end of file.

```
(desk calculator:
   is number,
      (is plus, read number, add them, *;
      is minus, read number, subtract them, *;
      print result, * desk calculator);
   is end of input;
   complain and skip incorrect input, *)
```

Notice that the first two repetitions repeat the directly surrounding group, starting with `is plus`, whereas the last two both repeat the group labeled with `desk calculator`.

2.3.7 Failure operator

The last member of an alternative may be a failure operator, viz. a minus sign. Its execution causes the execution of the procedure in which it occurs to be immediately terminated with failure.

As an example, this operator may be used to fail after reporting an error:

```
is normal case;
report error, -.
```

This operator should be used sparingly and with extreme caution since it disrupts the obvious flow-of-control. It might also lead to unwanted side-effects, but we will see later that these are to a large extent found and reported by the compiler. The explicit break in the flow of control by the failure-operator can in most situations be avoided by a careful structuring of algorithms (and a judicious use of "not" in the name and semantics of predicates and tests). It should be used only where unavoidable.

2.3.8 Success operator

A counterpart to the failure operator is the success operator, which may also occur as the last member of an alternative. It is written as a plus sign.

It is present purely for reasons of symmetry and orthogonality, and is always redundant: if we ever reach it, we are already in a successful mood! Its only sensible use is to make an empty alternative more visible.

2.3.9 Abort operator

The last member of an alternative may be an abort operator, written as a question mark. Its execution results in termination of the program. The abort operator is meant for extricating the program from a hopeless situation, after having suitably reported on that situation and having mended it as far as possible, by returning control to the operating system. The program has no further chance to do anything whatsoever.

This may be the right thing to do in case a more subtle reaction is not possible, e.g. for a compiler that runs out of space.

2.4 Procedure declarations

We will now indicate how to bind a name to a group and equip it with parameters. A procedure declaration consists of a procedure *heading* and a procedure *body*, separated by a colon and followed by a period.

The heading starts with the type and the name of the procedure. The body is a group.

As an example, we can declare a procedure `read number`, assuming the existence of `is number`, as follows

```
ACTION read number:
    is number;
    report error.
```

Notice that `read number` cannot fail provided `report error` is an action. (Whether it does something sensible is another matter.)

Notice that the type of the procedure is not a name, but a magic word, written in capital letters or, for ancient terminals or for operating systems that do not allow both upper and lower case letters, as a word between apostrophes.

We can now embark on the complete definition of a desk calculator recognizing

$$[number\ [\ \{+|-\}\ number]^*\ ]^*\ end\text{-}of\text{-}file$$

We will proceed in a Top-down fashion, first postulating abstract algorithms and then declaring them as we go along. We will describe only the algorithms and assume that the values found are manipulated implicitly e.g. as global objects. We will leave the consideration of objects until the next chapter.

```
ACTION desk calculator:
    is number,
       (is plus, read number, add them, *;
       is minus, read number, subtract them, *;
       print result);
    is end of file;
    complain and skip file, * desk calculator.
```

We assume the availability of suitable predicates for recognizing plus and minus signs, digits and end-of-file, as well as actions for arithmetic and printing.

Bridging the gap, we program:

```
ACTION read number:
    is number;
    report number missing, assume zero.
```

A number is a nonempty sequence of digits *digit digit**, which can be expressed as

```
PREDICATE is number:
    is digit,
       (is digit, *; ).
```

This example is still quite unsatisfactory, because all computations take place as implied side-effects. At this point it has become evident that we must introduce mechanisms for manipulating objects, and for communication between algorithms.

2.5 Communication between algorithms

During the execution of a program, it must be possible to communicate information between algorithms. Generally, programming languages provide three mechanisms for such communication:

- input/output, which is slow, ponderous and ill-defined, suitable only for low levels of interaction (or large quantities)

- global variables, with all their concomitant problems of unwanted access and side-effects
- parameters and local variables, the most explicit and best defined mechanism.

In CDL, parameters and local variables can be associated with an algorithm. Due to the roots of this language in *affix grammars* [KOS71b] these are termed *affixes*. This term is also used for (global) variables, constants and lists, which we will describe in the next chapter. Each affix is denoted by a name.

affix

2.5.1 Call with actual parameters

In calling algorithms, actual parameters can be supplied. The notation

```
alg + par1 + par2
```

means that the algorithm `alg` is called with actual parameters `par1` and `par2`. An actual parameter must be a single affix (there are no composed actual parameters). The plus sign can be read as "with" and serves purely as a syntactic separator. It has nothing whatsoever to do with addition.

2.5.2 Formal parameters

In CDL2, formal parameters must be specified according to their *direction*. Three possibilities are distinguished:

- *input parameters*, e.g. `alg + >inpar:`

  Obviously, `inpar` goes in.

- *output parameters*, e.g. `alg + outpar>:`

  Here, `outpar` goes out.

- *transient parameters*, e.g. `alg + >transpar>:`

  `transpar` goes both in and out.

Input parameters are passed *by-value*, i.e. upon entering the algorithm, each formal parameter obtains a copy of the value of the corresponding actual parameter. In the body, the input parameter may be used as a local variable with no effect on the corresponding actual parameter.

Output parameters are passed *by-result*, they are local variables whose values are copied to the corresponding actual parameters upon leaving the algorithm successfully. Upon failure, they are not copied.

Transient parameters are passed both by-value and by-result, in the sense given above. This parameter mechanism resembles that of ALGOL W, rather than that of PASCAL.

The reasons for preferring call-by-value/result over call-by-reference or call-by-name should be obvious: in an implementation language, the decoupling of calling and called side is desirable to prevent side effects.

Language gourmets may further note that CDL2 avoids the dynamic alias problem by not having composed actual parameters, and that due to various restrictions the static alias problem is decidable. Thus, an implementation of CDL2 may use call-by-reference instead, if the hardware makes this preferable (good facilities for indirect addressing) provided the implementation checks for equivalence with the canonical value/result mechanism).

2.5.3 Local variables

In the heading of a procedure, local variables can be declared after the parameters in a notation using a minus sign. As an example, an algorithm `walk tree` with a local variable `p` can be declared

```
ACTION walk tree - p: ...
```

The heading can be read as "walk tree *without* p". The notations involving the signs plus and minus are quite ancient, dating back to the first Affix Grammars [MEE62]. Completely unimportant things like this notational quirk seem to have an eternal life, whereas more important things come and go.

Many people have, in learning CDL, found the unconventional use of the plus and minus sign to be most unsettling, if not an insurmountable barrier. Habits and conventions are hard to change. For this reason, some CDL software provides another concrete syntax for the language, using the more familiar brackets and comma's for parameters (whereupon the language closely resembles TURBO PROLOG ...). PROLOG

The reader will in time, no doubt, get used to the notation in this book.

2.5.4 Syntax of algorithm-headings

The heading of an algorithm consists of the following elements in order:

- the type of the procedure
- the name of the procedure
- the (optional) formal parameters
- the (optional) local variables.

An example of a heading containing all those elements is:

```
ACTION enter pair + >hd + >tl + ref> - plist
```

2.6 Example: `is number`

Suppose we already have a

```
PREDICATE is digit + d>
```

with the properties:

- if the next symbol of input is indeed a digit
  - the input is advanced by one symbol
  - `d` gets the value of the last digit read, and
  - `is digit` returns true
- if the next symbol of input is not a digit
  - the input is not advanced
  - the value of `d` is not defined, and
  - `is digit` returns false.

We call such an algorithm an *exact recognizer* for digits, because it not only recognizes a digit correctly, but also recognizes a non-digit as such without any effect on the input.

We can use it to build an exact recognizer for numbers, which computes the value of the number as a side-effect while recognizing the number.

```
PREDICATE is number + val> - dig:
   is digit + val,
       (is digit + dig, append digit to val, *;
       +).
```

By itself this is a rather simple piece of program, but it contains one fuzzy spot: how can we append a digit to `val`? We must perform some calculation, but it cannot be expressed in CDL itself, which has no arithmetic. We need yet another mechanism.

2.7 Elementary algorithms: macros

CDL has no concrete algorithms at all: the kernel of the language is empty. On the other hand it has a general facility for borrowing algorithms from some environment, viz. the target language to which the CDL program is to be translated. In this way CDL is *semantically extensible*.

In this chapter we will exemplify this mechanism by borrowing semantic primitives from PASCAL, not because CDL has anything to do with PASCAL but because that language may serve as a lingua franca amongst programmers. In general, macro bodies are formulated in the assembler language of the target machine, or – as a kind of "universal assembler" – in C.

C

An algorithm which is borrowed from the environment, rather than being expressed as a procedure in CDL, is called a *macro*.

macro

Every algorithm is either a procedure or a macro. The word "algorithm" will be used whenever the distinction between procedures and macros is not relevant.

2.7.1 Macro declarations and calls

A macro heading looks exactly like a procedure heading. In particular, a macro is allowed to have local variables. A macro can be recognized by the equals sign which serves as a separator between the heading and the *macro body*. An example is

```
ACTION one more digit + >val> + >dig =
   val ":=" 10 "*" val "+" dig.
```

A macro body consists of one or more of the following *macro elements*:

- texts, enclosed between quotes
- decimal numbers
- affixes
- layout

A macro call looks exactly like a procedure call.

2.7.2 The meaning of a macro

The CDL compiler exists in many versions, for different target languages and operating systems. The occurrence of some macro call at a specific place in a CDL source program results, after translation to the target language, say $\mathcal{L}$, in an *expansion* of that macro at the corresponding place in the target program. The macro body therefore must be such that its expansion has the required meaning in the context of $\mathcal{L}$. It must be written in $\mathcal{L}$.

The expansion of a call of some macro consists of the transliteration of each of its elements in order. The transliteration of macro elements is as follows: transliteration

- a *text* which is enclosed in quotes is copied literally, but some *escape sequences* have a special meaning: escape sequence

  `$l` is transliterated as a new line

  `$t` is transliterated as a tabulator spacing

  `$"` is transliterated as a quote sign

  `$$` is transliterated as a dollar sign.

  Blanks occurring in a text are copied literally, outside of a text they are ignored. A text is not allowed to extend over one line (to reduce problems with forgotten quotes).

- *decimal numbers* are transliterated to their semantic equivalent in the target language, be that a decimal number or some disgusting hexadecimal blurp.

- *affixes* are transliterated to their semantic equivalent in the target language, which may look like the same name or like something quite different (e.g., some construction involving an offset and an index register). In this process, actual parameters are substituted for their corresponding formal parameters.

- *layout* is not transliterated but ignored.

As an example, a possible transliteration to PASCAL of a call

```
..., one more digit + value + digit, ...
```

might be

$$value := 10 * value + digit$$

It should be emphasized that the meanings of macro bodies are wholly outside the understanding of the CDL compiler.

For each version of the compiler, the manual details the transliteration scheme used. Care has been taken in every instance that the scheme used is as helpful, understandable and efficient as the target language allows.

Different versions of the compiler for one same target language may make use of different transliterations, as detailed in the manuals.

2.7.3 Application: `is number`

Using the primitive action `one more digit`, we can now program

```
PREDICATE is number + val> - dig:
    is digit + val,
       (is digit + dig, one more digit + val + dig, *;
       +).
```

A possible transcription into PASCAL (which could obviously be obtained mechanically) is

```
function is number(val : integer) boolean :
  begin integer dig;
    if is digit(val)
    then
      begin
        while is digit(dig)
        do val := 10 * val + dig;
        is number := true
      end
    else is number := false
  end
```

This example should clarify the notion of transliteration and substitution. An equivalent transliteration into a low-level language, like Assembler for IBM computers should be possible, provided the macro body for `one more digit` is rewritten accordingly.

2.7.4 Primitive tests

In this fashion we can also introduce primitive *tests*, i.e. Boolean procedures without side-effects, serving to control the execution. Obvious examples are

```
TEST less  + >x + >y = x "<" y.
TEST equal + >x + >y = x "=" y.
TEST lseq  + >x + >y = x "<=" y.
```

We have here tacitly assumed that the values of affixes are integers, which is a good working hypothesis for the time being.

As it turns out, a surprisingly small collection of primitive actions and tests is sufficient to comfortably express arithmetic algorithms. Although CDL programmers have an unprecedented freedom in their choice of primitives, most programs use (a subset of) the macros given as an appendix in every compiler manual. Creative phantasy is possible but rare.

2.8 Classification of algorithms revisited

An algorithm, whether defined as a procedure or as a macro, has two (independent) attributes

- whether it returns a *value* (does it always succeed, or may it also fail)
- whether it has a global *effect*

By a global effect is meant "a change to the observable universe upon returning true". The philosophy of effects will later be described more precisely. For the moment we will resort to the reader's intuition, defining a global effect as: "a side effect otherwise than through a parameter". We will in the classification of algorithms ignore any effects through parameters.

Crossing these two attributes leads to four types of algorithms:

specification	returns a value	has a global effect
`TEST`	yes	no
`PREDICATE`	yes	yes
`ACTION`	no	yes
`FUNCTION`	no	no

Let's give some examples of abstract algorithms of each type.

`TEST equal + >x + >y` tests for numerical equality.

`TEST is small letter + >x` determines whether the value of `x` is the code of a letter in lower case (e.g. in ASCII: between 97 and 122).

`PREDICATE is number + val>` an exact recognizer for numbers, computing their value. The effect is on the input.

`PREDICATE can pop + x>` returns false if the global stack is empty; otherwise pops an element, assigning its value to `x` and returns true. The effect is on the global stack.

`ACTION push + >s + >x` aborts if the stack `s` is already full; otherwise, pushes `x` onto `s`.

`ACTION print int + >x + >width` prints `x` in `width` positions. The effect is on the output file.

The fourth type, `FUNCTION`, may at first seem superfluous: no value returned, no global effect. One possible application is to achieve effects that the programmer deems not to form part of the process itself, e.g. tracing

`FUNCTION trace + >x` the current value of `x` is printed on a trace file.

The major application is, however, to achieve effects solely through the parameters, as is usually the case in computations:

`FUNCTION one more digit + >val> + dig>` as defined above, where it was in fact specified incorrectly

`FUNCTION add + >a + >b + c>` an addition in three-address style

`FUNCTION make + y> + >x` assign `x` to `y`.

The programmer has some degree of freedom in specifying, but is well advised to specify as exactly as possible: The specifications form the basis for a *semantic check* on side effects, checking what the programmer says he wants to do against what he actually does [FEU78]. More about this in a chapter 9 on static semantic checks.

2.9 Example: Quicksort

As an example of somewhat higher complexity, we will look at an implementation of Quicksort. We assume the algorithm to be well known to the reader. We will not bother to represent the array which is to be sorted, but assume access to it through the macros

```
TEST smaller element + >p + >q
```

asking whether the element with index `p` is smaller than that with index `q`, according to the intended ordering, and

```
ACTION swap element + >p + >q
```

which swaps the values of the elements with index `p` and `q`.

Furthermore, we assume the following arithmetic functions and tests, whose names and parameters should be selfexplanatory:

```
FUNCTION add + >a +>b +c>
FUNCTION incr + >x>
FUNCTION decr + >x>
FUNCTION halve + >x>
FUNCTION make +x> +>y
TEST less +>x +>y
```

Using these algorithms, Quicksort can be implemented as:

```
ACTION quicksort + >from + >to - p - q:
    less + from + to, split + from + to + p + q,
       quicksort + from + q, quicksort + p + to;
    +.

ACTION split + >i + >j + p> + q> - m:
    make + p + i, make + q + j, add + i + j + m, halve + m,
      (again: move p up + j + p + m, move q down + i + q + m,
         (less + p + q, swap element + p + q,
            incr + p, decr + q, * again;
          less + p + m, swap element + p + m, incr + p;
          less + m + q, swap element + q + m, decr + q;
          +)).

FUNCTION move p up + >j + >p> + >m:
    less + j + p;
    smaller element + m + p;
    incr + p, *.

FUNCTION move q down + >i + >q> + >m:
    less + q + i;
    smaller element + q + m;
    decr + q, *.
```

This example should show the power of the unconventional control structures of CDL.

2.10 Exercises

In learning to program algorithms in-the-small it is helpful to make, at this point, the following exercises.

1. Write three versions of an action `fill the pot`, which serves to fill a pot with marbles. The pot can hold some specific number of marbles, at least one. There is an adequate supply of marbles.

   For each version, a different structure is suggested in the form of a little program in PASCAL. Make use only of the elementary algorithms appearing in each respective program:

   - **while** *pot not full* **do** *add a marble*
   - **while not** *pot full* **do** *add a marble*
   - **do** *add a marble* **until** *pot full*

2. Write an action `follow the circle` that steers a plotter to follow from the inside the first quadrant of a circle on the unit grid of a given radius. Try to proceed Top-Down, introducing (auxiliary) abstract algorithms as you go along.

2.11 Transput

transput

The term *transput* is used here to cover both input and output.

CDL has (as you have guessed) no concrete elementary transput algorithms, and the primitives for abstract ones will have to be borrowed from the target environment through macros.

For an implementation language, which is supposed to make minimal assumptions about its environment and may well be used to define transput itself, this is an acceptable situation.

There is, however, a kind of de-facto standard: on every computer where a CDL2 compiler is running, the transput system of the compiler is made available to the users. It is of course heavily oriented towards its particular task of supporting the CDL2 compiler, but may serve for other applications. In particular, because of its wide availability, it may serve as a portable transput interface.

It typically contains

```
ACTION open + >channel
ACTION close + >channel
```

The parameter `channel` should be a small integer, which in a machine-depen dent way is associated with a specific file or device (as in FORTRAN). Typically, channel zero is for input and channel five for output.

```
ACTION outchar + >channel + >char
ACTION inchar  + >channel + >char
```

Characterwise input and output are buffered. The character code is ASCII.

```
ACTION outintfo + >channel + >int + >width
```

outputs an integer, formatted to the `width` given.

```
ACTION outtext + >channel *string
```

serves to output a string.

```
ACTION newline + >channel
```

It causes the current buffer to be written (for output channels) or the next buffer to be read (for input channels).

```
ACTION newpage + >channel
```

is useful for printed output; on other channels, it is equivalent with `newline`.

```
ACTION console + >channel
ACTION printer + >channel
```

serve to dynamically switch a channel to the terminal and the printer, respectively.

Details are given in every CDL2 compiler manual.

2.12 Strings

A widespread activity in compilers and other programs is the writing of messages, e.g. to report errors.

With the language means given, the necessary string handling would have to be laboriously programmed. In particular the denotation of strings would be cumbersome or impossible.

On the other hand, nearly all environments to which CDL compilers exist are high-level enough to support some limited form of string-denotation and string-handling, albeit in the form of Hollerith constants.

Under pressure from CDL1 users, we were forced to include in CDL2 a syntax and semantics for string handling, as follows:

- There is a fourth form of parameters passing, called "substitution parameters", for passing strings.
- A formal substitution parameter `msg` is indicated by a star, e.g.

  ```
  ACTION report *msg:
  ```

- An actual substitution parameter is allowed to be either a text, enclosed between quotes, or again a formal substitution parameter, e.g. in the call

  ```
  report + "stack full"
  ```

We will see that there exist no string variables and constants. A string can be denoted and passed through any number of calls, but within the language nothing else can be done with it. Luckily the standard transput contains a primitive

```
ACTION outtext + >chnl *msg
```

which prints the text of `msg` on the channel `chnl`.

This is quite similar to the string handling in ALGOL 60, from which it is in fact derived.

This facility is limited but useful enough. Of course it forms an extremely ugly balcony attached in an adventurous fashion to the clean orthogonal walls of the rest of the language. Such is life.

Chapter 3

Programming in-the-small: objects

Programming in-the-small in CDL seems to differ most from that in other languages with respect to its treatment of objects. In this chapter we will discuss the syntax and semantics for declaring and manipulating objects, as well as the underlying philosophy.

3.1 Data types

There are two data types in CDL, viz. data

- words word
- linear arrays of words

Every value is conceptually a (non-interpreted) machine word. Any assumptions about the type are in the programmers hand. CDL is essentially a typeless language. Neither the CDL compiler nor the run-time system distinguishes types, and there is no facility for the user to declare abstract data types. Sorry for this (apparent) move against the historical trend.

For objects, only their *declaration* is provided, all operations have to be borrowed through macros.

Nothing is assumed about the word-size. Individual CDL implementations may well have differing word-sizes, as provided by the target hardware. Of course specific CDL programs may demand a minimal word-size for e.g. the storing of addresses and indices. This is, however, a property of the program, that should be stipulated by its writer. A reasonable rule-of-thumb is to require that on any machine the word-size is enough to hold an address — although on a byte oriented microcomputer even that might not always be the case!

3.2 Global variables

A single one-word variable can be declared

```
VAR buffer.
```

and similarly for a list of variables

```
VAR char, pbuffer, count.
```

Such declarations may be mixed freely with procedure declarations, and the order of declarations plays no role whatsoever. It is advisable, however, to carefully choose a conspicuous and logical place in the program for each variable declaration.

3.3 Constants

A named constant is declared in a way which is strongly related to macro declarations, e.g.

```
CONST zero = 0,
    new line code = 5,
    bytes per word = 6,
    length = total "DIV" bytes per word.
```

The parts following the equal signs are in fact sequences of macro elements. The most obvious choice for such a constant body is a (decimal) number, which will be translated to its semantic equivalent in the target environment, probably some hexadecimal obscenity. Alternatively, the CDL programmer could himself write down some formulation in the target language, enclosed between quotes. It should be stressed that the expanded constant bodies are interpreted in the target language and are subject to the conventions and restrictions of that language. Thus, in an Assembler target environment, the body of the constant `length` will be subject to the rules and possibilities of assembly-time arithmetic, i.e. address calculation.

3.4 Example: `is digit`

As an example, we can now define the predicate `is digit`. We assume the existence of an action `nextchar` that reads one character, assigning it to its only parameter. We furthermore assume that the character encoding has sensible properties, e.g. that the codes for the digits have consecutive increasing numeric values. Let those values be from `162` to `171`.

```
VAR char. # next nonconsumed input character #

TEST can be digit + >code =
    "(" code ">=162) and (171>=" code ")".

FUNCTION convert to digit + >code + dig> =
    dig ":=" code "-162".

PREDICATE is digit + d>:
    can be digit + char, convert to digit + char + d,
       nextchar + char.
```

comment

Notice the *comment* on the first line, enclosed between hash signs. Such a comment may not extend over one line (in order to prevent synchronization problems when the closing mark has been inadvertently omitted).

This piece of program realises a small reading administration, suitable for a simple lexical analyzer. The important aspect is the global variable `char` which buffers one character in advance. Every time a character has been recognized, the next one is read in. This strategy is far superior to an unbuffered version where `is digit` starts by reading a character and may find to its horrification that it is not a digit and has an unwanted character on its hands.

One should further notice that this piece of program was (on purpose) not written in the best style: what happens when we want to adapt it to another target environment with a different code? We have to change the codes `162` and `171` throughout the program into other values. This is a nuisance and (with a conventional string-oriented editor) rather error prone.

It would have been better to introduce two named constants

```
CONST zero code = 162, nine code = 171.
```

and to rewrite

```
TEST can be digit + >code =
    "(" code ">=" zero code ") and (" nine code " >=" code ")".

FUNCTION convert to digit + >code + dig> =
    dig ":=" code "-" zero code.
```

This looks better and is also adapted more easily to different character encodings. The first version was unnecessarily concrete — a mortal sin in programming.

Because of the assumptions made about the codes for digits, we may further simplify this using standard arithmetic macros to

```
TEST can be digit + >code:
    lseq + zero code + code, lseq + code + nine code.

FUNCTION convert to digit + >code + digit>:
    sub + code + zero code + digit.
```

3.5 Lists

A linear array of elements, with fixed bounds, can be declared

```
CONST minbuff = 1, maxbuff = 4095.
LIST buffer (minbuff:maxbuff).
```

The semantics of such a list-declaration depends on the particular CDL-implementation. A memory area is grabbed, whose size is determined by the bounds, and made accessible. The elements might be bytes, half-words or full-words as stated in the manual accompanying that implementation.

CDL *provides no list access.* Again it has to be borrowed. As an example, in the PASCAL version we may well write

```
ACTION put to buff + >i + >x =
    buffer "[" i "]:="  x.

FUNCTION get from buff + >i + x> =
    x ":=" buffer "[" i "]".
```

providing both reading and writing access to the list `buffer`. Notice that writing is an action, whereas reading is a function — the two are not symmetrical. Now we can write

```
get from buff + pbuff + char
```

which is transliterated to

$$char := buffer[pbuff]$$

Now only the codes for zero and nine are machine dependent. In many circumstances, a large part of the machine dependence of a program can be expressed in a few strategic constants.

3.6 Access philosophy

Lists are themselves not data structures, but the clay out of which data structures are built.

A data structure is a coherent set of *access algorithms*, operating on a *representation* in terms of lists and variables. representation

Upon the clay, layers of access algorithms are successively built at increasing levels of abstraction, security and comfort.

This is an essentially Bottom-Up view of the data world, which contrasts with the extremely Top-Down view taken at the algorithmic side. The two viewpoints are complementary.

Large data structures, of which in general only one instance exists (such as the symbol table in a compiler) are made visible only as a set of access algorithms. Objects of which more than one instance exists are either small (in the sense that they can conveniently fit into one word) or they are components of a larger data structure and access to them may well be provided through a small object (e.g. a pointer into a table).

Take stacks, as an example: preferably, the number of stacks used in a program should be at most two (for obvious engineering reasons). If we are in a situation where we have to deal with more than two stacks, it is just as much work to implement n stacks as it is to implement three: we may build an administration suitable for dynamically creating and manipulating any desired number of stacks, or even a stack of stacks, involving quite some hidden memory management. Each individual stack we will access through a pointer (either to its top or to some control block). Again the interface to such a data structure involves only access algorithms and small objects, which will conveniently fit within an affix.

In the sequel we will give a number of examples to elucidate this philosophy of data structures and its consequences.

3.7 Example: two-dimensional array

array

In this example we will try to make clear that a list is a different kind of animal than an array, even if its realization in most high-level languages probably involves an array.

Consider the declaration of and access to a two-dimensional array of integers with the fixed upper-bounds n and m.

In a high-level language these can be expressed very easily:

var a : **array** $[1..m, 1..n]$ **of** *integer*
$\ldots$ $a[i, j]$ $\ldots$

In CDL2 the introduction of such an array involves much more work for the programmer:

Step 1 Choose the desired access algorithms for the array

```
FUNCTION get array sub + >i + >j + x>
ACTION put array sub + >i + >j + >x
TEST is within array + >i + >j
```

Step 2 Start with the clay

(We assume constants `n` and `m` to have been previously defined)

```
CONST min arr = n "+" 1,
      max arr = n "*" m "+" n.
LIST storage (min arr:max arr).
```

A flattened version of the array is represented by the list `storage`. Notice that the bounds have been chosen so as to minimize index computations.

Step 3 Zero-level access

```
FUNCTION get + >k + x> =
    x ":=" storage "[" k "]".


ACTION  put + >k + >x =
    storage "[" k "]:=" x.
```

These zero-level accesses are insecure, not to be given to the user. Notice the asymmetry `FUNCTION/ACTION`.

Step 4 first-level access

```
TEST is within array + >i + >j:
    lseq + one + i, lseq + i + m,
    lseq + one + j, lseq + j + n.


FUNCTION get array sub + >i + >j + x> - k:
    is within array + i + j, addmult + i + n + j + k,
       get + k + x;
    report reading out of bounds, make + x + zero.
```

Notice that even in case of an error, a value for `x` must be computed.

```
FUNCTION addmult + >a + >x + >b + res> =
    res ":=" a "*" x "+" b.


ACTION put array sub + >i + >j + >x - k:
    is within array + i + j, addmult + i + n + j + k,
       put + k + x;
    report writing out of bounds.
```

Decidedly this is more work than in PASCAL — but also we have no need for a PASCAL translator and run-time system. The CDL2 version is efficiently expressible in most target languages, thus assuring portability.

The explicit thinking about data structures which CDL forces on the programmer is not only beneficial in teaching the fundaments of data structures, but also aids in adapting the data structure to the application. A two-dimensional array, after all, is an abstract object, a matrix, an instance of a type that happens to have been entombed by historical accident in conventional programming languages. It is certainly not a bad decision to have matrices as concrete objects in a language intended for numerical computations, but it is a nuisance to have to make do with a matrix in order to realise a symbol table.

Removing all concrete data structures from the programmers hands may serve very well to clear the cobwebs from his mind. In practice, systems programmers find themselves designing and realizing all kinds of specialized data structures for which it would have been useless to foresee concrete data structures in the language. Of course, a number of motifs such as subscription, sequencing and linking reappear continuously, but it is no great hardship to express those explicitly, once experience has been acquired.

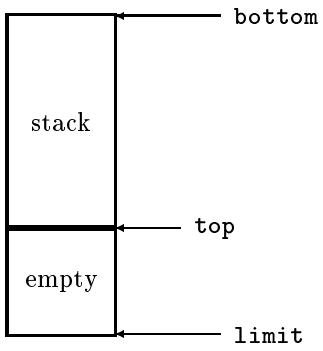


Figure 3.1: A stack

3.8 Example: stack

stack

Anybody discussing abstract data types always chooses the stack for an example. We will make no exception to this rule. In designing a representation for a data structure, the drawing of a picture may be of help. We will depict a representation of a stack as a sequence of consecutive elements (figure 3.1). The picture is intended to convey some information, but it leaves two degrees of freedom:

- The constant `bottom` appropriately points to the first word of the stack, and `limit` to the last word available for the stack. Due to an unfortunate aspect of our writing conventions, in informatics the bottom is usually at the top, and vice versa. Therefore the picture does not disclose whether the value of `bottom` is actually smaller or larger than that of `limit` (i.e., whether the stack is a stalagmite or a stalactite). We will have to make a choice, and abide by it.
- The variable `top` can be used to point either at the first free element or at the last filled element, but not both.

We will implement a stack, delaying our choices as long as possible.

Step 1 choosing the access interface

```
ACTION initialize stack
ACTION push + >el
ACTION pop + el>
TEST  is full
TEST  is empty
```

Step 2 clay

```
CONST bottom = 1, limit = 2047 # or some such #.
LIST work (bottom:limit).
VAR  top.
```

The stack will be a stalagmite, because the bottom is lower than the limit.

Step 3 zero-level accesses

```
FUNCTION get + x> + >i =
    x ":=" work "[" i "]".
```

```
ACTION  put + >x + >i =
    work "[" i "]:=" x.
```

These accesses are to be used only to implement others.

Step 4 first level accesses

We choose to let `top` point to the first free location.

```
ACTION initialize stack:
    make + top + bottom.

ACTION push + >el:
    is full, shout to high heaven, ?;
    put + el + top, incr + top.

ACTION pop + el>:
    is empty, shout to low hell, ?;
    decr + top, get + el + top.

TEST is full:
    less + limit + top.

TEST is empty:
    equal + bottom + top.
```

Since we would not like the clay and the zero-level access to fall into the hands of the user of the stack, some form of encapsulation of this data structure is necessary. We will deal with this issue in chapter 4 on programming-in-the-large.

Now suppose we do not have just one stack but must be able to deal with any number of stacks. We may choose to allocate to each stack a fixed-size part of a large list, or alternatively use a linked representation where the stacks have no individual restrictions on their size but where there would be a collective upper limit.

In both cases, the access interface could look like

```
ACTION create new stack + >size + p>
ACTION push + >p + >el
ACTION pop + >p + el>
TEST is full + >p
TEST is empty + >p
```

The value of `p` could be a pointer to a control block, containing the length of the associated stack and its base address. Programming this generalization is left as an exercise to the reader.

3.9 Example: a singly-linked list

As a non-trivial example we will implement a singly-linked list serving for some spare-parts administration. Each spare part will have a name and a type.

In PASCAL, this object could be declared as a chain of elements, as follows:

```
type
  refdlist = ↑ dlist;
  dlist =    record
               name : string;
```

```
            type : integer;
            next : refdlist
        end;
var partslist : refdlist
```

In any environment where the identifier *partslist* is visible, the following accesses are automatically possible:

partslist.↑*type*
partslist.↑*name*
partslist.↑*next*

and through this last access we can follow the whole list. A problem lies in the fact that these accesses cannot be denied in any way: any idiot can write

partslist.↑*next* := **nil**

thus destroying the list, or even

partslist.↑*next* := *partslist*

thus tying it in knots.

Anybody who has access to the list can do anything to it he damn pleases and there is no way to grant e.g. only reading access. Passive security (keeping the field selectors secret, choosing unguessable names) is (as life with OS 360 shows) not enough: for active security it must be possible to selectively make some or all of these accesses inoperative. Therefore they must not be something special, like a field selector, but an ordinary named algorithm (e.g. a procedure). Only entities that have mentionable names can be hidden.

For the CDL2 version we will start with a picture of one list element as part of a storage area holding the complete list, pointed at by a pointer `p` (see figure 3.2)

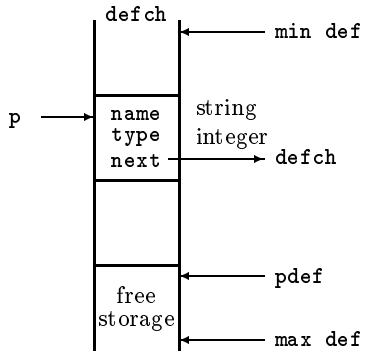


Figure 3.2: Linked list storage

quasi-stack

A list element has three fields, each depicted with its type. The third is again a pointer into `defch`. The list will probably be administered as a quasi-stack: it only grows, until everything is discarded at once (there is no pop-operation). Its stack pointer will be `pdef`. There can be a large number of lists in the storage area `defch`, each pointed at by its own pointer. We use a special pointer value to denote an empty list.

Step 1 accesses

The access algorithms

```
FUNCTION new list + p>
TEST is null list + >p
```

must be such that `new list + x, is null list + x` always succeeds. The action

```
ACTION add to list + >p> + >addr + >name
```

appends an element at the *head* of the list pointed to by `p`. This is not only much simpler than appending it at the end, but in many situations it happens to be just what we want, e.g. when we must inspect the elements in reverse historic order. Notice that `p` is a transient parameter.

Step 2 clay

```
CONST min def = 1, max def = 2047,
    null = min def "-" 1.
LIST defch (min def:max def).
VAR  pdef.
```

Step 3 zero-level access

A list is to be accessed through a pointer to its first word. We choose, initially, to pack one field per word. We must demand from the target architecture that a word is large enough to hold an index into `defch`, an integer or the address of a string.

```
CONST element size = 3.

FUNCTION get name + >p + n> =
    n ":= defch "[" p "]".

ACTION put name + >p + >n =
    defch "[" p "]:=" n.
FUNCTION get type + >p + t> =
    t ":=" defch "[" p "+1]".
    ...
ACTION  put next + >p + >n =
    defch "[" p "+2]:=" n.
```

(three function/action pairs in all).

Step 4 first-level access

```
FUNCTION new list + p> :
    make + p + null.

TEST is null list + >p :
    equal + p + null.
```

Before inserting an element into the list pointed to by `p`, we have the situation shown in figure 3.3. After insertion we must have the situation of figure 3.4.

We can program this:

```
ACTION add to list + >p> + >nm + >tp - new p:
    add + pdef + element size + new p,
       (less + max def + new p, shout to high heaven;
       put name + pdef + nm,
          put type + pdef + tp, put next + pdef + p,
             make + p + pdef, make + pdef + new p).
```

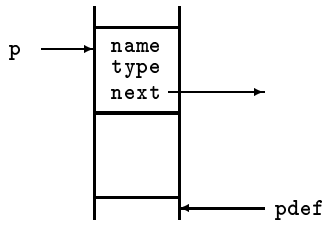


Figure 3.3: Linked list before insertion

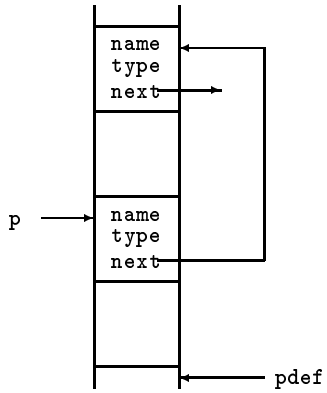


Figure 3.4: Linked list after insertion

A somewhat cumbersome piece of pointerchasing, but not too difficult to invent.

At this point we can start thinking about the storage efficiency (packing) and the portability of this realization. Ideally, we should have to change only the zero-level accesses in order to deal with these issues.

On a byte-addressed microcomputer, we may choose some suitable multiple of a byte for each field. The number may well cost two bytes; the name-field would probably hold a pointer into a text administration and therefore take two bytes, just like the next-field. We could therefore choose `element size` to be six, change only the level-zero accesses and use a crosstranslator to the micro.

Can this program be ported onto, say, a 60-bit machine? Choosing the constant `element size` to be one and performing some hideous masking and shifting in the access macros would get us there.

Programming in the style given in these examples does give a good degree of portability. But following this style all too slavishly can cost efficiency. Once the program is running correctly, it is possible to regain some efficiency by adding "larger" primitives. In the present example, we note that the three fields of an element are always written together, so that on many machines it is preferable to write them all at once rather than singly. This suggests the introduction of a zero-level primitive

```
ACTION put elem + >p + >nm + >tp + >nx
```

which can be used in

```
ACTION add to list + >p> + >nm + >tp - new p:
    add + pdef + element size + new p,
```

```
(less + max def + new p, shout to high heaven;
put elem + pdef + nm + tp + p,
   make + p + pdef, make + pdef + new p).
```

This version is both more perspicaceous and more efficient on 60-bit machines (less shifting and masking) as well as on byte-addressed microcomputers (avoiding repeated index calculations).

3.10 A binary tree

We will now implement a symbol table for some compiler in the form of a binary tree. The tree consists of cells, one of which is the root. Each cell has a left and a right-pointer, which is either nil or points to another cell.

Each cell contains a part with a fixed size (containing pointers and attributes of the symbol) followed by a variable-size part of one or more words, the respresentation of the symbol. Into each word of the representation are packed a number of characters (bytes, if you wish). The last word of the representation may have to be padded out with nullchars. The cells are stored in a quasi-stack `text` with stackpointer `ptext`. It can in fact hold any number of binary trees, each accessible through a pointer to its root element. One text cell has the structure shown in figure 3.5.

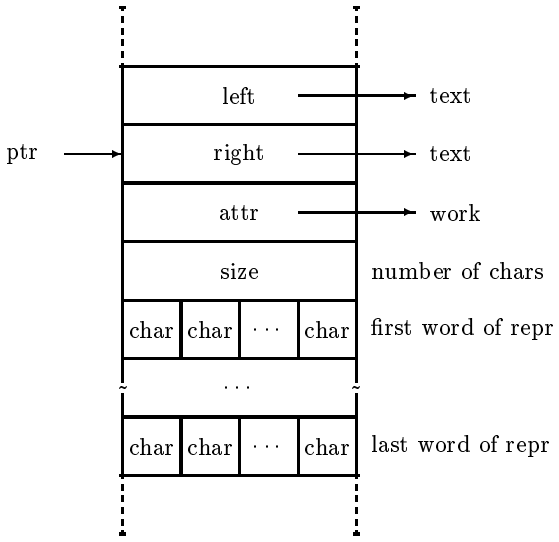


Figure 3.5: Text cell

We will first program the low-level accesses:

```
CONST nil=0, min text=1, max text=5000, cell size=4.
LIST text(min text:max text).
VAR ptext.
```

The access macros are expressed this time in VAX-assembler:

```
FUNCTION get from text + >ptr + >offs + left> =
    "$L;regs:p1=r,p2=r"
    "$L  movl " text "[" ptr "](" offs ")," left .
```

```
ACTION put to text + >ptr + >offs + >left =
    "$L;regs:p1=r,p2=r"
    "$L  movl " left "," text "[" ptr "](" offs ")".
```

The procedure `put to text` writes a word into the list `text` at the index `ptr` — counted in words — incremented by `offs` bytes. The offsets in bytes for the elements of each cell are defined like:

```
CONST left offs = 0, right offs = 4, attr offs = 8,
      size offs = 12, repr offs = 16.  # offset in bytes !#
```

Splitting even the zero-level access further, access procedures for the cell elements now can be written in terms of the two macros `put to text` and `get from text` and the offsets:

```
FUNCTION get left + >ptr + left> :
      get from text + ptr + left offs + left .
ACTION put left + >ptr + >left :
      put to text + ptr + left offs + left .
      ...
FUNCTION get repr + >ptr + repr> =
      get from text + ptr + repr offs + repr.
ACTION put repr + >ptr + >repr =
      put to text + ptr + repr offs + repr.
```

Obviously, such repetitive prose should be written using a suitable text-editor.

```
ACTION initialize texttree:
    make + ptext + min text.
```

Symbols are put into the text table by some other part of the program, the lexical analyzer. It recognizes a symbol and stores their constituent characters one by one into the text table. We give it the interface

```
ACTION start text
ACTION add to text + >char
ACTION finish text
```

Before the first character of the symbol is stored, the lexical analyzer should call `start text` once. Then it can call `add to text` one or more times with the successive characters of the symbol and finally `finish text` is to be called once. The new text entry is built up at the top of the stack for later incorporation into a tree. The technique employed for packing characters into a word is explained in section 5.7 of this book.

```
VAR new, size, buffer, buffer count.

ACTION start text:
    make + new + ptext, make zero + size,
       add to + ptext + cell size,
          (less + ptext + max text, put left + new + zero,
             put right + new + zero, put attr + new + zero,
                make zero + buffer, make zero + buffer count;
          system error + "Text table overflow", ?).

ACTION add to text + >char:
    incr + size,
       (equal + buffer count + bytes per word, stack textword,
          make + buffer + char, make + buffer count + one;
       append byte + buffer + char, incr + buffer count).
```

```
ACTION finish text:
    equal + buffer count + bytes per word,
      stack textword, put size + new + size;
    add to text + null char, *.

ACTION stack textword:
    lseq + ptext + max text, put repr + ptext + buffer,
      incr + ptext;
    system error + "Text table overflow", ?.
```

When the symbol has been completely stored, it must be entered in the right place in the binary tree. We provide an

```
ACTION enter text + >p> + key>
```

which takes a pointer to the root of a binary tree, and returns a pointer to a cell `key` with the property that equal entries get equal keys. One same symbol is stored only once.

```
ACTION enter text + >p> + key> - old - prev:
    equal + p + nil, make + p + new, make + key + new;
    make + old + p,
       (try next entry: make + prev + old,
          (lex before + new + old, get left + old + old,
             (equal + old + nil, put left + prev + new,
                make + key + new;
              * try next entry);
           lex before + old + new, get right + old + old,
             (equal + old + nil, put right + prev + new,
                make + key + new;
              * try next entry);
           make + ptext + new, make + key + old)).

TEST lex before + >p1 + >p2 - w1 - w2 - s1 - s2:
    get size + p1 + s1, get size + p2 + s2,
      (next word: get repr + p1 + w1, get repr + p2 + w2,
         (less + w1 + w2;
          equal + w1 + w2, decr + s1, decr + s2,
             (is zero + s2, -;
             is zero + s1, +;
             incr + p1, incr + p2, * next word))).
```

Finally we provide an algorithm that, given a key, prints the corresponding symbol.

```
ACTION write text + >t - s - w:
    get size + t + s,
       (is zero + s;
       get repr + t + w, write word + w,
          incr + t, decr + s, *).

ACTION write word + >w - c:
    is zero + w;
    isolate byte + w + c, write word + w,
       (equal + c + null char;
        put char + c).
```

We should not forget to initialize the text tree when using this data structure, by a call `make + ptext + min text`.

3.11 Discussion

Why are data structures in CDL not richer?

The fact that CDL has no data structures does not come from a lack of understanding of the language and implementation issues involved. Since SIMULA and ALGOL 68, computer scientists know what data structures are.

CDL intentionally starts at the lowest possible level, viz. the possibility to declare "clay", and the mechanisms for semantic extension and abstraction on the algorithmic side.

Some arguments for this position:

- The language mechanisms given are *sufficient* to express any data structure
- The CDL solution is the *most general* one that could be taken, since it does not involve any preconcieved concrete data structures.

  (A quote from A. van Wijngaarden, the father of ALGOL 68: "We can be silent in full generality".)
- CDL programs make *minimal assumptions* about the environment in which they run. Thus, CDL (cross)compilers can work to quite hostile target environments.
- *Portability* of data structures is achieved through portability of access algorithms. Thus, only one kind of problem has to be solved. As a consequence, both the language and programs written in it are portable to an unusual degree.

Some further arguments pertain to the access philosophy given:

- Accesses and calls on macro's and procedures look exactly the same. This *uniform referent* property [ROS70] enhances adaptability. It looks like a small pedantic point, but has important practical consequences.
- The fact that access is only through named algorithms makes possible any desired degree of *security*, through a general mechanism for restricting the visibility of names (see the next chapter).

Summing up, in CDL a data structure is an access discipline plus a controlled mapping of objects onto data representations.

The relation between objects and their representation is one of difference in degree of abstraction. Rather than the usual dichotomy between a *defining environment* (where everything is possible) and an *applying environment* (where only safe things are possible) in CDL there is a tendency to distinguish a threefold hierarchy

- representation (clay and 0-level access)
- function (1-level access)
- application

This tripartition will recur in the structure-in-the-large of CDL2 programs.

Chapter 4

Programming in-the-large

In this chapter we introduce a conceptual model and some terminology for dealing with the structure-in-the-large of programs, as well as the corresponding language mechanisms in CDL2. We will discuss the notions of abstraction and construction, visibility, abstract machines and interfaces.

4.1 Abstraction

The word *abstraction* has been bandied about in Informatics for a number of years. Like many other terms, its popularity seems to be inversely proportional to the precision of its use. Webster refers us to the verb "to abstract" for which it gives a number of meanings, the most important being

abstraction

> "to take away from"
> "to consider apart from any particular instance".

These definitions, though etymologically impeccable, are not greatly helpful in understanding the various technical uses of the word "abstraction" in Informatics, and such terms as

> "abstract data types"
> "abstract machines"
> "abstract syntax"
> "levels of abstraction"

Many different views regarding abstraction can be distinguished. Somewhat exaggerating their differences, these include in particular:

- The *algebraic* notion of abstraction: "It is abstract if it is mathematical".
- The *technological* viewpoint: "Abstractions are what you make by means of (some variant of) the class concept".
- In contrast to these we would like to stress that abstraction is *conceptual*: abstraction is an aspect of the programmers thinking.

4.1.1 The programmer thinks!

Programming is a mental activity. The program is not only the result of the programmers thinking, but also a record of his or her thoughts. This can be made plausible by introspection, by observing yourself thinking aloud while programming, but also by inspecting the work of others. If you happen to read somebody else's program, it will become immediately clear that its author must have been thinking something: otherwise he could never have written down such strange formulations.

As an example, let us consider some fragment from a (hypothetical) compiler written in an ALGOL language. Suppose we observe a program fragment like

if $fp[i, 2] = 16$ **then** ...

It is obvious that the author must have been thinking something, because the fragment cannot possibly mean what it says. Take e.g. the 16. 16, according to Peano, is the follower of 15. In this context however it is to be suspected that 16 is used as a bit pattern, the encoding of something. It might e.g. be the encoding for some type of parameter and the programmer would have done better to introduce a constant, e.g. *FunctionType*, with the appropriate value. The program fragment would then have said

if $fp[i, 2] = FunctionType$ **then** ...

which is obviously more abstract.

A second suspicious element is this constant index 2. Experienced programmers know that matrices indexed by a constant often are used to simulate the record-concept in a programming language which does not have it, e.g. FORTRAN or ALGOL 60. In PASCAL the programmer would rather have written (introducing also some more mnemonic identifiers)

if $formalpar[i].type = FunctionType$ **then** ...

Now it has become obviously much more abstract and it is easy to deduce what is intended to take place here. By the introduction of a suitable function for testing the type,

function $HasFunctionType(x : parameter) : boolean$;
begin $HasFunctionType := x.type = FunctionType$ **end**;

the programmer's intention would have become even clearer:

if $HasFunctionType(formalpar[i])$ **then** ...

Is this now some close recording of what the programmer thinks? I doubt that the programmer has square brackets in his mind and from introspection know that while writing such things our thinking goes more like

if *ThatParameterIsaFunction* **then** ...

where we are sure that, when necessary and at another place, a suitable definition of *ThatParameterIsaFunction* can be given.

The point is that the programmer does not want to think in detailed fashion. At this place he expresses only *what* he wants to do, at another place he will formulate *how* to do that. This leads to the following definition of abstraction

Abstraction = the conscious omission of detail.

4.1.2 Forms of abstraction

In programming languages, the following techniques for omitting detail can be distinguished:

naming: giving a name to an entity allows us to use that name instead of the entity

hiding of detail: hiding the realization of an entity makes it impossible to capitalise on knowledge of its details

generalization: extension from known concepts.

Not all of these are supported by present day programming languages, but it is hoped that future programming languages will explicitly supply mechanisms for these and other forms of abstraction.

4.1.3 Naming

The insight in the value of naming has grown slowly. It can be seen in retrospect that the greatest step forward in assemblers was the introduction of symbolic labels. By its obligatory declarations and its "mnemonic" identifiers, ALGOL 60 made programmers much more aware of the value of naming.

Giving suitable names is a skill, to be acquired by long experience. How should names be chosen?

- objects

  var *CurrentBn*, *NumberOfPages* : *integer*

  The name should verbalize the function of the object in the form of an invariant property.

- algorithms

  procedure *swap*(**var** i, j : *integer*);
  function *atEndFile*(**var** f : *file* **of** *sometype*) : *boolean*;

  The name should verbalize *what* the algorithm does (but not *how* it does it) and indicate its type (e.g. action or test).

- types

  type *complex* = **record** *re*, *im* : *real* **end**;
  matrix = **array** $[1..n, 1..m]$ **of** *real*;

  The name should identify the type clearly and shortly — more can hardly be expressed in a type name.

4.1.4 Information hiding

Information hiding, made popular by Parnas [PAR72], will be the subject of much of the rest of this chapter. Its presence allows us to introduce the notion of strength of abstraction. We will speak of *weak abstraction* when the abstract entity has the same properties and attributes as its realization. We will speak of *strong abstraction* when the abstract entity has different (e.g. fewer) properties than its realization.

Information hiding is made possible by the encapsulation of the realization of entities and the control of the visibility of the names of entities through interfaces.

4.1.5 Generalization

The main mechanism for generalization in present day programming languages is the *genericity* of algorithms. As an example, the operator + is available for a number of different types of operands, with different semantics: adding two integers involves different circuits in the computer than adding two reals or two strings. In e.g. ADA the programmer may add further versions of the + operator at will (e.g. between two sets) so that any operation that feels to him like an addition can get the same name +.

Another example of genericity is provided by output procedures, which write in different ways according to the number and type of their parameters, but which all have the same name, e.g. *write*.

In programming languages like ALGOL 68 and ELAN [KOS87], where genericity is explicitly available to the programmer as an abstraction mechanism, all procedures can be generic, so that any number of procedures with the same name can be present, provided their parametrizations differ.

Genericity is a widespread phenomenon in natural languages: A verb like "to drink" has different meanings, depending on the beverage to be drunk — cold beer or cold Aquavit, hot tea or irish coffee. Even the borderline between drinking and other activities is a matter of personal taste (does one drink soup or eat it?) and even of cultural background (In Malay, the verb "minum" denotes our concept of drinking but it is also used for the smoking of a pipe or cigarette; the same holds for the Turkish verb "içmek").

4.2 Construction

A second principle in programming is the composition of entities (algorithms, objects and types) out of (simpler) entities. This principle, which is at the heart of structured programming, we will call *construction*. It can be compared to the building of masonry out of stones and mortar.

construction

4.2.1 Constructors for algorithms

In programming languages, the constructors for algorithms are typically the *control structures* and various forms of parametrized *algorithm calls*. As an example, a single identifier a occurring as the right-hand-side of an assignment is a piece of program for taking the value of a. We can write a similar one for b and stick them together by means of an operator, to make

$a > b$

We can go further with this construction process and write a program fragment like

if $a > b$ **then** a **else** b

In principle we can proceed in this fashion until we have obtained a program realizing the function we want. We may however choose to interpose an abstraction step, declaring e.g.

```
function max(a, b : real) : real;
  begin
    if a > b then max := a
    else max := b
  end
```

After this we may write at will

$max\,(x, 0)$

or

if $x > 0$ **then** x **else** 0

with one same meaning. Strictly speaking, such abstraction steps are superfluous. Given adequate control structures, every function that can be realised at all can be realised by construction alone. The limited resources of the human mind can however be employed much more effectively by alternating the use of abstraction with the construction steps.

4.2.2 Constructors for objects and types

The constructors for objects and types are typically the *data structures*. From a primitive type `real` we may construct a row of reals

array $[1..n]$ **of** *real*

or a record with two real fields

record re, im : *real* **end**

We may choose to interpose some abstractions like

type *compl* = **record** re, im : *real* **end**

after which we can write with one same meaning

var z : *compl*

or

var z : **record** re, im : *real* **end**

4.2.3 Limitations of construction

By repeatedly using construction, we obtain a program in which the intermediate steps in our thinking are not expressed immediately but are encoded in the final result. Bottom-Up programmers and mathematicians may very well be able to construct programs which are correct in spite of their monolithic complexity. Experience shows, however, that for the implementation and maintenance of large programs with fallible human beings, it is essential that the program should retain the abstractions of its author. Comments are not enough, because they are not a functional part of the program and there is no reason for them to retain any correspondence with the actual programming text. In the "structured programming" debate, almost all the stress was laid on construction mechanisms, viz. the control structures. Experience shows that for systematic programming abstraction is at least as important. The greatest sin in programming is being needlessly concrete.

CDL has gone to the extreme of practically banning construction. It offers minimal control structures, sufficient for local composition and decomposition of algorithms, but for the rest relies on a simple and powerful procedure mechanism, which invites abstraction:

- no nesting of procedures

- declarations allowed in any order
- low visual overhead in parameter passing
- encapsulation and explicit control of visibility
- optimizations reduce the overhead involved in calling short procedures.

It provides no construction mechanism for data structures and suggests the use of abstract algorithms instead.

In this way, CDL2 attempts to encourage algorithmic abstraction in programming, even at the expense of constructional convenience.

4.3 Visibility

visibility
interface

As a basis for the discussion of the structure-in-the-large of programs, we will introduce in this chapter the notions of *visibility* and *interface*, which allow a generalization of scope rules. We prefer to consider the notion of visibility rather than the more conventional "uses"-relationship between objects, because we are interested in the (static) structure of programs rather than their (dynamic) behaviour.

Designing the modular structure of a program was, until a decade ago, the exclusive domain of practicing informaticians, scorned by academics. It was felt at that time that one intelligent scientific programmer could program anything that was worth programming; if programming in industry took many people writing many modules under close management, that was their problem. Even after Parnas [PAR72] and others have made modularity an academic issue and means for the expression of structure-in-the-large are an accepted part of programming languages, designing the modular structure of a significant program remains an art.

We will discuss a conceptual model of modular program structure, based on the notions of visibility and abstract machines. In particular we will investigate the relationship between hierarchical and modular structure.

We will take a view strongly oriented towards programming languages but firmly based on a conceptual model of programs and the programming process.

We will restrict ourselves to purely sequential systems and will not deal with such issues as the modular structure of cooperating sequential processes, or the instantiation of modules. The practical issues of precompilation and separate compilation, overlaying etc. have to be taken care of at another place; we will limit our attention to conceptual issues here.

4.3.1 Basic concepts

Units

In the sequel we will avoid talking in terms of any specific programming language, and therefore will choose our terms either vague or universal. We visualize a program as consisting of *units*, where a unit may be a small syntactic construct (e.g. declaration, statement, expression) or a larger one (Class, Cluster, Form, Module, Packet, Package, Prelude or whatever). Such units have the property that they may

- *define* some entities, and
- *apply* some entities.

Entities and names

entity

By an *entity* we mean, depending on the particular programming language, anything which has a name. The names (identifiers, underlined names, special operator tokens etc.) are the denotations within the program for the objects, types and routines allowing their manipulation.

The declarations are constructs for associating a name with an object, type or routine, thus *defining* an entity. An occurrence of the name of an entity otherwise than in its declaration is an *applied* occurrence of that entity. Since the association between an entity and its name is so close, we usually use the identifier to denote the entity when no confusion arises.

definition

application

Visibility of entities

A name may, according to the identification rules of the particular programming language, *identify* in some context a particular entity with which it has been associated.

identification

We say that the entity x from the unit A *is visible* in B if any occurrence of x in B identifies the same entity that it identifies in A. This state of affairs is pictured in figure 4.1.

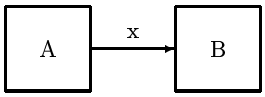


Figure 4.1: A visibility relation

Alternatively, we may we say A *exports* x, and B *imports* it.

export

import

Notice that x is visible in B irrespective of the question whether B actually does contain an applied occurrence of x. Thus, visibility is a necessary, but not sufficient condition for the use of an object accross unit boundaries.

Interfaces

With every unit we may associate two (possibly empty) lists of names

- the list of names of those entities, which have a defining occurrence within the unit and can have an applied occurrence outside it, is termed the *defines-part*

defines-part

- the list of names of those entities, which have an applied occurrence within the unit but a defining occurrence outside it, is termed the *applies-part*

applies-part

Together, the defines-part and the applies-part comprise the *interface* of the unit. Whereas in traditional algorithmic languages the interfaces between units tended to be implicit (according to the scope rules), in more recent programming languages some constructs are equipped with explicit interfaces, allowing much finer control of visibility (e.g. [WIR80]). In particular, there may be a more or less elaborate notation [DRE75] for connecting the defines-part of one unit to the applies-part of another. By the *visibility structure* of a program we mean a graph of its constituent units together with the visibility relations between them, as delimited by the interfaces.

interface

Taxonomy of interfaces

Interfaces can have many different forms and styles. Some of the alternatives are:

- *distributed:* (e.g. each entity indicating at its definition whether it is public or private), *centralised* (e.g. at the head of the module), or even *separate* defines-parts (a specification part listing the visible entities with their properties and a textually separate module body containing their implementation as well as private entities),
- *itemized* (listing each visible entity by name) or *global* applies-parts (e.g. giving only the name of the module from which entities are to be imported)
- *positive* (stating what should be visible) or *negative* defines-parts (stating what should be private).

For each of these choices we can point at actual programming languages with various syntactic forms for the expression of interfaces.

4.3.2 Visibility rules

We want to study methods for restricting by means of interfaces the visibility of entities between units, as a generalization of traditional scope rules.

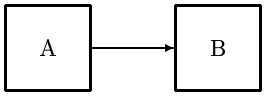


Figure 4.2: Strictly controlled visibility

Denoting the defines-part of a unit A by *def(A)* and its applies-part by *appl(A)* we will use figure 4.2 if the following holds:

$$x \text{ from } A \text{ is visible in } B \Leftrightarrow x \in def(A) \cap appl(B)$$

The visibility of the name x depends on mutual agreement between A, which must export it, and B which must import it.

Specific classes of units may have more liberal interfaces in the sense that they implicitly export any entity defined in them, or import any entity applied but not defined in them. We will indicate this situation by marking with a small letter o (for open) the begin or end, respectively, of the visibility arrow.

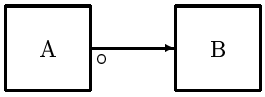


Figure 4.3: Only import is restricted

Thus, figure 4.3 is an expression of

$$x \text{ from } A \text{ is visible in } B \Leftrightarrow x \in appl(B)$$

and similarly, we indicate as in figure 4.4 the situation where B imports all objects exported by A and as in figure 4.5 the situation where there is no restriction at all on the visibility between A and B.

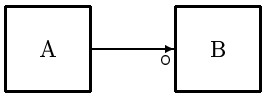


Figure 4.4: Only export is restricted

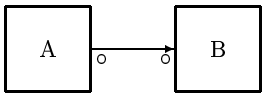


Figure 4.5: No restrictions on import/export

4.3.3 Examples of visibility structure

Block-structure

In languages with *block-structure*, like PASCAL, the visibility between a block X and another block Y nested directly in it is as in figure 4.6 unless an entity with the same name as x is declared in Y.

```
begin
  var x : integer;
  begin
    Y
  end;
  rest of X
end
```

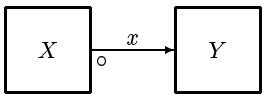


Figure 4.6: x defined in X is visible in Y

From this observation, two drawbacks of block-structure follow [WUL73]:

- Since X imposes no export restrictions, any variable declared in X can be modified from Y, which poses problems for sensitive information.
- If a declaration for x is forgotten in Y, another entity with the same name may creep in from X. This "trojan horse" problem is rather insidious, and may lead to surprises. Imagine a forgotten declaration of a loop variable (of course with the name i) within the body of another loop with (of course) the same loop variable. Another i creeps in, and the program becomes surprisingly fast. In larger programs such unintended modifications of globally visible objects can lead to hard to find bugs.

Anti-block-structure

We will use the term *anti-block-structure* for a situation where a unit Y is nested in a unit X but the visibility arrow goes from Y to X. Anti-block-structure occurs in block-structured languages,

e.g in procedure declarations in ALGOL 60:

begin
  procedure *heading of P;*
    body of P;
  rest of block
end

with the visibility structure of figure 4.7 since, of all the entities declared in the heading, only P

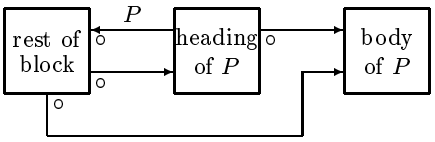


Figure 4.7: Visibility structure of procedure P

is exported back to the surrounding block. For other entities, block-structure visibility rules hold. In the rest of the block, no other entity with the name P may be defined.

A further example of anti-block-structure can be found in the relationship between a block X and a class Y nested directly within it. In this case all (public) local entities of the class are exported to the block X.

It can be argued that this "deviation" from block structure contributes noticeably to the success of the class concept.

Limitations of (anti)block-structure

One example should suffice to make clear that not all useful visibility structures can be achieved through a mixture of block- and anti-block-structure.

Suppose we want to define a procedure *read identifier*, using two procedures *read letter* and *read digit* that in their turn make useof a procedure *read char* which is used nowhere else and should be visible nowhere else, as in figure 4.8.

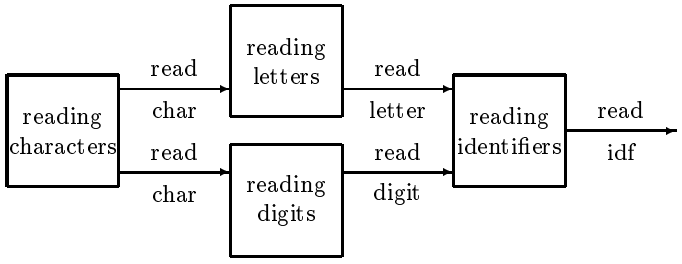


Figure 4.8: Tiny fragment from a parser

Using anti-block-structure, the right half of the diagram can be realised by declaring both *read letter* and *read digit* local to the procedure *read identifier* but the procedure *read char* would have to be local to both *read letter* and *read digit*, which would then only be possible by having two copies.

Making the procedure *read char* global to those two procedures gives rise to an unwanted visibility arrow, see figure 4.9. A more precise control of visibility is needed.

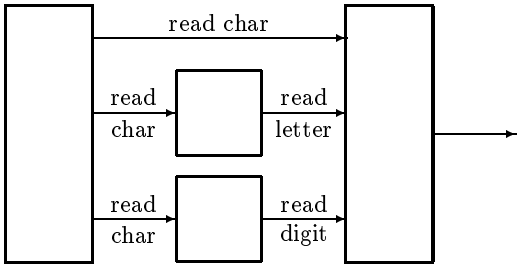


Figure 4.9: Parasitic visibility

4.3.4 Chaos

A visibility structure which is widely used in assembly language programming is one where exported objects are visible wherever they are explicitly imported: objects exported by some unit (module) can be imported by all other modules, but all import/export is explicit (figure 4.10). We will term this state of affairs *controlled chaos*. On the one hand there is no superimposed structure

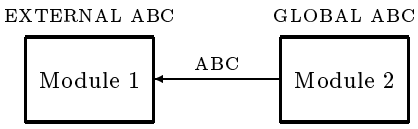


Figure 4.10: Visibility structure with GLOBAL/EXTERNAL

(and in principle the number of arrows may be quadratic in the number of units) but on the other hand there can be quite precise control over all visibility relations through explicit interfaces.

Many successful systems have been written with such visibility structures. It can be argued that, for large programs, this structure is more suitable than block-structure and anti-block-structure combined.

4.3.5 Hierarchy

Widely advocated since [PAR72] and widely accepted in computer science circles is the *hierarchical* structure, where the visibility graph can be divided into levels such that

- from level 0 no entity is visible, that is to say, level 0 imports no entities
- from level i only entities on levels $< i$ are visible, and at least one entity is in fact imported from level $i-1$.

Such a *weak hierarchy* allows "jumping" over any number of levels. We can obtain a *strong hierarchy* by insisting that from level i only entities on level $i-1$ are visible. This leaves open the possibility of controlled level jumping through $\longrightarrow^+$ (see figure 1 in [ROB75]) but makes level jumping somewhat harder, so that hopefully the programmer will spend more energy on avoiding such jumps.

Both block- and anti-block structure are special cases of hierarchies. Notice that the definition given excludes recursion, so that consequently it has been excluded from e.g. Concurrent PASCAL [HAN74]. Since for many systems (e.g. compilers) recursion is essential, the second condition is often weakened to "only entities on levels $\leq i$ are visible". But as a consequence a complex system

PASCAL

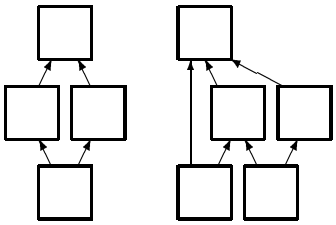


Figure 4.11: Example of a hierarchy

can be considered as a hierarchy in very many ways and the establishment of levels becomes highly arbitrary.

4.3.6 Nested units

nesting

Various structures can be obtained by nesting units within units, allowing exports to and from the enclosing unit but not through it. Maybe the most profitable way of envisaging the structure of large systems is by seeing them as structures nested within structures.

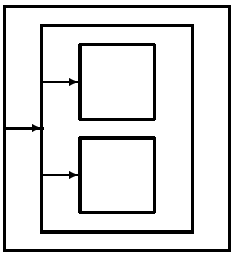


Figure 4.12: Nesting of units

Block structure can easily be depicted as nesting, as shown in figure 4.12. The same holds for anti-block-structure or any hierarchy.

In particular it is possible to have a more relaxed structure (e.g. controlled chaos) within the units of a hierarchy, which gives an adequate solution to the problem posed in section 4.3.3. In this approach, the global structure in the large is highly regular (a hierarchy) whereas locally some control over visibility is given up for the sake of convenience, the local visibility being controlled separate from the global visibility.

4.4 Abstract Machines

A program (say, for weather forecasting), together with a concrete machine on which it runs, can be seen as an abstract machine(see figure 4.13) which can perform only one task, however complicated, for which it has been specially tailored. This abstract machine, consisting of both hard- and software, would obviously at present be too expensive to build as a concrete machine, in hardware. A program is a means to obtain various abstract machines economically.

If we inspect the machine more closely, we see that it in turn contains another, even more concrete machine and a concrete program consisting of some firmware and some software sitting atop of it, (figure 4.14).

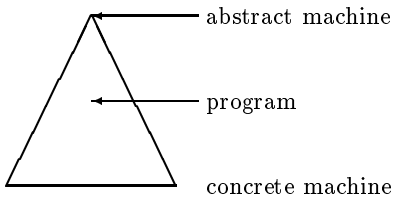


Figure 4.13: Abstract machine = concrete machine + program

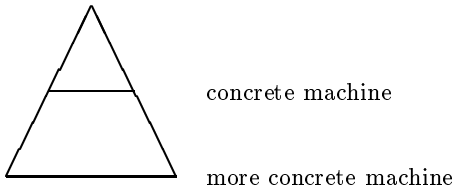


Figure 4.14: Levels within the machine

In fact, it turns out that we can make cuts on different levels in an abstract machine, obtaining each time another machine and a program. In the same way we may cut the program, splitting it into a concrete program, which executes on the concrete machine, and an abstract program (figure 4.15).

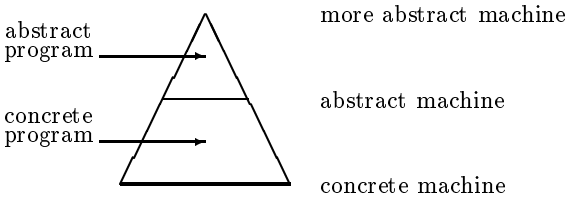


Figure 4.15: Within an abstract machine

The attributes "abstract" and "concrete" turn out to be relative to an ordering imposed by our understanding.

A program needs for its execution services and resources from the underlying machine. These are denoted in the program as algorithms and objects, exported by the machine. This interface of algorithms, objects and types between the machine and the overlying program we will call the *abstract language* implemented by the machine. Thus, the program as a whole implements a language consisting of one algorithm.

Of course a program is not some species of strange animal, whose structure has to be determined by analysis after its discovery. The systematic design of a program entails the design of a number of coherent interfaces at various degrees of abstraction within its abstract machine.

4.4.1 Layers of abstraction

An abstract machine with cuts at various levels can be depicted as consisting of *layers* of units, like in figure 4.16 [POO73]. layers

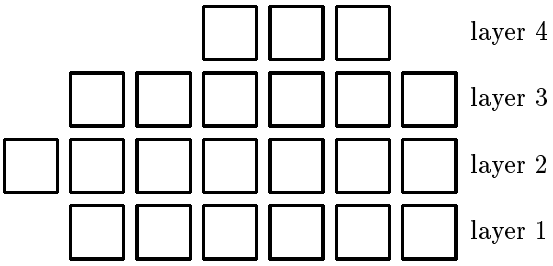


Figure 4.16: Abstract machine composed of units

As is common understanding, the layers each correspond to a *level of abstraction.* As an example, in the parser of a compiler these layers might correspond to the character level, token level, symbol level and syntactic level, respectively. The units within one layer conspire to present a more abstract, a higher level interface than the underlying layer. One layer defines a number of abstract algorithms, objects and types, in terms of which the next higher layer is realised.

In our hierarchy of layers we will therefore term the vertical direction the direction of *abstraction.*

4.4.2 Extension

One layer of the abstract machine is composed of units on one same level of abstraction. The units within one layer need entities from the immediately preceding layer, and from the other units within this layer.

The units within a layer serve to incrementally extend the power of the abstract machine while remaining at one same level of abstraction. The horizontal direction of visibility we will therefore term *extension.* The notion of extension is useful to describe the gradual and piecewise

extension

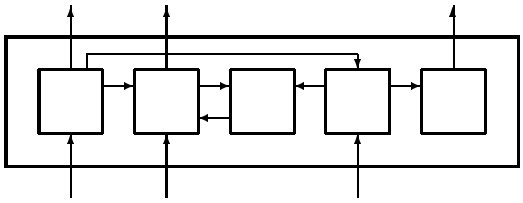


Figure 4.17: Abstraction and extension

realization of the entities from an interface. It also prevents the consideration of very deep but weak hierarchies, with a consequent high degree of level jumping.

The layer can serve as a unit within which other units are nested, where only a small part of the entities defined by those units are exported.

Within the layer, units may in their turn have interfaces and be further nested, allowing control of visibility as needed.

4.4.3 The three-layer-model of programming

In programming, we wish to realise a specific abstract algorithm (the program) in terms of the expressive means given by a concrete language on a concrete machine.

In designing the structure-in-the-large of a program we can begin by making a Top-Down exploration (figure 4.18), inventing and refining useful abstract algorithms and objects as we go

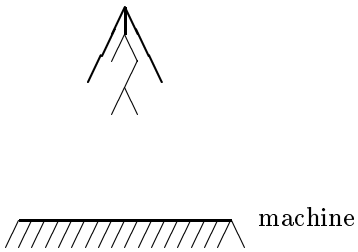


Figure 4.18: Top-Down exploration

along. We may even in this way succeed in deducing the whole program, but for others than toy programs this is highly unlikely. As we make more and more decisions, we will make decisions in several places that are difficult to reconcile at a lower level. We may even be in danger of in a sense "missing" the concrete level, i.e. arriving at a program formulation that cannot be realised well in terms of the given means.

When we notice we start having trouble in taking the right decisions for every refinement, we may be at a good point to step back and consolidate the progress reached until now in the form of an interface (figure 4.19).

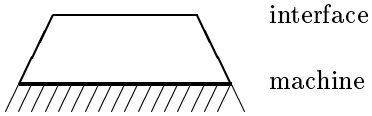


Figure 4.19: Consolidating the interface

We have gained one thing: we now know quite a lot about the services and resources necessary for the working of the program ("We will need a hashed association table and two quasi-stacks, input and output of such symbols, ..."). We can now realise these as algorithms, instances of data structures and types by a Bottom-Up technique.

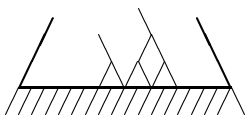


Figure 4.20: Bottom-Up exploration

Again, it is highly unlikely that we will, by proceeding in this way, match the interface found in the Top-Down phase without meeting further problems. If we proceed too far Bottom-Up, we may introduce entities which will turn out to have no use. We are even in danger of "missing" the interface altogether. But, once we notice that we start having trouble inventing further primitives, we are at a good point to step back and consolidate the progress reached in the form of another interface.

The distance between those two interfaces can now hopefully be covered by the usual programmers mixture of Top-Down-programming, Bottom-Up-programming and legwork, thus closing the

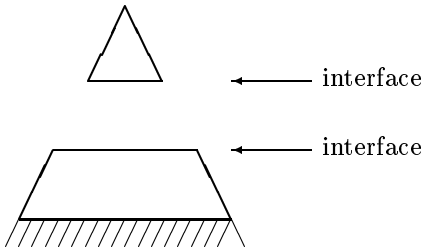


Figure 4.21: Consolidating again

gap in three rounds. At this point the programmers knowledge of programming techniques, e.g. the choice of suitable data structures, and his experience and common sense in finding the "right cut" have a profound influence on the overall quality of the program constructed.

Somewhat tongue-in-check, it may therefore be argued that all nontrivial programs consist of exactly three layers, e.g., for a parser:

```
syntax
symbols
characters
```

or, more in general

application
function
representation

The application is written in a language enriched with just the right functional primitives; the representation layer hides both machine dependencies and pragmatic representational decisions. In the function layer, knowledge of the subject area is incorporated.

In the CDL2 experience this "trichotomy" was found to be much more helpful than the conventional dichotomy

applying environment
defining environment

even when this dichotomy is used recursively.

4.4.4 Functional modularity

An abstract machine can not only be decomposed into layers, but many other criteria for decomposition can be of importance (unit of work for one programmer, unit of compilation, pass or phase, overlay etc.).

module

One intuitively obvious way is to decompose the program into *modules*, by which we will mean units that in some sense realise one common service or function, e.g., where the dotted arrow denotes some means of communicating information between units (see figure 4.22).

The notion of "communicating information" between units is dangerously vague: it might comprise file transfer, shared variables or (mutual) procedure calls. (That is why in the figure we dotted the arrows: they mean very little).

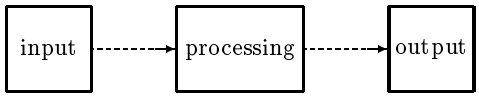


Figure 4.22: Functional decomposition

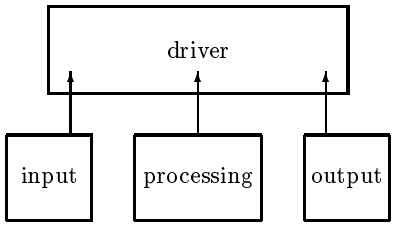


Figure 4.23: The corresponding visibility structure

A much more precise notion is that of visibility, introduced earlier in this chapter. Expressing the previous structure as a visibility structure, each unit defines some specific entities which can be used by some driver to realise the communication needed (see figure 4.23).

Apart from the driver, those units are on one same level of abstraction, but within those units layers of abstraction may be useful (see figure 4.24).

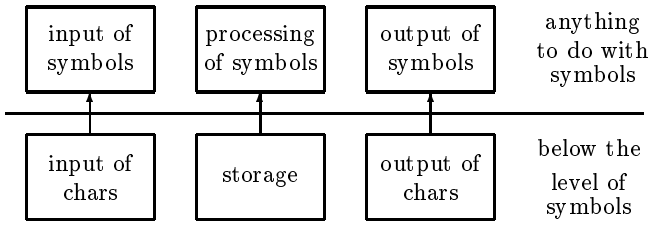


Figure 4.24: Splitting into layers

The functional decomposition into modules and the abstractional hierarchy of layers are orthogonal to one another.

We can decompose the structure of this program in two orthogonal dimensions (figure 4.25). In one direction the program is split into layers, each on a coherent level of abstraction and with minimal export interfaces. Orthogonal to that partitioning, the program can be decomposed into modules, each implementing a complete and coherent function or service.

Of course not all modules of a program need have all three layers, and further subdivisions may be useful, so that a more realistic example may look like the one in figure 5.1. An abstract machine with three layers and three modules can be depicted as in figure 4.26.

We hope to have given sufficient arguments for a model of program structure in which hierarchy and modular decomposition are orthogonal to one another. Of course the model does not give more than broad indications of the purpose and use of layers and modules. Both are artifacts that we can employ at will to master the complexity of the design and implementation of large programs.

Functional decomposition, which can be achieved through the use of a number of design methods (such as the Structured Analysis and Design Technique of Ross [ROS??]), can therefore well

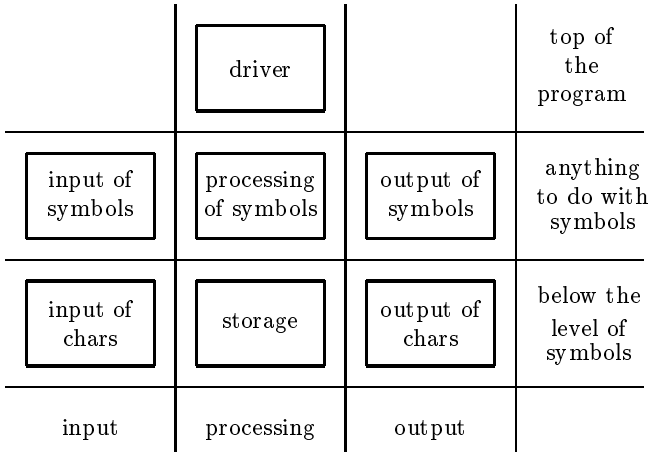


Figure 4.25: Two dimensional decomposition

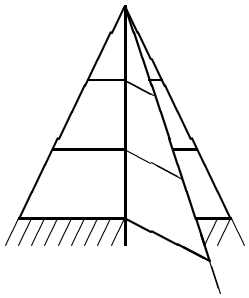


Figure 4.26: A modular abstract machine

be combined with design techniques based on abstraction and both can be supported by language mechanisms for programming-in-the-large.

4.5 Structure in-the-large of CDL2

The structure-in-the-large of CDL2 programs has evolved since 1974 in a number of steps.

Programs, written in the predecessor language CDL1, were certainly "structured" in the then prevailing sense: They consisted of a large collection of small procedures and other declarations, with mnemonic identifiers and impeccable control structures, but without any types or interfaces. Programs were up to a few thousand lines long, typically written by one programmer. The need for imposing a structure-in-the-large came mostly from the desire to program much more ambitious systems (in this case an ALGOL68 translator) as multi-person tasks.

Modularity, at that time, was a mysterious commodity that hardheaded practical programmers seemed to know all about, but which they jealously kept away from academic scrutiny.

4.5.1 First model

When we started the design of CDL2 in 1974, we initially chose a simple scheme with centralised explicit itemised defines- and applies-parts, like the one which appeared later in the implementation

language MODULA [WIR80]: chaos controlled through explicit interfaces.

This scheme was reasonably successful (there are still quite some CDL implementations with this mechanism around) but we were not satisfied with it, because by itself it gives no guidance at all about what should be the structure of a program.

4.5.2 Second model

The second scheme we developed reflects already many of the arguments that have been given in the previous sections.

A program consists of a strong hierarchy of layers with explicit interfaces. Within a layer are nested one or more modules whose visibility structure is chaos controlled through explicit interfaces. Nested within the modules, a third grouping mechanism was to be found, the section, again with explicit interfaces.

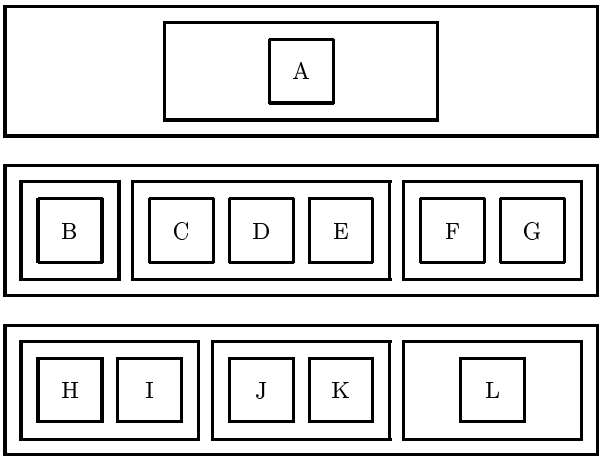


Figure 4.27: Sections within modules within layers

As an example, a structure similar to that given in figure 4.25 would have to be expressed as in figure 4.27. This model was implemented and experience gathered from use in teaching and one project. In this way we enforced a shallow but strict hierarchy. It soon turned out that this notion of module was very disagreeable to use. Modules were strange collections of sections, incomplete because limited to one layer.

An enormous amount of interfaces was necessary. This presented no problem (apart from boredom) as long as interfaces were decided beforehand and then never changed, as they should. But, human weakness being what it is, changes were needed incidentally. And such a simple thing as changing the name of an entity would involve at least six interfaces as illustrated in figure 4.28!

4.5.3 Third model

After some experimentation we arrived at a third model in september 1976.

A program consists of a strict hierarchy of layers which themselves have no interfaces but are merely administrative bodies. Within a layer we have a controlled chaos of sections.

All interfaces are centralised at the *sections*. Each section has three interfaces (see figure 4.29): section

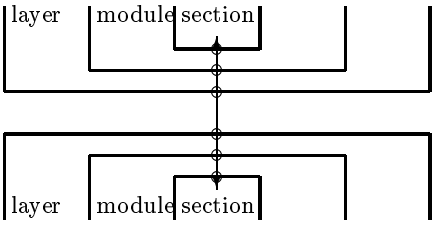


Figure 4.28: Six interfaces involved in one modification

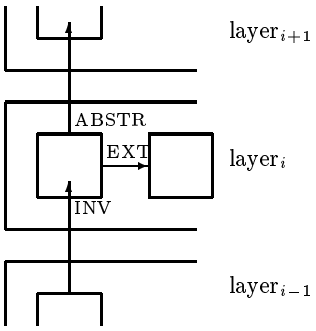


Figure 4.29: Interfaces of a section

- The ABSTR-interface indicates which entities it is willing to export to any section in the next higher layer.
- The EXT-interface indicates which entities it is willing to export to any section in the same layer.
- The INV-interface indicates which entities it must import from another section (either in the same layer or in the preceding one).

This scheme has been in use since 1976 in a great many implementation projects. The idea of attaching all interfaces to the sections works well, although the itemised INV-interfaces listing each entity (rather than the section from which they are imported) lead to long interface lists (up to 20% of source code size), which can be quite boring for the programmer.

4.5.4 Fourth model

The previous model works fine, and is still in use for many applications. For large projects, the idea of one monolithic program, even when structured in the way just sketched, is untenable. Separate compilation demands an other partitioning. So MODULEs were reintroduced, this time as sequences of layers consisting of sections. Interfaces were kept with the sections:

- ABSTR-, EXT-, INV-interfaces as before.
- The EXPORT-interface of a section lists the entities to be made available to other MODULEs.
- The IMPORT-interface lists the entities to be imported from other MODULEs. Furthermore, a specification of the entities must be given (essentially in the form of procedure headings) to enable consistency checks accross compilation units.

4.5.5 Present situation

This last model is the one supported by the CDL2 LAB [BAY81]. In its use, a certain "style" of program design and structuring has developed. It is considered good practice to write IMPORT-sections, which consist solely of imports and specifications of entities, making them available to the module under consideration via the EXT and ABSTR mechanisms.

Programs constructed by modules thus have multiple hierarchies. The layering in different modules may well differ. Since the export- and import-interfaces of the modules may occur in any layer this does not give rise to practical problems, but it damages the simple picture given in figure 4.26.

A good example is the second phase of the CDL2 compiler: The global analyzer with optimizer and lineariser is one module and the code generator another, both hierarchically structured according to their respective needs. They communicate via exported and imported procedures.

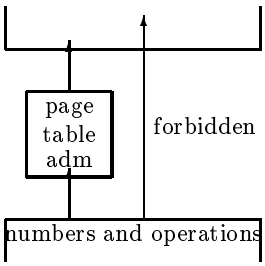


Figure 4.30: A problem with strict hierarchies

The second reason for having different layer structures is that some abstractions seem to be much more pervasive than others. As an example, a page frame, a matrix and a number are all three objects, but they should have rather different scopes, which cannot be fitted well in a strict hierarchy.

Suppose e.g. that numbers and arithmetic operations are necessary to implement page tables, but that we want to use arithmetic operations again in the next higher layer (figure 4.30). We cannot achieve this in CDL2 by level jumping, since the hierarchy of layers is strict.

We may achieve it through a kludge, by renaming the operations, as shown in figure 4.31.

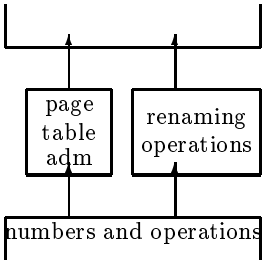


Figure 4.31: Renaming

Or we can have the numbers and operations in a separate module and accept the fact that different modules have different "rates" of layering as depicted in figure 4.32.

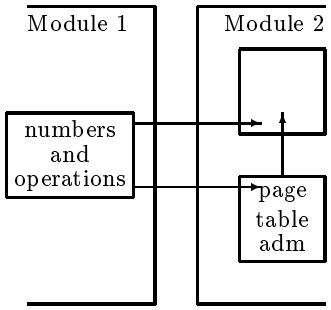


Figure 4.32: A breach of hierarchy

4.6 Modules and specifications

An object, which is to be exported from a module `give` to a module `take`, must be declared and explicitly exported in the module `give` and explicitly imported and specified in the module `take`. A specification for some object has the form of a declaration without a body. Just like ABSTR-, EXT- and INV-interfaces, IMPORT- and EXPORT-interfaces are attached to sections.

Only algorithms and constants can be exported. A variable or list can not be exported otherwise than as a set of access algorithms, thus making global updates highly conspicuous.

As an example, the exporting module `give` could contain the following definitions:

```
MODULE give .
        ...
    SECTION arithmetics
       EXPORT incr, add, maxint, ... .
           ...
       FUNCTION incr + >x> =
            "$L INCL " x.
       FUNCTION add + >x + >y + res> =
            "$L ADDL3 " x "," y ", " res ".
       CONST maxint= 4711.
```

and the importing module might contain as specifications:

```
MODULE take.
    SECTION imports from give.
       EXT incr, add.
       ABSTR maxint.
       FUNCTION incr + >x> .
       FUNCTION add + >x + >y + res> .
       CONST maxint.
```

For the design and maintenance of large systems it proves useful to construct specific *import sections*, which contain nothing but the specifications of the imported objects, the corresponding imports and the necessary EXT- and ABSTR- interfaces to make the imported objects available to the other sections of the importing module. One such import section should be constructed for each module from which objects are to be imported and the name of that section should reflect the name of the exporting module. Note that the CDL2 LAB provides tools which automatically construct such import sections from modules exporting objects.

4.7 Example: a file reformatter

We will introduce the precise syntax of the program structure by a (synthetic) example. Suppose we want to realise structure depicted in figure 4.33.

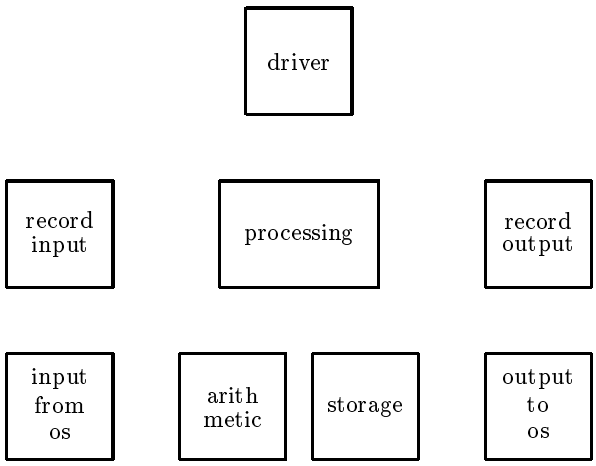


Figure 4.33: Structure of a file reformatter

We will assume this to be the structure of a program for reformatting files, reading them in one format and writing them, after some processing, in a different format. The structure was chosen to be singularly symmetrical.

The program may well be divided into three modules as shown in figure 4.34. Within each

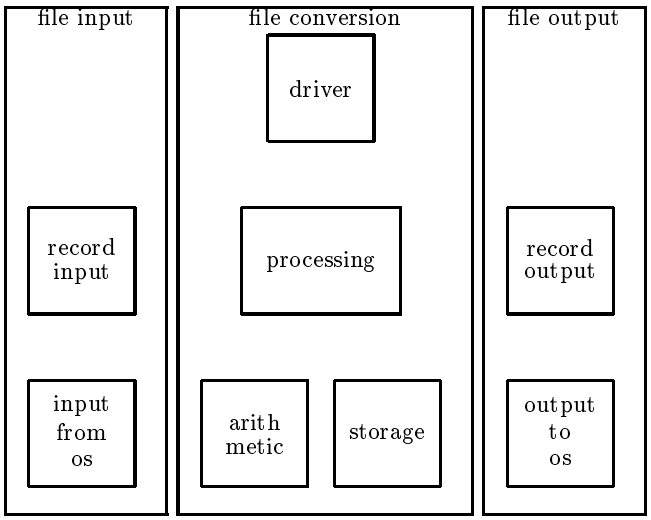


Figure 4.34: Module structure

module, we will make use of the same layer structure shown in figure 4.35. As can be seen from this figure, eight sections are distributed over these layers and modules.

When inputting a two-dimensional structure by linear means, such as the keyboard of a terminal, we will have to enter its components in some specific order. The modules can be entered in

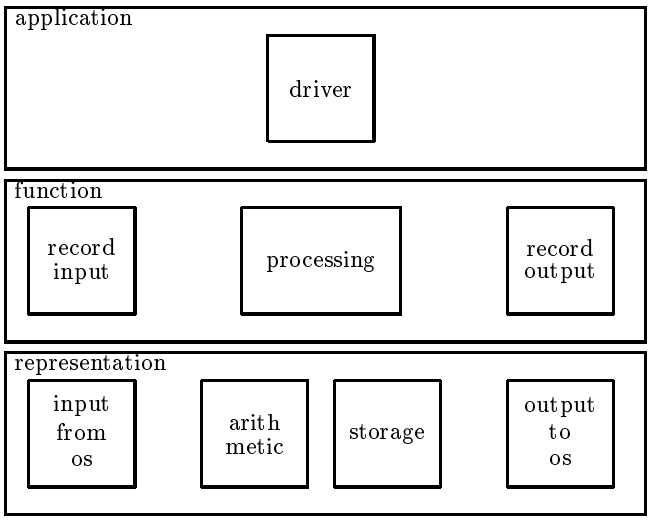


Figure 4.35: Layer structure

any order, and their order is immaterial.

Within one module, we have to enter the layers in Bottom-Up order, giving the lowest layer first. For the understanding it might be better to first look at the highest layer, which unfortunately comes last. Within each layer, the ordering of the sections is immaterial. Every section begins with its optional EXPORT-, IMPORT-, ABSTR-, EXT- and INV- interfaces, in arbitrary order. The module `file input` will have to be entered as

```
MODULE file input.
    LAYER representation.
       SECTION input from os.
          IMPORT make, equal, ... .     # from the other modules #
          ABSTR ... .                  # there is nothing to EXT #
             specifications of imported entities
             declarations belonging to this section
       ENDSEC input from os.
    ENDLAY representation.

    LAYER function.
       SECTION record input.
          EXPORT ... .        # for the module 'file conversion' #
          INV ... .           # from the section 'input from os' #
             declarations belonging to this section
       ENDSEC record input.
    ENDLAY function.
ENDMOD file input.
```

Notice the syntactic style used, bracketing the major components between sentences, each starting with a delimiter (in uppercase) followed by a name and ending with a period. All constructs in CDL2 have such a sentence-like structure.

Similarly, the module `file conversion` will be entered as

```
MODULE file conversion.
    LAYER representation.
```

```
      SECTION arithmetic.
      EXPORT make, equal, ... .          # to the other modules #
      ABSTR add, incr, decr, equal, make, ... .
                                         # to the next layer #
      EXT make, equal, ... .         # to the section 'storage' #
         declarations belonging to this section
      ENDSEC arithmetic.

      SECTION storage.
         INV make, equal, ... . # from the section 'arithmetic' #
         ABSTR get elem, put elem, ... .    # to the next layer #
         declarations belonging to this section
      ENDSEC storage.
   ENDLAY representation.

   LAYER function.
      SECTION processing.
         ABSTR ... .                        # to the next layer #
         INV add, incr, equal, get elem, put elem, ... .
                                  # from the previous layer #
         declarations belonging to this section
      ENDSEC processing.
   ENDLAY function.

   LAYER application.
      SECTION driver
         IMPORT ... .                     # from other modules #
         INV ... .                       # from previous layer #
         specifications for the imported entities
         declarations belonging to this section
      ENDSEC driver.
   ENDLAY application.
ENDMOD file conversion.
```

Notice that the section `driver` cannot directly invoke any entity from the lowest layer, so that it cannot e.g. perform immediate arithmetic.

Assuming the last module to be similar to the first, we have now seen the syntactic means for expressing the structure-in-the-large of a huge collection of declarations, partitioned into layers and modules. But how do we set this whole machinery into motion?

4.8 Roots, preludes and postludes

Somewhere buried amongst the declarations in the CDL2 program, presumably in one of the sections of the highest layer of one of the modules, can be found the *root* of the program, an algorithm which when executed puts the whole program into motion. root

Furthermore, there will be a number of sections at all levels containing one or more algorithms that have to be called in some specific order in order to initialize the various administrative variables in the program. Such initialization algorithms will be called the *preludes* of these sections. They could, of course, be invoked directly from the root, but it is advantageous to consider the collective activity of the preludes as a separate activity, preceding the invocation of the root.

Finally, there may be a number of sections containing one or more algorithms that have to be called in some specific order to finalize various administrations, close files, report statistics etc. after the execution of the root has ended. Again, such *postludes* may profitably be considered as a collective activity, to be executed after the root.

Of course the various preludes and postludes (and, in case there is more than one, the roots) have to be performed in some order which is to be specified by the programmer and which may very well differ from their textual order. To this end, the programmer must specify a *control* for the program, as well as the individual modules and the individual sections.

A control consists of three optional parts, a `PRELUDE`-part, a `ROOT`-part and a `POSTLUDE`-part. Each part contains one or more names in a specific order.

The *program control* specifies in what order the constituent modules of the program have to be preluded, rooted and postluded (sorry for those terms but, as is well known, in the english language all nouns can be verbed; the terms used here are intended to be descriptive but do not form part of the CDL2 jargon).

The *module control*, at the end of each module, specifies a similar order for sections. Finally, the *section control* at the end of a section may specify the order of any initializations appearing in that section.

Note that there is a slight syntactic difference concerning the program control between the CDL2 compiler and the CDL2 LAB. In the CDL2 compiler, one module can be made the main module by specifying the program control after the `ENDMOD`-sentence.

To continue the previous example, the program control for the compiler may look like:

```
ENDMOD file conversion.
PRELUDE file input, file conversion, file output.
ROOT file conversion.
POSTLUDE file input, file conversion, file output.
```

In the CDL2 LAB, the program is described by a separate syntactic construction, the `PROGRAM`, which lists the constituting modules as `PART`s and contains the program control:

```
PROGRAM file converter.
    PART file input, file output, file conversion.
    PRELUDE file input, file conversion, file output.
    ROOT file conversion.
    POSTLUDE file input, file conversion, file output.
ENDPROG file converter.
```

The module control for the first module might be

```
PRELUDE input from os, record input.
POSTLUDE record input.
```

and is to be inserted immediately before the `ENDMOD`-sentence.

The module control for the second module might well look like

```
PRELUDE storage, processing, driver.
ROOT driver.
POSTLUDE driver, processing, storage.
```

As is often the case, it makes sense to perform preludes Bottom-Up and postludes Top-Down.

The module control for the third module might be just

```
PRELUDE record output.
POSTLUDE record output, output to os.
```

Now turning to the sections, we might have the following section controls

```
SECTION              CONTROL

input from os        PRELUDE open input file.
record input         PRELUDE initialize record count.
                     POSTLUDE report number of input records.
arithmetic           --
storage              PRELUDE initialize work storage,
                         initialize stack.
processing           PRELUDE initialize processing,
                         initialize accounting.
                     POSTLUDE complete accounting,
                         send the bills.
driver               ROOT convert my files.
output to os         POSTLUDE close output files.
record output        PRELUDE initialize output count.
                     POSTLUDE report number of output records.
```

The structure resulting from all those controls can be depicted as follows

```
PRELUDE
       file input
            input from os
                 open input file
            record input
                 initialize record count
       file conversion
            storage
                 initialize work storage
                 initialize stack
            processing
                 initialize processing
                 initialize accounting
            driver
                 (has no prelude)
       file output
            record output
                 initialize output count

ROOT
       file conversion
            driver
                 convert my files

POSTLUDE
       file input
            record input
                 report number of input records
       file conversion
            driver
                 (has no postlude)
            processing
                 complete accounting
                 send the bills
            storage
                 (has no postlude)
       file output
            record output
                 report number of output records
```

```
output to os
     close output files
```

Upon scanning the algorithm calls in the rightmost column, it can be seen that the order imposed is indeed sensible.

This quite elaborate control mechanism has been introduced in order to allow the programmer to concentrate as much as possible on the correctness of his algorithms and the visibility relations between them, while leaving consideration of overall order as much as possible to a later phase in programming. The programmer can, without dangers, restrict his horizon to the section he is currently working on.

To the knowledge of the authors, there is no comparable feature in any other programming language. We hope that, in spite of its uniqueness, the reader will readily familiarize himself with this feature and exploit its benefits.

Chapter 5

Example: A text editor

As an example, we will study the design of a simple text editor. We choose this particular example because an editor, like an interpreter, typically can be seen as a collection of semantic actions (table administration etc.) controlled by the syntactic structure of the input. It can also be used to highlight some of the issues in portability.

5.1 The user interface

The task is to design and implement a very simple editor. The editor is line oriented, and deals with a *text* consisting of *lines*, each line containing a *number* and an *entry*. Lines should be kept in the order of increasing line numbers.

The editor has four types of commands:

- The insert-command `<numb>=<entry><RET>`
  where

  `<numb>` is some positive integer (the line number),

  `=` serves as a separator, and

  `<entry>` is some sequence of characters, ending on

  `<RET>` the end-of-line character.

  Effect: If a line with line number `<numb>` is present in the text, it is overwritten with this `<entry>`. Otherwise, a line with number `numb` and this `entry` is inserted.

- the delete-command `d<numb><RET>`
  where

  `d` is the letter d, and

  `<numb>` is a line number.

  Effect: The line with number `<numb>` is deleted from the text if present, otherwise an error is reported.

- the list-command `l<RET>`
  where `l` is the letter l.

  Effect: All lines of the text are listed in the order of increasing line number.

- the quit-command `q<RET>`
  where `q` is the letter q.

  Effect: The editor halts, nothing is saved.

This editor is so crude that nobody would like to work with it, but it serves well as a simple example because in its design important issues are addressed which are not smothered by irrelevant complexity and detail.

5.2 Top-Down exploration

We will first make a Top-Down exploration to find the elements of the problem. At the highest level of abstraction, the editor is a simple interpreter which reads one command line at a time and executes it, until it meets with the quit-command. We have to take into account something which is customarily left out in the problem specification, viz. the fact that human users of an interface make errors. We therefore have to include error treatment and deal with such issues as the skipping of blanks.

```
SECTION editor.
    ACTION simple editor:
        skip blanks,
           (is insert command, simple editor;
           is deletion command, simple editor;
           is list command, simple editor;
           is quit command;
           incorrect command, simple editor).
    ROOT simple editor.
ENDSEC editor.
```

Notice that this algorithmic structure is somewhat harder to achieve in languages with conventional control structures than in CDL. In case the reader feels put off by the many right-recursions in this procedure, she may be happy to learn that all right-recursions are resolved by the implementation of CDL2, see section 8.2.

In realizing the various commands, we will try to separate the recognition of a command from its execution, the two phases communicating through affixes.

```
SECTION commands.
    PREDICATE is insert command - nmb - key:
        is number + nmb, should be equals symbol,
           read entry + key, enter line + nmb + key.
```

The action `read entry` will read up to and including the next end-of-line, so that the command is one complete line long.

```
    PREDICATE is deletion command - nmb:
        is delete symbol,
           (is number + nmb, delete line + nmb,
              skip rest of line;
           incorrect deletion command).
```

The action `skip rest of line` has to deal with the possibility that, after the deletion command but before the end of line, some rubbish will have to be skipped. We might have chosen another convention instead, allowing more than one command per line.

```
    PREDICATE is list command:
        is list symbol, list lines, skip rest of line.

    PREDICATE is quit command:
        is quit symbol, skip rest of line.
ENDSEC commands.
```

5.3 Subdivision into sections

We have been inventing new elementary actions and predicates as we went along. It is now possible to classify these as belonging to specific sections.

```
SECTION editor
    ACTION simple editor

SECTION commands
    PREDICATE is insert command
    PREDICATE is deletion command
    PREDICATE is list command
    PREDICATE is quit command

SECTION error handling
    ACTION incorrect command
    ACTION incorrect deletion command

SECTION symbol transput
    PREDICATE is delete symbol
    PREDICATE is list symbol
    PREDICATE is quit symbol
    ACTION should be equals sign
    PREDICATE is number + n>
    ACTION skip rest of line

SECTION directory adm
    ACTION enter line + >nmb + >key
    ACTION delete line + >nmb
    ACTION list lines

SECTION entry table adm
    ACTION read entry + key>
    ACTION list entry + >key #for reasons of symmetry#
```

5.4 Structure in-the-large

We now split these sections into three layers, according to the scheme

- application
- function
- representation

In the process we will postulate some further sections for basic i/o, arithmetics and basic list operations. The resulting structure of the editor is depicted in figure 5.1.

5.5 Table administration strategy

We will maintain an entry-table holding all entries, and a directory storing for each line its line number and (a pointer to) its entry. The structure of the directory and the entry table can be seen in figure 5.2.

The accesses to the directory will be

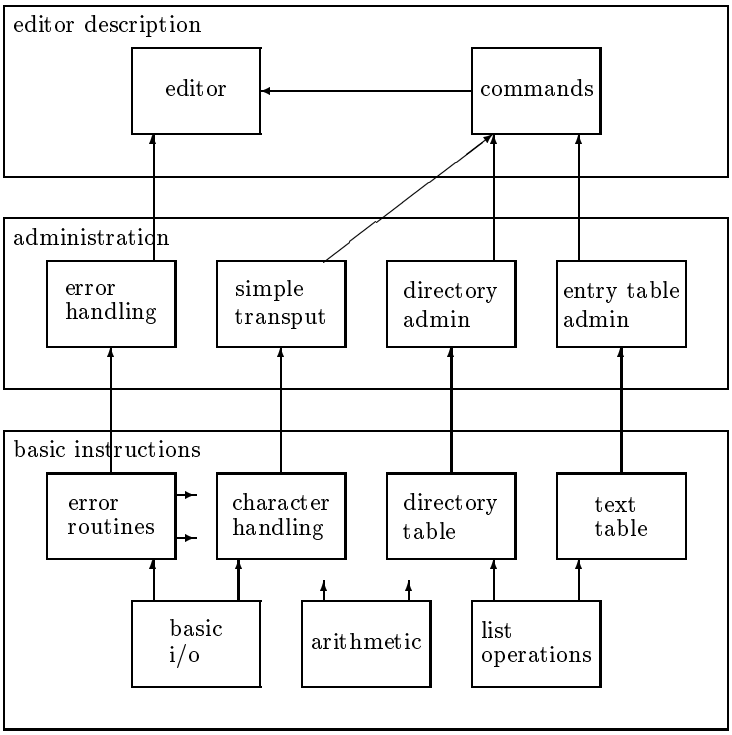


Figure 5.1: Structure of the editor

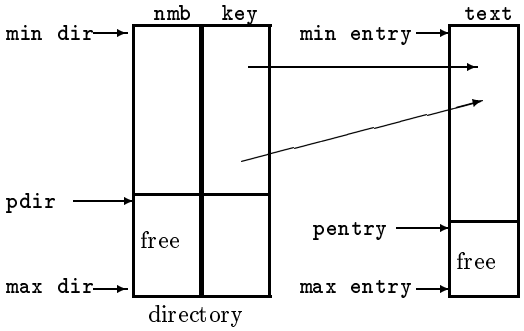


Figure 5.2: Structure of the directory

```
FUNCTION get nmb + >k + nmb>
ACTION put nmb + >k + >nmb
FUNCTION get key + >k + key>
ACTION put key + >k + >key
```

where the first parameter (k) is to be an offset in the directory.

The entry-table can be accessed by

```
FUNCTION get from entry + >k + word>
ACTION put to entry + >k + >word
```

where the first parameter is to be an offset in the entry table and one word at-a-time is to be read or written.

For the (inclusive) bounds of these tables we assume

```
CONST min dir, max dir
CONST min entry, max entry
```

We declare all these in a section `list operations`.

```
SECTION list operations.
    EXT min dir, max dir, min entry, max entry,
        get nmb, put nmb, get key, put key,
        get from entry, put to entry.

    CONST min dir = 1, max dir = 50,
        min entry = 1, max entry = 2000.

    LIST nmb (min dir : max dir).
    LIST key (min dir : max dir).
    LIST text (min entry : max entry).

    FUNCTION get nmb + >k + y> =
        y ":=" nmb "[" k "]".
     ...
ENDSEC list operations.
```

5.6 Storing entries

How should an entry be represented? We must somehow know where an entry ends. Two possibilities are:

- to prefix each entry by its length
- to store a special character at the end of each entry.

We choose the second alternative, using the end-of-line character (which cannot otherwise occur in a text) as a terminator.

Do we pack a text entry when a character is much smaller than a word, e.g. on a word-oriented machine? We choose not to perform this optimization initially, but to leave it until later when the need arises. We must carefully design the interface to be such that the initial realization is as simple as possible but later optimization is possible. First we want to get it right, later we may make it more efficient. We therefore choose a rather general interface for storing the characters of an entry.

```
ACTION first char + >c + key>
ACTION next char + >c
ACTION last char
```

The algorithms will make use of a hidden variable that gets its initial value through **first char** and is automatically incremented by each call of **next char**. Through this artifice, we avoid explicit computations of the positions of subsequent characters. We choose similarly for retrieving them

```
ACTION first char from entry + c> + >key
ACTION next char from entry + x>
```

In retrieval, the end of the entry will be recognized by the end-of-line character, so that those two actions are sufficient. Under this simple strategy, the text is the result of the sequence of calls

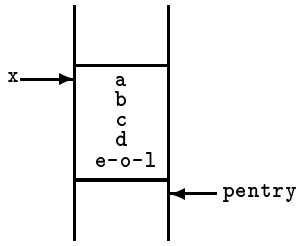


Figure 5.3: One entry

```
    first char + a + z, next char + b, next char + c,
       next char + d, last char.
```

How do we delete a text entry? We don't, we will just leave it in the entry table until (possibly) garbage collection or (more likely) the end of execution liberates the space.

We can now employ this interface in a section to read and write texts.

```
SECTION entry table adm.
    ABSTR read entry, list entry.

    INV first char, next char, last char, equal, eol char,
        read char, out char, first char from entry,
        next char from entry.

    ACTION read entry + key> - c:
        read char + c, first char + c + key,
           (equal + c + eol char, last char;
           read char + c, next char + c, *).

    ACTION list entry + >key - c:
        first char from entry + c + key, out char + c,
           (equal + c + eol char;
           next char from entry + c, out char + c, *).
ENDSEC entry table adm.
```

We have now defined a purely algorithmic interface for access to one entry, with implicit pointer movement. A first implementation of the underlying entry table, as straightforward as possible, is the following:

```
SECTION entry table.
    ABSTR first char, next char, last char,
        first char from entry, next char from entry.

    INV min entry, max entry, get from entry, put to entry,
        make, incr, eol char.

    VAR pentry.

    ACTION init entry:
        make + pentry + min entry.

    ACTION stack entry + >x:
        lseq + pentry + max entry, put to entry + pentry + x,
           incr + pentry;
           garbage collection, *.

    ACTION garbage collection: ?.

    ACTION first char + >x + key>:
        make + key + pentry, stack entry + x.

    ACTION next char + >x:
        stack entry + x.

    ACTION last char: +.

    VAR pfrom.

    ACTION pop entry + x>:
        get from entry + pfrom + x, incr + pfrom.

    ACTION first char from entry + x> + >key:
        make + pfrom + key, pop entry + x.

    ACTION next char from entry + x>:
        pop entry + x.

    PRELUDE init entry.
ENDSEC entry table.
```

With the writing of a few more sections, each short and simple, the reader can complete this example.

5.7 Handling packed entries

Let us now assume that the issue of space-economy in storing the entries has become important. The editor is complete and correct, but its entry-table takes up too much space. We must pack more than one character to a word.

We can distinguish two different methods of packing (see figure 5.4):

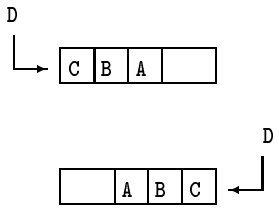


Figure 5.4: Packing characters in a word

- from right to left, adding characters at the right-hand side and removing them from the left. In most machine architectures this is easy to achieve.

  If it appears difficult to remove characters from the left, it is possible to remove them from the right but revert their order in a recursive printing procedure.

- from left to right, adding at the left and removing at the right.

In both cases the word may have to be padded in some way (e.g. with null-characters) to ensure that it contains the right number of characters and therefore can easily be unpacked again.

We may opt for either method, assuming two complementary algorithms

```
FUNCTION add a byte + >word> + >byte
FUNCTION remove a byte + >word> + byte>
```

During the storing of an entry we will maintain a one word buffer `word` filled with `count` characters, where `0 < count < bytes per word`. We will assume the character code to be such that our characters assume only values < 75 (enough for 2×26 letters, 10 digits and some special characters). This will allow us to put 5 characters into a 32 bit word, just to spite IBM, at the price of performing a multiplication or division by 75 in packing and in unpacking. If the reader prefers, he may change this section to conform to the usual 8 bit per byte standard. In this case the macros `add a byte` and `remove a byte` become quite simple and efficient.

```
SECTION packed entry table.
    ABSTR ...
    INV add a byte, remove a byte, ...

    VAR pentry, word, count.

    ACTION init entry: make + pentry + min entry.

    CONST one = 1, maxbyte = 75, bytes per word = 5.

    ACTION stack entry + >x: ... .

    ACTION first char + >x + key>:
        make + key + pentry, make + word + x,
           make + count + one.

    ACTION next char + >x:
        equal + count + bytes per word,
           stack entry + word, make + word + x,
              make + count + one;
           add a byte + word + x, incr + count.

    ACTION last char:
        equal + count + bytes per word, stack entry + word;
        next char + nil char, *.
```

5.8 Conclusion

The intention of this example was to show the design strategy leading to a structure of sections on different levels of abstraction, each with their specific tasks, separating out the application, function and representational issues.

The example can be carried somewhat further by establishing a module structure orthogonal to the layers, but is really too small for that. A more complete and realistic example is given in Appendix B.

We hope to have shown that a two-dimensional structure with explicit interfaces is a suitable vehicle for partitioning and directing thought and effort, of great help in more realistic design efforts.

Chapter 6

Syntax-Driven programming

In this chapter we will be concerned with the construction of parsing procedures in CDL2. In this way we will illustrate the notion of syntax-driven programming.

6.1 Context-Free Grammars and their notation

We start out from the well-known BNF-form for Context-Free grammars. A grammar for a fragment of the English language might, in this notation, look like

⟨sentence⟩ ::= ⟨subject⟩ ⟨verb⟩ ⟨object⟩
⟨subject⟩ ::= the man | the dog
⟨verb⟩ ::= bit
⟨object⟩ ::= the man | the dog | his wife

This grammar consists of four rules. The words enclosed between sharp brackets are the so-called *nonterminal symbols*. Words occurring without such brackets are to be taken literally, they are called *terminal symbols*. The curious sign ::= separates the *left-hand side* from the *right-hand side* of a rule. Rules with one same left-hand side can be taken together, separating the *alternative* right-hand sides by a vertical bar. Given a grammar for a language, we can generate sentences of the language, check whether a given string is a sentence of that language, determine the structure of a sentence and depict its structure.

6.1.1 Generating sentences

Following this grammar, we can generate sentences like: “the man bit the man” or “the dog bit the man”.

Starting from the nonterminal symbol ⟨sentence⟩, we repeatedly substitute for some nonterminal symbol one of its alternatives (doing one *derivation step*), e.g.:

⟨sentence⟩
→ ⟨subject⟩ ⟨verb⟩ ⟨object⟩
→ the man ⟨verb⟩ ⟨object⟩
→ the man bit ⟨object⟩
→ the man bit the man

This generation process stops when there is no further nonterminal symbol to be substituted for. We have then derived from the initial symbol ⟨sentence⟩ by repeated derivation steps according to the grammar a sequence of (zero or more) terminal symbols, a *sentence*. sentence

sentential form

At any point in the generation process we have a string of terminal and nonterminal symbols (a *sentential form*) from which we may choose any nonterminal for the next derivation step. The order in which derivation steps are performed is immaterial. In the example we have always chosen the leftmost nonterminal (it is a *leftmost derivation*).

6.1.2 Drawing syntactic structures

A pictorial representation of the derivation of a sentence, in the form of a tree whose root is the initial symbol and whose arcs represent the derivation applied, is called a *parse tree*. A simple example is shown in figure 6.1.

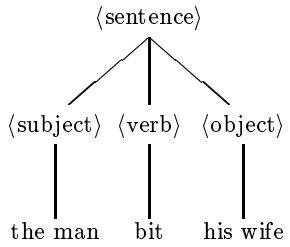


Figure 6.1: A parse tree

Notice that the parse tree is a notation which abstracts away from the derivation order.

6.1.3 Analysis

parsing
recognizing

The reverse of the derivation process is called analysis. We can try to *recognize* whether a given sequence of symbols is a sentence. Furthermore, if it is a sentence, we can *parse* it to build a parse tree, according to the rules of the grammar. Obviously, recognition is a less ambitious process than parsing, since it gives only a yes/no answer and does not attempt to build up a parse tree.

In the next section we will introduce a simple parsing technique.

6.2 Recursive descent parsing

Assume the following grammar, consisting of only one rule:

⟨sentence⟩ ::= a ⟨sentence⟩ | b ⟨sentence⟩ | #

This grammar generates the infinite language

$$L = \{\#, a\#, b\#, ab\#, ba\#, aa\#, \ldots\}$$

A naive recognizer for this language can be obtained by writing a CDL2 predicate

```
PREDICATE is sentence:
    is letter a symbol, is sentence;
    is letter b symbol, is sentence;
    is hash symbol.
```

This can be seen as a procedure yielding a Boolean value, which calls upon three other procedures (`is letter a symbol`, `is letter b symbol`, `is hash symbol`) which each recognize the symbol that their name suggests: `is letter a symbol` should have a realization like

if *next character of input* = "*a*"
  then *increment input pointer and yield true*
  else *yield false*

An input sentence like `ab#` would be recognized in the following fashion (in this trace the) sign before a name denotes calling the procedure of that name; the + sign denotes returning successfully, the - sign denotes returning with false).

```
> is sentence
 > is letter a symbol
 + is letter a symbol        a
 > is sentence
  > is letter a symbol
  - is letter a symbol
  > is letter b symbol
  + is letter b symbol        b
  > is sentence
   > is letter a symbol
   - is letter a symbol
   > is letter b symbol
   - is letter b symbol
   > is hash symbol
   + is hash symbol           #
  + is sentence
 + is sentence
+ is sentence
```

This simple example might lead one to believe that any Context-Free grammar can be recognized by a (recursive) combination of recognizers. This, unfortunately, is not the case.

Let us follow a trace of calls for the input `abx#`

```
> sentence
 > is letter a symbol
 + is letter a symbol       a
 > is sentence
  > is letter a symbol
  - is letter a symbol
  > is letter b symbol
  + is letter b symbol      b
  > is sentence
   > is letter a symbol
   - is letter a symbol
   > is letter b symbol
   - is letter b symbol
   > is hash symbol
   - is hash symbol
  - is sentence
 - is sentence
-  is sentence
```

The recognizer fails, but `a` and `b` have been read. Why is this a problem?

6.2.1 Backtracking

Some terminology: A predicate `x` is a *recognizer* for some nonterminal if, in case the remaining input starts with a terminal production of that nonterminal, the predicate will consume that

recognizer

production from the input and return true, whereas otherwise it returns false. An *exact recognizer* is a recognizer that, upon returning false, leaves the input as it was. Assume the following, even simpler grammar

⟨x⟩ ::= a b | c

Obviously,

$$L(x) = \{ab, c\}$$

We assume that we possess exact recognizers for `a`, `b` and `c` and construct the corresponding recognizer

```
PREDICATE x : a, b; c.
```

This naive recognizer will unfortunately recognize the language

$$L'(x) = \{ab, c, ac\}$$

which was obviously not our intention.

Two possible solutions offer themselves:

backtracking

- To use a different type of parser, which performs explicit *backtracking* to reset the input. It could work schematically as follows

  ```
  PREDICATE x:
      a, ( b; backtrack over a, c ).
  ```

  Such a parser may be extremely inefficient and, as we will not show here, for many grammars it will still allow unintended parsings. We will not pursue this alternative any further.

- To introduce error reporting.

  Upon looking more closely at the problem, we notice that once an `a` has been recognized, a `b` should always follow. If a `b` does not follow, we know the input is incorrect. We will have to report this fact to the outside world:

  ```
  PREDICATE x :
      a, (b; report error); c .
  ```

Our recognizer now has three possible outcomes:

- If it returns true without giving an error message, it has successfully recognized a sentence.
- If it returns false, it has not succeeded in recognizing a sentence and we know for sure that there was no sentence to recognize.
- We have added a third case, viz. that it returns with true after having given an error message. It has now successfully recognized an incorrect input as incorrect, which is equivalent to the second case.

  A more fundamental way of looking upon this solution is to notice that we have extended the language with a collection of erroneous strings, and that all erroneous strings will be reported as such.

We actually have now a three-valued recognizer. Of course the second and third alternative can easily be combined to give a two-valued exact recognizer.

6.2.2 Application to the previous problem

Let us extend the previous grammar with error productions.

```
PREDICATE is sentence:
    is letter a symbol,
       (is sentence;
       error + "incorrect sentence");
    is letter b symbol,
       (is sentence;
       error + "incorrect sentence");
    is hash symbol.
```

Another style for expressing the same is in the form of parsing predicates (whose name will conventionally start with `is`) and parsing actions (whose name will start with `should be`).

```
PREDICATE is sentence:
    is letter a symbol, should be sentence;
    is letter b symbol, should be sentence;
    is hash symbol.
```

This predicate is now an exact recognizer for the language we intended. For incorrect input it either fails (without consuming any input) or gives an error message (upon which the state of the input is immaterial). We can cause it to give an error message for all incorrect input by turning it into an action.

```
ACTION should be sentence:
    is sentence;
    error + "incorrect sentence".
```

The recognition procedures obtained in this way form a *recursive descent parser*, which will correctly work for grammars satisfying the so-called LL(1) restriction [ASU86].

6.2.3 When is an error alternative necessary?

For the sake of the discussion, assume a production rule

⟨x⟩ ::= ... ⟨p⟩ ...

with its corresponding recognizer

```
PREDICATE is x : ... , is p, ... .
```

In this alternative `is p` is a predicate. Obviously, before this predicate we can allow only tests and functions, since they have no side effects. After this predicate, we cannot allow the alternative to fail, so we can allow only actions or functions. At any rate, there may never be two predicates in one alternative, because that is sure to lead to a backtrack situation.

Let us give the name *activity* to any change in the observable environment. Of course what constitutes the observable environment (global variables, files or output media) is a matter of arbitrary human decision. In the context of parsing, we are mainly interested the activity of consuming the input to be parsed. activity

We will use the term *effect* for an activity upon success and the term *defect* for an activity upon failure. The backtrack philosophy of CDL2 is very simple: all defects are forbidden! effect defect

There is, as it were, a positive trace along which all intended computations lie. There should be no activities other than on this positive trace through the computation. Both the CDL2 compiler and the CDL2 LAB enforce this philosophy.

6.3 Sketch of a small interpreter

As an exercise we will sketch the structure of a small interpreter. In this process we will introduce other CDL2 constructs, so that the original syntactic motivation of the language becomes clear.

The interpreter will serve to recognize and execute straight-line programs with the following syntax:

⟨program⟩ ::= ⟨statement sequence⟩ **.**
⟨statement sequence⟩ ::= ⟨statement⟩ **;** ⟨statement sequence⟩ | ⟨statement⟩
⟨statement⟩ ::= ⟨assignation⟩ | ⟨write command⟩
⟨assignation⟩ ::= ⟨variable⟩ = ⟨expression⟩
⟨variable⟩ ::= a | b | c | d | e | g | h | i | j | k | l | m |
n | o | p | q | r | s | t | u | v | w | x | y | z
⟨write command⟩ ::= **!** ⟨variable⟩

The syntax of ⟨expression⟩ will be dealt with later (in 6.7). The top or driver for such an interpreter can take the form of a recognizer for ⟨program⟩s, which performs its computations as a side effect of the recognition process.

We obtain it by straightforward transliteration from the grammar:

```
PREDICATE is program:
    is statement sequence, should be end symbol.

PREDICATE is statement sequence:
    is statement,
       (is semicolon symbol, should be statement, *;
       +).

ACTION should be statement:
    is statement;
    error + "incorrect statement", ?.

PREDICATE is statement:
    is assignation;
    is write command.

PREDICATE is assignation:
    is variable, should be equals symbol,
       should be expression.

PREDICATE is write command:
    is write symbol, should be variable.
```

6.4 Reading ahead

We will have to implement a number of procedures for reading input. At each position of the input different symbols may stand, indicating different syntactic constructions. The recognition routines will have to read those symbols in order to look at them.

It is not a good idea to let each recognition procedure read a symbol, only to find out in many cases that it cannot use this symbol. We will use a systematic read-ahead method: we will have a special variable `symb` which at any point in time contains the first nonconsumed symbol of the input. Whenever we recognize a symbol, we read the next symbol of the input into this variable. If we do not recognize a symbol we do not change the value of the variable. This look-ahead buffer may be realised as follows:

```
VAR symb.   #first nonconsumed symbol#

ACTION next symbol:
     read symbol + symb.

PRELUDE next symbol.
```

Note that, initially, the first symbol is read into the look-ahead buffer. The following procedures provide an interface to the look-ahead buffer:

```
TEST ahead+>x:
    equal + symb + x.

PREDICATE is+>x:
    ahead + x, next symbol.

ACTION should be+>x :
    is + x ;
    report error + "expected" + x.
```

Using these, the previous example (in 6.2) can be completed to

```
PREDICATE is letter a symbol:
    is + code of symbol a.

ACTION should be letter a symbol:
    should be + code of symbol a.
```

6.5 Continuation of the interpreter

The interpreter deals with expressions, that have to be read and whose value has to be computed. We will assume the following syntax for expressions, similar to the one in PASCAL

⟨expression⟩ ::= ⟨term⟩ ⟨plusminus⟩ ⟨expression⟩ | ⟨term⟩
⟨plusminus⟩ ::= + | −
⟨term⟩ ::= ⟨factor⟩ ⟨timesby⟩ ⟨term⟩ | ⟨factor⟩
⟨timesby⟩ ::= × | ÷
⟨factor⟩ ::= ⟨primary⟩
⟨primary⟩ ::= ⟨variable⟩ | **(** ⟨expression⟩ **)**

We have somewhat simplified the rule for ⟨factor⟩.

The transliteration is not so simple due to the fact that the alternatives in ⟨expression⟩ start with the same nonterminal symbol ⟨term⟩. In fact the grammar in this form does not satisfy the LL(1) restriction.

We rewrite the rule for ⟨expression⟩ to two rules

⟨expression⟩ ::= ⟨term⟩ ⟨expression tail⟩
⟨expression tail⟩ ::= ⟨plusminus⟩ ⟨expression⟩ |

which has the effect of factoring out the common initial parts of the alternatives.

A similar transformation is to be applied to ⟨term⟩, introducing a ⟨term tail⟩. The resulting grammar is now indeed of type LL(1).

The transliteration to CDL2 of the rule for ⟨expression⟩ first leads to

```
PREDICATE is expression:
    is term, should be expression tail.

ACTION should be expression tail:
    is plusminus, is expression; +.
```

Eliminating the defect in the second rule leads to

```
ACTION should be expression tail:
    is plusminus, should be expression; +.

ACTION should be expression:
    should be term, should be expression tail.
```

Finally, we substitute the body of the last two procedures for their call and replace right-recursion by repetition.

6.5.1 Recognition of expressions

The resulting transliteration of the complete syntax for ⟨expression⟩ reads as follows:

```
PREDICATE is expression:
    is term,
       (is plus minus, should be term, *; +).

PREDICATE is term:
    is factor,
       (is times by, should be term, *; +).

PREDICATE is factor: is primary.

PREDICATE is primary:
    is variable;
    is constant;
    is open symbol, should be expression,
    should be close symbol.

ACTION should be term:
    is term;
    error + "term expected".

ACTION should be expression:
    is expression;
    error + "expression expected".
```

We have now obtained a recursive descent recognizer for expressions.

Such a recognizer is in itself a very uninteresting and dull algorithm, because it only (at best) answers with yes or no. While going through the tremendous effort of recognizing expressions, it might just as well do something useful. Keeping in mind that we are building an interpreter, we will try to compute the value of the expression recognized as a side-effect of the recognition process.

6.5.2 Interpreting expressions

We equip each of our recognizing predicates and actions with one output parameter, which will return the value computed.

```
PREDICATE is expression + value> - op - val:
    is term + value,
       (is plus minus + op, should be term + val,
          compute + value + op + val, *;
       ).


PREDICATE is term + value> - op - val:
    is factor + value,
       (is times by + op, should be term + val,
          compute + value + op + val, * ;
       ).


PREDICATE is factor + value> :
     is primary + value.


PREDICATE is primary + value> - place:
    is variable + place, get val + place + value;
    is constant + value;
    is open symbol, should be expression + value,
       should be close symbol.
```

The recognition actions must be such that they also yield a value in error situations:

```
ACTION should be term + value> :
    is term + value;
    error + "expression expected", make zero + value.
```

The existence of suitable recognition predicates `is plus minus` and `is times by` and an action `compute` is assumed.

6.6 Completing the interpreter

All the ideas described can now be combined to construct a program with the following overall structure (figure 6.2).

The driver will now not only recognize the syntax of the language but will also call the necessary semantic actions. In particular we have to modify the following algorithms (from 6.3):

```
PREDICATE is assignation - place - value:
    is variable + place, should be equals symbol,
       should be expression + value,
          put val + place + value.


PREDICATE is write command - place - value:
    is write symbol, should be variable + place,
       get val + place + value, write + value.
```

The following elementary semantic actions still have to be provided:

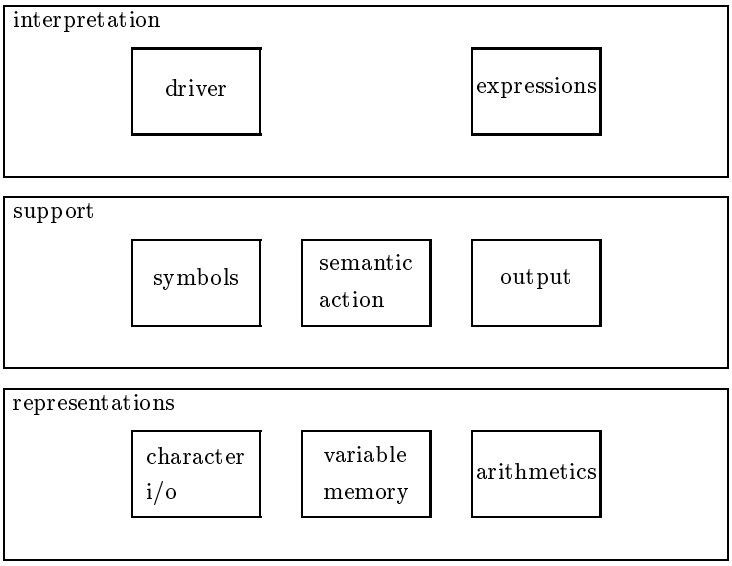


Figure 6.2: Structure of the interpreter

```
ACTION write+>value
ACTION put val+>place+>value
ACTION get val+>place+value>
ACTION copy+>place+>value
ACTION compute+>value>+>op+>val
```

as well as some other utilities and housekeeping actions. We leave their definition to the reader. This completes our outline of the syntax-driven design and implementation of a simple interpreter.

6.7 Translation of expressions

As an excursion we will now look at a somewhat more complicated problem: translating expressions into the so-called reverse Polish form [ASU86].

We define the reverse Polish form of an expression as follows: Let e be an expression and $P(e)$ its reverse Polish form. Then $P(x)$ is defined by

- If x is a variable or a constant then $P(x) = x$.
- If x is $e_1 \odot e_2$ where e_1 and e_2 are subexpressions and $\odot$ is an operator, then $P(x) = P(e_1)\,P(e_2)\,\odot$.

As an example, the translation of

$$a + 5 * b - (4 + c) * d$$

into reverse Polish form is

$$a\ 5\ b\ *\ +\ 4\ c\ +\ d\ *\ -$$

Notice that in the reverse Polish form no brackets are needed. That is an important reason for introducing it. There is an obvious relationship between the reverse Polish form of an expression and a program for a stack computer. Many pocket calculators (and even some mainframes) need a program like

```
push a
push 5
push b
  *
  +
push 4
push c
  +
push d
  *
  -
```

The operations `*`, `+` and `-` find their parameters on the stack. How then can we generate this reverse Polish form from the original infix expression?

We assume an `ACTION generate+>x` to perform the generation of one symbol of the output, and write

```
PREDICATE is expression - op:
    is term,
       (plus minus + op, should be term, generate + op, * ; ).

PREDICATE is term - op:
    is factor,
       (is times by + op, should be factor, generate + op, * ; ).

PREDICATE is factor - var - const + :
    is variable + var, generate push, generate + var;
    is constant + const, generate push, generate + const;
    is open symbol, should be expression,
       should be close symbol.
```

After the addition of suitable realizations of `should be term` and `should be factor`, we have obtained a transducer from infix expressions into reverse Polish form.

6.8 Precedence directed parsing

If we have more operators, we may of course continue in the same style, adding one recognition procedure for each priority level. On the other hand we can exploit the priorities of operators to make a simple general recognition scheme.

Let us look at one operator in the middle of an expression, the $*$ in

$$a\ **2 - b * c\ **2 + d$$

We assume the operators to have the conventional priorities, as shown in figure 6.3.

How far do the operands of that multiplication extend? The operands are obviously those expressions immediately to the left and to the right of the operator whose operators have a priority *not lower* than that of the multiplication.

In a hybrid of mathematical and CDL notation we might write:

$$\text{expr}_i : \text{expr}_{i+1}, (\text{op}_i,\ \text{expr}_{i+1},\ *;\ ).\quad \text{for}\quad 0 \le i < max$$

In case $i = 0$ we see that $\text{expr}_i = \langle\text{expression}\rangle$, whereas if $i = max$ then $\text{expr}_i = \langle\text{primary}\rangle$.

This idea immediately leads to the following solution in CDL2:

priority	operators
2	OR
3	AND
4	$=$ $\neq$
5	$<$ $>$ $\leq$ $\geq$
6	$+$ $-$
7	$*$ $/$ DIV
8	$**$

Figure 6.3: priorities of operators

```
PREDICATE is expression:
    is expr of priority + zero.

PREDICATE is expr of priority + >i - i plus 1 - op:
    equal + i + max,
       (is primary; -);
    add + i + one + i plus 1,
       is expr of priority + i plus 1,
          (is operator + i + op,
          should be expr of priority + i plus 1,
             generate + op, *; ).

ACTION should be expr of priority + >i:
    is expr of priority + i;
    error + "expression expected".
```

Upon recognition of an operator, its priority has to be deduced:

```
PREDICATE is operator + >prio + op> - p:
    operator ahead + p + op, equal + p + prio, next symbol.

TEST operator ahead + prio> + op>
    can be operator code + symb, make + op + symb,
       get priority + op + prio.
```

We have in this generalization captured all possible levels of priority, of course at the expense of somewhat more computing effort. The extra effort comes from the fact that we do not exploit the priority of the following operator directly, but try all possible higher priority operators.

A more intelligent and final version is the following

```
PREDICATE is expr of priority + >i - next i - op:
    is primary,
       (next op: operator ahead + next i + op,
          (lseq + i + next i, next symbol, incr + next i,
              should be expr of priority + next i,
                 generate + op, * next op;
          );
       ).
```

This is also much more compact than the original formulation.

In this chapter we have illustrated the notion of syntax-directed programming, touching upon some subjects from the field of compiler construction (parsing and semantic actions).

Chapter 7

Portability

In this chapter we intend to develop terminology, concepts and notations, suitable for thinking and talking about the portability of languages and programs.

7.1 Programs and translation

The reason we have computers is the fact that we want to solve problems. We solve a problem by programming the computer to have a specific input/output behaviour.

More abstractly, a *program* is a constructive realization of some function

$F : A \longrightarrow B$

which maps any argument in a given domain A (the "input") to an element of another domain B (the "output"; we disregard in this section the issue of non-determinism). A program in $\mathcal{L}$ realizing F differs from the mathematical function F by the fact that, given an implementation of $\mathcal{L}$ on some computer, we can physically provide some input x from A, run the program for some time (hopefully not too long) and obtain some output y from B such that $y = F(x)$.

By an *implementation* of $\mathcal{L}$ we mean a machine (abstract or concrete) such that it executes $\mathcal{L}$-programs correctly, i.e., given F and x in $\mathcal{L}$ it does compute y.

7.1.1 T-diagrams

We will at this point introduce a notational system [EAR70] suitable for the discussion of programs, as distinct from the discussion of functions.

We use a *T-diagram* (see figure 7.1) for a program, written in $\mathcal{L}$, that realises a mapping F T-Diagram

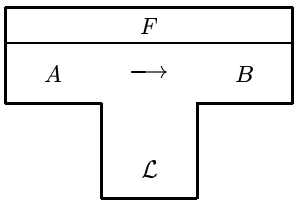


Figure 7.1: T-diagram of a program

from A to B.

We call

A = the input domain
B = the output domain
$\mathcal{L}$ = the object language

A program whose object language is $\mathcal{L}$ we will call an $\mathcal{L}$-program (e.g. a FORTRAN program).

An implementation of a language $\mathcal{L}$ can, in its simplest form, be depicted as a black box, let us say an $\mathcal{L}$-machine: $\boxed{\mathcal{L}}$. If we put the previous $\mathcal{L}$-program on top of the $\mathcal{L}$-machine, we obtain an (abstract) F-machine (figure 7.2).

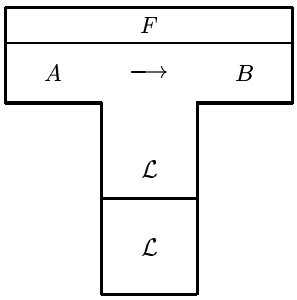


Figure 7.2: An abstract F-machine

When we feed it some input x in A we obtain some output in B, viz. $F(x)$.

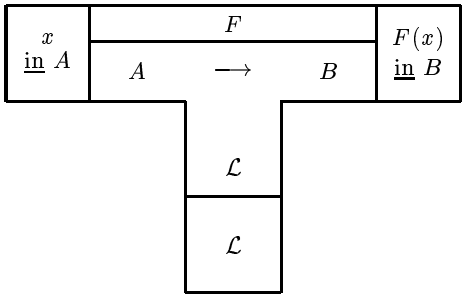


Figure 7.3: Application on the abstract F-machine

7.1.2 Compilers and interpreters

Of course, upon closer inspection an implementation of $\mathcal{L}$ will turn out not to be a concrete $\mathcal{L}$-machine, but to consist of a concrete M-machine with an $\mathcal{L}$-interpreter on top of it (figure 7.4).

Or again, it may contain a *compiler*. A compiler (figure 7.5) is a program with the special properties that

- its input domain is some programming language $\mathcal{S}$
- its output domain is another programming language $\mathcal{T}$
- it translates $\mathcal{S}$-programs into *equivalent* $\mathcal{T}$-programs.

The object language of a compiler is often called its implementation language.

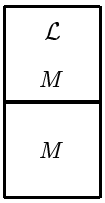


Figure 7.4: An interpreter on top of a machine

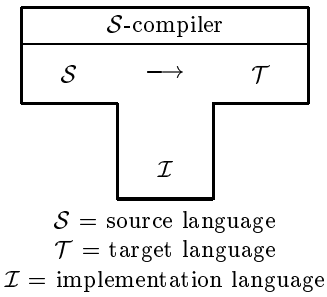


Figure 7.5: T-diagram for a compiler

7.1.3 Some further terminology

A compiler whose source language is $\mathcal{S}$ is called an *$\mathcal{S}$-compiler* (e.g., a COBOL-compiler)

A compiler whose target language is $\mathcal{T}$ is called a *compiler towards $\mathcal{T}$* (e.g., a compiler towards 360 Assembler). By transference, if the language $\mathcal{T}$ is amongst those available on some machine M, then a compiler towards $\mathcal{T}$ may also be called a compiler towards M (through $\mathcal{T}$) (e.g., a compiler towards 360 (through Assembler)).

A compiler whose implementation language is available on some machine M is called a *compiler on M* (e.g., a compiler on CDC Cyber).

Finally, a compiler whose implementation and target languages are each available on different computers may be called a *cross-compiler*.

As an example, the T-diagram in figure 7.6 depicts a PASCAL cross-compiler on VAX towards PDP11 assembler under UNIX.

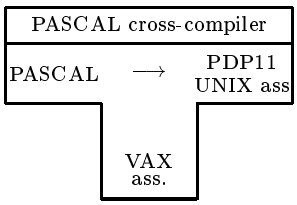


Figure 7.6: A cross-compiler

Using this compiler it is possible to execute PASCAL programs, provided the necessary machines are available (we abstract here from the by no means trivial problems in communicating between the different computers involved).

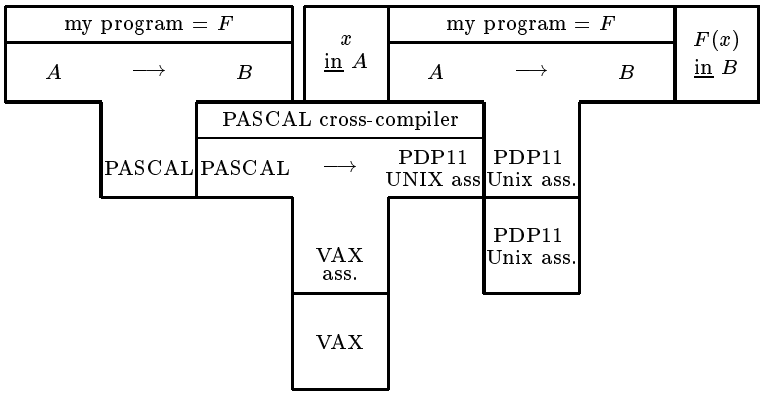


Figure 7.7: Using a cross-compiler

The T-diagram notation by itself does not actually enlarge our understanding of programs, but it serves to represent the obvious in a picturesque and helpful fashion.

7.2 Bootstrapping

The use of a formalized notation for translation processes like the T-diagram becomes essential if we want to discuss bootstrapping. In informatics this word is bandied about in various places, but we will mean specifically: the translation of compilers.

Let us put a PASCAL program, which has the function of translating BASIC-programs into UNIVAC assembler (a BASIC compiler in PASCAL), through a PASCAL compiler on UNIVAC (figure 7.8)

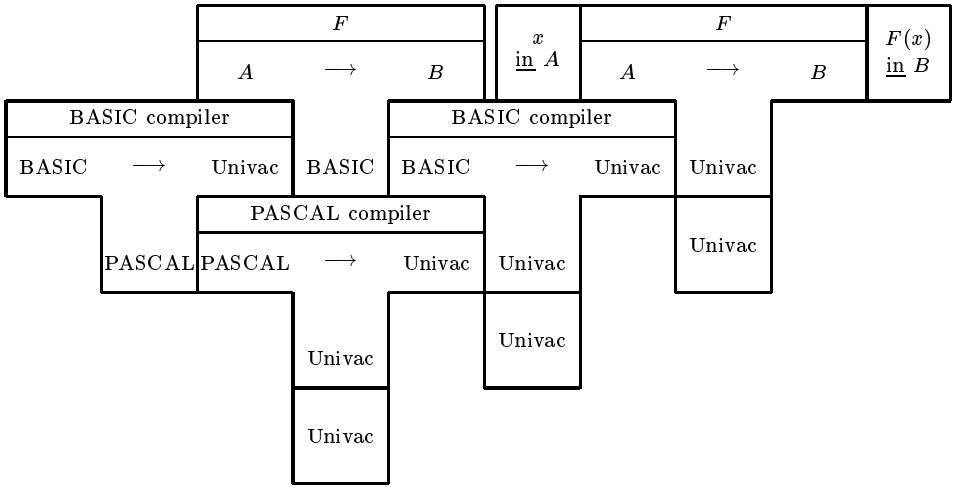


Figure 7.8: A bootstrap

Since the PASCAL compiler preserves the semantics of the programs it translates, the result is another BASIC compiler, but this time in the target language of the PASCAL compiler, viz. UNIVAC assembler, and therefore executable on a UNIVAC machine.

We have compiled a compiler by means of the PASCAL compiler. The original BASIC compiler, written in PASCAL, may well have been a beautiful piece of craftsmanship, but it was not directly executable, due to the absence of a concrete PASCAL machine. By contrast, the result of putting it through the PASCAL compiler is an ordinary compiler, executable on the UNIVAC machine.

There is some wonderful magic about getting an executable compiler from a non-executable one, which has led people to the name *bootstrapping*: pulling yourself up by the straps of your own boots. Somehow a whole compiler was pulled out of a hat. bootstrapping

The situation becomes even more interesting if we put the same BASIC-compiler in PASCAL through a PASCAL compiler on another machine, say the VAX.

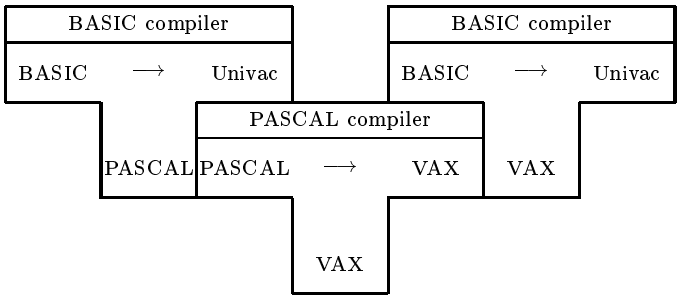


Figure 7.9: Generating a cross-compiler

The result is a BASIC-compiler running on the VAX but producing code for a UNIVAC machine: a BASIC-cross-compiler on VAX to UNIVAC. If we wish instead to get DEC20 assembler as a target language, we will have to modify either the PASCAL or the VAX version of the BASIC compiler. Writing a compiler in a high-level language by itself is not enough to make it portable.

A compiler, in order to be portable, has to be *retargetable*, i.e. it must be so designed that it can easily be adapted to different target languages, e.g. by a process of parametrization or by the adaptation of a rather small part of it, the concrete code-generator. The CDL2 compiler is an example of a retargetable compiler.

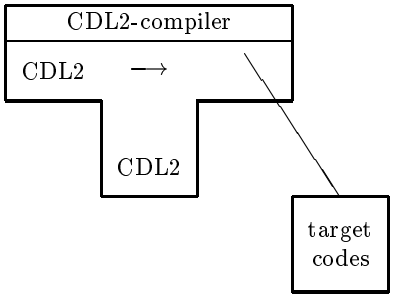


Figure 7.10: A retargetable compiler

As an example, let us discuss one possible technique for implementing some high-level language LAN. Being high-level, LAN should contain some small subset LAN* suitable for writing LAN compilers. Therefore we will write a retargetable LAN compiler in this subset LAN*. We assume some low-level target language ASM to be available on our target machine.

We will describe two techniques to port this LAN compiler to some specific machine, both based on retargeting.

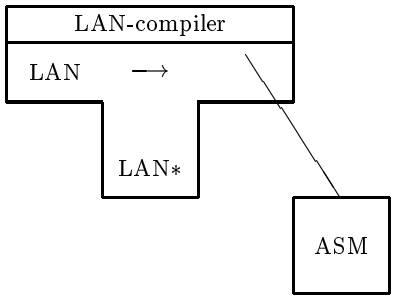


Figure 7.11: A retargetable LAN compiler

7.2.1 Pulling bootstrap

Assume we have produced a retargetable LAN compiler in LAN*. As a next step, we make in ASM, the assembly language of the machine to which the compiler is to be ported, a quick-and-dirty implementation of LAN* on the target machine, which is relatively easy for a number of reasons:

- It can assume that it will never be confronted with incorrect programs. In fact it may even assume it will never be confronted with any other program than the LAN-compiler.
- It can restrict itself to LAN* and need not implement any feature of LAN which is not absolutely essential. Since we have the extent of LAN* completely in our hands, we can stick to things which are easy to implement.
- It will have to run successfully only once, as we will see. Therefore it need not at all be efficient.

This gives a quick-and-dirty (interpreted) LAN compiler, which will probably be slow and inefficient.

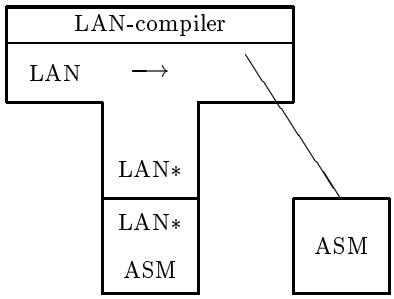


Figure 7.12: Interpreting the LAN-compiler

We then compile the compiler with itself (figure 7.13).

The result is a version of the LAN compiler, running on our machine which (if we have done our work well) will be much more efficient than the initial quick-and-dirty version. Through bootstrapping we have obtained an efficient compiler. Notice that in this *pulling bootstrap* all the work is done on the target machine.

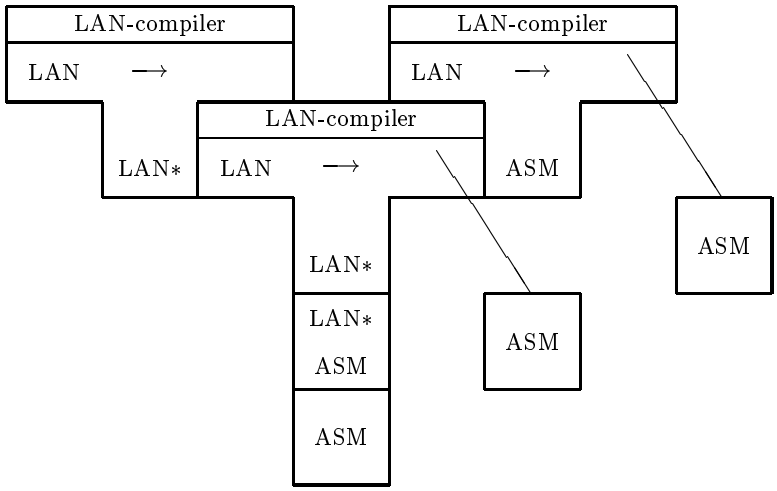


Figure 7.13: A pulling bootstrap

7.2.2 Pushing bootstrap

An alternative technique is to first make a LAN to ASM compiler on the object machine X. We do this by suitable parametrization of a retargetable compiler on X.

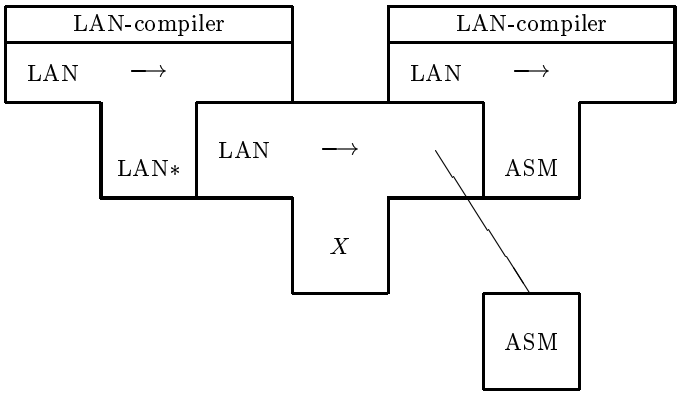


Figure 7.14: A pushing bootstrap

Next, this resulting compiler is carried bodily to the target machine in the form of an ASM program, which is then run as a retargetable compiler on the target machine, with as target ASM.

In this *pushing bootstrap* technique, most of the work is done on the object machine. Pushing bootstrap may be advantageous over the more popular pulling bootstrap in case the target computer is small and uncomfortable or difficult to operate (like most computers).

For software producers, producing various versions from one same portable compiler, pushing bootstrap is advantageous for security reasons (the source text of the original compiler is never stored in the target computer) and for maintenance (all versions can be maintained on one same object machine).

In academic circles, pulling bootstrap may be preferred because all the work (and all the responsibility) can then be left to the recipient of the software, working on the machine best

known to him.

The CDL2 technology supports bootstrap by cross-compilation techniques.

7.3 Achieving Portability

Portability

Intuitively, a program that is running on some machine is *portable* if it can be taken under the arm, carried to some different machine and run on it.

Two reasons for which portability is a desirable property of a program can be distinguished:

- There is an obvious *economic* advantage in being able to run one program on a variety of machines, avoiding repetition or duplication of work and broadening the marketing basis for software.
- There is also a possible advantage in *quality* of the software: Experience shows that by the time a program system is completed, debugged, stabilized and its user interface has been optimized, its object machine becomes obsolete. Many a high quality program, utility, compiler or operating system has, in this way, gone down the drain. A portable program has a longer expected lifetime, and is therefore worth spending more effort on.

The second argument is at least as important as the first, in the long term. In fact, a program seems to gain in quality when carefully ported from one environment to another, correcting some details in the process, like a fledgeling plant growing sturdier from repotting.

7.3.1 Universal programming languages

An obvious technique to achieve portability is to invent some *universal programming language* $\mathcal{U}$, implement it on all machines and write all programs in $\mathcal{U}$. The interest in universal programming languages that existed in the sixties has largely disappeared, for a number of reasons.

- The language must be *completely defined* in order to enable us to implement exactly the same language on different machines. There can be no fuzziness around the edges, no areas left to the discretion of the particular machine. There must be a complete formal definition, including all semantics, which is a useful basis for its implementations on different machines, allowing the proof of their equivalence. At the moment, we are technically not quite able te achieve this goal.
- The language must be *efficient* on all machines; it is not enough if it is available. Since efficiency of execution depends on the utilization of machine-dependent features, this is an important problem (see next section). This precludes the use of elegant and universal formalisms that cannot be realised efficiently on present day hardware (Turing machines, lambda-expressions) as well as beautiful and powerful universal programming languages with inherently expensive features (e.g. a heap with garbage collection).
- The language must be *suitable for all applications*, i.e. as suitable for expressing numerical computations as for artificial intelligence and business administration. It must be a truly general purpose language. Since there is no such thing as a general purpose, general purpose languages remain a pion's hope (as the experience with PL/1 shows).

7.3.2 Problems of machine dependency

Most programming languages are to some degree machine dependent, i.e. programs will behave differently on different machines. A non-exhaustive list of problem areas:

- the physical properties of the memory
  - the *size of the memory* (main memory and background memory) restricts the size of programs and data.
  - the *addressing scheme* of the machine may make some language features cheap or expensive (e.g. stacks and recursion, call by reference or by name) so that each language choice is suboptimal on some existing machines.
  - the *size of memory cells* as well as questions of alignment etc. can be hidden only at the expense of efficiency.
- the semantics of arithmetic instructions
  - the *size of integers*. Either one accepts whatever arithmetic the machine supplies, but writes the algorithms such that they take the machine dependencies into account (the road taken in the ALGOL languages) or one lets the user prescribe a size, like in PL/1 `BIN FIXED (31)`, but think of the inefficiency of `BIN FIXED (33)`!
  - the *radix of integers* is certainly not ten or two by a law of nature. Presumably three would be a better radix if we had other hardware components. Certainly the radix used decides the suitability for a particular machine.
  - the *representation of integers* may vary even between machines with the same radix and word-size: either one is faced with the `+0/-0` problem or the integers have a skew range.
  - the *size and precision of reals*. In principle one has the same choice as with integers, but the situation is complicated by the fact that one should also not provide more precision than desired: some numerical algorithms depend for their error behaviour and for their very convergence on the magnitude of rounding errors.
  - the *radix and rounding* of reals would not be a problem if a major manufacturer had not introduced a radix 16 floating point representation with systematically incorrect round-off, which it is nearly impossible to hide in the semantics of any language implementation.
- dependencies on architecture (machine speed; presence of traps and interrupts) and features of the operating system (e.g., concurrency). As an example, on most other hardware than the IBM mainframe series the full language PL/1 cannot be implemented efficiently, because its ON-conditions reflect typical properties of that hardware. Likewise it is quite inefficient to run C programs which copy strings with `while (*q++=*p++)` on machines which do not have auto-increment mode and byte addressing like the PDP11, for which C was designed. For the same reason, C forces one specific concrete representation of strings. C
- factors belonging to pragmatics, such as the quality of the user interfaces of implementations, the degree of media compatibility between computers, the access to computers and the cooperation from the systems staff have an enormous influence on the ease or difficulty of porting a program.

7.3.3 Solutions

The classical solution to portability problems is to ignore them: stick to one range of computers, keep that operating system alive and stamp out all revolutionaries.

The solution taken by manufacturers is to provide one universal concrete machine (through microprogramming) and to keep the architecture constant over a long period of technological change. IBM has been quite successful at this, even to the point that other manufacturers copy the architecture (the "plug-compatibles".)

For people and institutes that do not have the clout of IBM, it will be necessary to live with the problem. No clever universal programming language will serve all our needs, but we will have to use for each application a language suited to it.

Portability will have to be a property of the individual program, obtained at the expense of much care. It will be necessary to carefully concentrate the machine sensitive parts of the program in one part, which is reduced to a minimum as far as efficiency allows, while maintaining small, clear and well defined interfaces to the rest of the program.

The portability of programs is therefore a matter of portability of compilers for the useful languages plus a programming discipline.

This attitude leads to a less naive idea of portability: we will call a program portable if the amount of work necessary to bring it up on another machine is acceptable. The question, what amount is to be considered acceptable, depends on our aims and our finances. In some instances a complete re-implementation may be just the right thing to do.

7.4 The UNCOL-problem

The classical UNCOL-problem [CON58] is the problem of constructing, on each of m machines, compilers for n languages.

Using the T-notation, what is needed is the collection of $n \times m$ compilers $\begin{array}{|c|}\hline L_i \longrightarrow M_j \\ \hline M_j \\ \hline \end{array}$ $i = 1 \ldots n$; $j = 1 \ldots m$ where L_i stands for the i-th language and M_j for the j-th machine. How do we obtain these without going to the trouble of writing $n \times m$ distinct compilers? Thinking on this subject has evolved in a number of steps.

7.4.1 Step 1: The UNiversal COmputer Language

The classical solution is to devise an intermediate language $\mathcal{U}$ available on every machine. Actually this solution was already suggested in the late forties for a similar linguistic problem, viz. the translation problems in the United Nations, which could be solved by choosing a UNiversal COmmunications Language UNCOL, and performing each translation as a two-step process.

Given the language $\mathcal{U}$, we have to implement the following:

- $\begin{array}{|c|}\hline \mathcal{U} \longrightarrow M_j \\ \hline M_j \\ \hline \end{array}$ for $j = 1 \ldots m$
- For each language, one compiler in (how clever!) $\mathcal{U}$:
  $\begin{array}{|c|}\hline L_i \longrightarrow \mathcal{U} \\ \hline \mathcal{U} \\ \hline \end{array}$ for $i = 1 \ldots n$

Now we have to implement only $n + m$ compilers, rather than $n \times m$. In order to execute some program $\begin{array}{|c|}\hline A \longrightarrow B \\ \hline L_i \\ \hline \end{array}$ on the computer M_j one performs the process shown in figure 7.15.

This picturesque diagram represents a rather complicated two-stage translation process. Notice the bootstrap of the L_i compiler in $\mathcal{U}$, which takes place only once. Obviously, the efficiency of this scheme depends critically on the efficiency of the code generated by the $\mathcal{U}$-compiler.

The properties of UNCOL

In order for the process indicated in the previous paragraph to be possible, the intermediate language $\mathcal{U}$ must possess a number of somewhat conflicting properties, stemming from the fact that it is used both as target, object and source language.

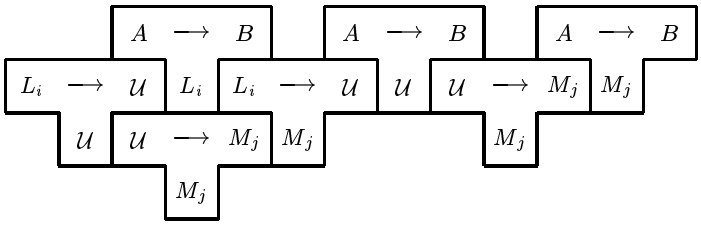


Figure 7.15: A two-stage translation process

- it must be *simple and general*, because it has to be implemented on every computer
- it must be *efficient*, because the efficiency of compilation and execution hinges critically on the efficiency of the compilers $\boxed{\mathcal{U} \longrightarrow M_j \;\boxed{M_j}}$ and $\boxed{L_i \longrightarrow \mathcal{U} \;\boxed{\mathcal{U}}}$
- it must be *rich and expressive*, because it must easily express the concepts of all languages
- it must be *problem-oriented*, because it will be used for the formulation of very intricate pieces of software, viz. the compilers $\boxed{L_i \longrightarrow \mathcal{U} \;\boxed{\mathcal{U}}}$
- it must be *precisely defined*, so that it can perform exactly the same on any computer.

In fact, it must combine all the good properties of high- and low-level languages.

Why is there no UNCOL?

It is obvious that the Universal Computer Language has not been found, otherwise there would be no portability problems. A number of technical reasons can be given: it is difficult to design and implement an UNCOL combining all the properties mentioned in the previous paragraphs.

But the more important reasons are sociological. Who actually wants UNCOL to be invented? The computer manufacturers? Certainly not, since they live by virtue of the fact that their products are compatible to some degree, but always in some respects better than (i.e. different from) their competitors. The software industry? They make a living selling the same software many times over in slight variations, and have trouble enough preventing unauthorized copying. The user? He actually likes his computer to be better (i.e. different). He'd rather use BASICPLUS than BASIC and is a sucker for more colours, higher resolution and more goodies than the standard product. And how about the Informatician? Well, he has different priorities. He is not interested in actually making something useful but in getting his Ph.D. If somebody would actually discover UNCOL, he would keep silent about it and try to get rich in the system software industry.

Of course the current widespread use of cheap personal computers is generating a new awareness for the virtues of standardization and compatibility. But this mostly shows up in the practice of cloning a successful computer, rather than in making more portable software.

A conclusion to be drawn is that, for various reasons, the idea of a universal intermediate language remains utopian.

7.4.2 Step 2: the universal abstract machine

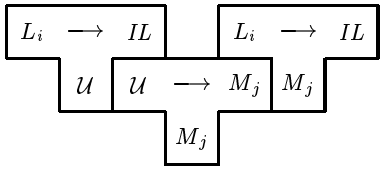
The sociological reasons we cannot change, but we can address the technical issue: the fact that human beings are not good in designing something universal. A partial solution to the technical problems mentioned lies in splitting UNCOL into two: *an implementation language* $\mathcal{U}$, and *a universal intermediate language IL*.

Since $\mathcal{U}$ serves only for implementing translators, and IL only for implementing the semantics of programming languages, this is indeed a simplification. In order to solve the UNCOL problem, we now have to

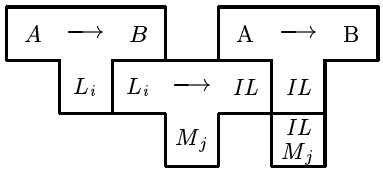
- design a compiler implementation language $\mathcal{U}$
- design a universal intermediate language IL
- write n compilers $\begin{array}{|c|}\hline L_i \longrightarrow IL \\ \hline \mathcal{U} \\ \hline \end{array}$
- write m compilers $\begin{array}{|c|}\hline \mathcal{U} \longrightarrow M_j \\ \hline M_j \\ \hline \end{array}$
- write m implementations of $\begin{array}{|c|}\hline IL \\ \hline M_j \\ \hline \end{array}$ as compilers, interpreters or even emulators.

In a sense, this has been done successfully by IBM, choosing IL = System/360 machine code, which is as near to a universal intermediate language as any language in this world, given the market share of IBM. But of course that is a very low-level choice of IL.

Using the components just described we bootstrap once



after which we can use any number of times



The efficiency of the result depends critically on the implementation of IL. But since only m are needed, these are worth some investment of effort.

Some examples of this approach

The reach for a universal intermediate language was a subject of research in the early seventies. A good example is the intermediate code JANUS [COL74] in the STAGE2 project.

In an experiment in 1974 with the CDL1 compiler, we compared the efficiency of directly generated 360 code with the code generated through JANUS. We found that the resulting code was about 50% less efficient in time and space. Of this loss of efficiency, about half can be ascribed to the fact that JANUS was too high-level (having a relatively expensive procedure mechanism with a display, whereas CDL procedures are much simpler due to the absence of block structure) and the rest to the fact that JANUS mismatches the 360 architecture. This second half might disappear if a very clever code-generator for JANUS on 360 was produced.

Various other intermediate languages have been used with more or less success in the implementation of more than one language, e.g. EM1 [TAN??] and the PASCAL P-code.

There is no denying that the use of one same intermediate code for the back end of a number of compilers (which is what the latter examples amount to) is an important factor in diminishing the cost of implementations and raising their homogeneity and quality.

But there is also a penalty to be paid, and the most efficiency-conscious implementations are still one-of-a kind. The point is that we are not able te define a truly universal abstract machine while retaining efficiency. We barely know how to define an abstract machine for a particular language!

7.4.3 Step 3: particular abstract machines

Any well-designed multipass compiler can be seen as a translator for the source language to some internal intermediate language *IL*, followed by a code-generator translating from *IL* to target code. (There may even be more than one such internal language.) In a picture:

Figure 7.16: Space reserved

An important phase in the design of such a compiler is the choice of an *IL*, such that

- it is machine-independent (the last machine-independent interface)
- it is well-mappable onto all target machines

Through a careful choice of this *IL*, we may obtain that

- the code-generator is simple for any target machine
- the translator is easily re-targetable.

This gives us another approach to the UNCOL problem, this time based on the idea to have a particular abstract machine IL_i for each language L_i. We now have to

- design a compiler implementation language $\mathcal{U}$
- design n intermediate languages IL_i
- write n translators $\begin{array}{|l}\hline L_i \longrightarrow IL_i \\ \hline \quad \mathrm{U} \\ \hline\end{array}$. Since these are few, they are worth spending some effort on. They may well include extensive machine-independent optimizations.
- write n implementations of n abstract machines $\begin{array}{|l}\hline IL_j \\ \hline M_j \\ \hline\end{array}$. Of course there are $m \times n$ of these, but the total amount of work may be two orders of magnitude smaller than the work of implementing so many compilers. Furthermore, effort can be saved by cannibalization. Finally
- write m compilers $\begin{array}{|l}\hline \mathcal{U} \longrightarrow M_j \\ \hline \quad \mathcal{U} \\ \hline\end{array}$ bootstrapped to $\begin{array}{|l}\hline \mathcal{U} \longrightarrow M_j \\ \hline \quad M_j \\ \hline\end{array}$. Since these are used heavily, it may be worthwhile to introduce extensive machine-oriented optimizations.

Summing up, we have now split UNCOL into:

- A compiler implementation language $\mathcal{U}$, which must be
  - well-suited for writing compilers
  - available on all machines
  - efficient on all machines

  but need not be suitable for any other purpose than the writing of compilers.
- For each language L_i an abstract machine IL_i which must be
  - well-suited for the semantics of L_i
  - easily implementable on any machine

  but need not be universal or machine-oriented.

This leaves research for many years:

- how to define an abstract machine for a specific language, and why so
- how to efficiently implement abstract machines.

UNCOL **is not a language but a technology.**

7.5 CDL2 as an UNCOL

It should not be difficult to see that CDL2 fits into the last model given. The language is well-suited to the writing of compilers. There exists a retargetable CDL2 compiler which can be parametrized

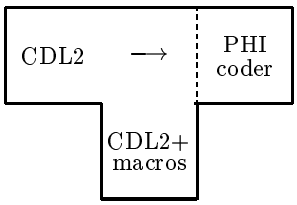


Figure 7.17: PHI-interface

with any target machine by essentially supplying a code-generator, according to the PHI-interface [HOL80]. Experience shows that making a CDL2 cross-compiler and porting the compiler to a new machine is 1 through 3 months of work, depending on the amount of adaptation done.

This compiler is available on a large number of machines, in historical order:

- IBM/370 under VM/CMS and OS/TSO
- SIEMENS 7000 series under BS2000
- CDC Cyber under Scope
- Honeywell Bull L60/66 under GCOS
- PDP11 under UNIX, RSX
- VAX-11 under VMS and UNIX

- MS-DOS machines
- a host of Motorola 68000 systems under various UNIXes

The target machines include all of those, plus

- interpreters on various microcomputers, with Z80, 8080 or 6502 processors
- Control Data CD180
- Prime
- microprocessors like Intel 8051 and Motorola 6809
- high-level languages like INTERLISP, PASCAL, C

There is a growing amount of software written in CDL2, (see section 1.2.3), which is of course the main measure of its success.

In each case, the application-dependent part (the IL_i) of the program consists of a collection of macro's (say between 30 and 100) which are mildly application-dependent and strongly machine dependent. With a reasonable amount of work these macro's can be adapted to a new target machine, after which code can be generated by a pushing bootstrap. Before that, the software can be tested out on the production machine. Finally, an adaptation step usually follows, to eliminate unnecessary inefficiency and to make the most of the target environment.

Indeed CDL2 has been used to produce software for machines for which the detailed specifications arrived only at the last stage of the project or in situations, in which the production and the target machine were separated by the atlantic ocean.

Chapter 8

Efficiency and adaptability

It is not hard to make a program highly portable, as long as efficiency is not an issue. On the other hand, it is much easier to make a program efficient if you can exploit the idiosyncrasies of one particular language implementation on one particular machine.

But how do you make programs both portable and efficient when it is impossible to exploit the peculiarities of any particular machine? That is the situation for which CDL2 was designed.

After the discussion of portability, in the previous chapter, we will now turn to the issue of efficiency and the techniques of adapting a program to a particular machine.

Just like portability, efficiency can best be obtained by designing the program in such a way that it is easy to adapt. First obtain correctness, then achieve the desired level of efficiency.

8.1 Efficiency

The efficiency of a program, i.e. the demand it makes on various resources for its execution, depends on many factors, of which only the last is related to the implementation language: efficiency

- The choice of algorithms and data structures
- The competence of the programmer in expressing her algorithms and data structures in the implementation language
- The inherent efficiency of the implementation language's implementation.

The first two points show the value of knowledge and experience. The third depends on the investment made in the support of the implementation language. It is to the last issue that we will address ourselves, viz. the relationship between efficiency and optimization.

8.1.1 On the value of optimization

In order to make programs, written in some implementation language, as efficient as possible, the implementation of the language will have to provide some form of automatic optimizations. It optimization should be noted that different things might be optimized:

- execution time
- space requirements
- time of the human programmer.

In the CDL2 project we take the point of view that it is the whole implementation process which has to be optimized, and that the traditional concentration on the technical issues of time and space is too narrow-minded. We want to achieve

- *avoidance of secondary behaviour* of the programmer, who is often willing to spend any amount of time to improve the efficiency of his program, and jeopardizes the correctness of his work without a second thought.
- *undoing the artifacts* of systematic programming, so that no important price is to be paid for the adherence to a programming methodology which is usually characterized by the introduction of many small procedures.
- *reduction of time and space* follows as a side-effect of the previous two considerations.

The main goal in automatic optimization is to *prevent the programmer from optimising.* Therefore the automatic optimizer should set out to perform the same optimizations that the programmer might be tempted to perform by hand, and perform them better than the human programmer ever can. The bag of tricks of the programmer includes the use of jumps to save repetitive pieces of code, the in-lining of procedures to save calls, the use of global variables for different purposes in different parts of the program, etc.

8.2 The CDL2 optimizer

The previous discussion explains why we chose to perform global optimization, which can be seen as source-to-source transformation, rather than local optimization on the target code. Furthermore global optimization can be implemented in a machine-independent fashion, so that it is effective for all target languages. Both the CDL2-Compiler and the CDL2 LAB perform global optimization of the control flow and of the data flow which are machine-independent, but controlled by information from the code generator about code sizes and target machine resources.

8.2.1 Control flow optimization

The particular control flow optimizations performed are the following:

- *Dead code removal.* When programming systematically, the program often contains procedures which are never called, or alternatives which cannot be reached due to statically evaluated tests. This holds especially in three cases:
  - data structures have been defined in a bottom-up fashion and the access algorithms are thus richer than needed.
  - test code has been introduced and is switched off later.
  - multiple versions of a program are integrated into one piece of source code and selected by static tests.

  The compiler removes this code, so that the programmer may not bother about it.
- *Opening of procedures* i.e. replacement of a call by a (suitably parameterised) copy of the body of the called procedure. There are two different criteria for this:
  - A procedure that is called from only one place in the program is an obvious candidate for opening. Such procedures often arise as a consequence of Top-Down programming, using procedures as a *refinement* mechanism.

refinement

  - A procedure whose body is not more complicated than a call is another candidate for opening. Such procedures arise as a consequence of a bottom-up *renaming* process, e.g.

```
ACTION define integer + >x :
   define + x + integer.
```

This opening-transformation is of great help in removing the artifacts of systematic programming.

- *right factorization.* Frequently, a number of alternatives in a procedure end in the same members, as in

```
ACTION print it:
   at new line, print + it;
   give new line, print + it;
```

The programmer's fingers are itching to write something like

```
ACTION print it :
   (at new line; give new line), print + it.
```

in an attempt to save code. Through the automatic right factoring, the same end is achieved without violating the CDL2 syntax. Another solution would be for the programmer to write explicitly

```
ACTION print it:
   make room, print + it.


ACTION make room:
   at new line;
   give new line.
```

which, through the opening optimization, would again lead to the same code.

- *uniting* of equivalent program points, and putting a jump instead of repeated code, as in uniting

```
... make + k + zero, get type + k + t,
      (equal + t + wanted type;
       incr + k, get type + k + t, *).
```

Here, the comma before the enclosed group is equivalent to the comma before the repetition operator `*`, since from both points the same computations follow. But since both are preceded by the same call of `get type + k + t` the same holds for the comma's preceding that call. The compiler will provide a jump after the call of `incr` to the point after the call of `make`, saving some code.

The major virtue of this optimization is to allow the programmers to work in a more relaxed fashion. Since such small structural transformations have no effect on the resulting code, the programmer learns not to bother about such details of coding. The same holds for

- *elimination of right-recursion*; which makes the choice between recursion and iteration in many cases immaterial. As an example, the procedure

```
ACTION print list + >p - x - q:
   get element + p + x, print + x, get next + p + q,
       (was nil + q, print text + ".";
        print text + ",", print list + q).
```

will be transformed into something resembling

```
ACTION print list + >p - x - q:
   get element + p + x, print + x, get next + p + q,
      (was nil + q, print text + ".";
       print text + ",", make + p + q, * print list).
```

- *combining hierarchies of procedures* (which are mutually nonrecursive) into one, and leaving multiple calling levels at once. This amounts to the replacement of a call of a procedure by a jump to that procedure. The resulting code is extremely tricky (any assembler programmer taking such risks would be fired) but of course completely safe.

8.2.2 Data flow optimization

The data flow optimization phase allocates registers and stack space globally (i.e. across the boundaries of mutually nonrecursive procedures). It uses a thorough analysis of the data flow and information about available resources of the target machine. The effect is threefold:

- local variables that do not coexist in time share the same place. This reduces the stack frames of (recursive) procedures. Since the aforementioned control flow optimizations have the effect of combining many procedures into one, this may save quite some stack space. At any rate it allows the programmer to use as many local variables as she wants, without any undue space penalty.
- The most often used variables are mapped onto registers, so that their access becomes cheaper to save time and code space. This register allocation is *not* directed by the programmer. Since certain machine instructions may require registers or register classes for their operands, this can be specified in the macros (see next section).
- Assignments and the passing of parameters are saved. Since the data flow optimization works globally, also parameters of procedures may be passed in registers. The copying is saved completely, if the actual parameter resides in the same register as the formal parameter of the called procedure. Passing a value to a procedure via a parameter is therefore in most cases less expensive than assigning it to a global variable and accessing that.

8.2.3 Macro writing

In conjunction with the data flow optimization explained above, the programmer may specify in each macro whether some parameter has to be passed in machine registers and if so, whether it should be in a specific register or in an arbitrary register administered by the data flow optimization.

The register specification must be contained in the first element of the macro body and usually takes the form of a target language comment so that it will be ignored on target level.

The register specification consists of the string `regs:` followed by a list of a *specification element*s separated by commas. A specification element can be one of:

`p<m>=<n>` or `p<m>=r`

which prescribe that the `m`-th parameter is to be passed in the `n`-th register, or in an arbitrary register, respectively.

`d=<n>`

designates that the macro destroys the `n`-th register

`destroy`

designates that all registers are destroyed by the macro (should only be used if unavoidable, since it negatively impacts data flow optimization).

```
make
```

designates that the macro is an assignment macro. No other specifications are allowed in this case, and that macro must have exactly one inherited and one derived parameter.

The number of registers available and the mapping of register numbers to machine registers depends on the code-generator used and is described in the corresponding manual.

Some examples (for VAX/VMS Assembler):

```
FUNCTION make + x> + >y=
    "$L;regs=make"
    "$L MOVL " y "," x.
```

tells the data flow optimization that this macro is an assignment.

```
FUNCTION get elem +>index + elem> :
    "$L; regs:p1=r"
    "$L   MOVL "list"[" index"]," elem.
```

will cause the parameter `index` to be passed in some register, so that it can be used as an index register in the `MOVL` instruction.

Obviously these issues are mostly of interest to the programmer responsible for some delicate adaptation. Usually, one makes use of predefined macros containing the necessary specifications.

8.2.4 Effectiveness of the automatic optimizations

Of course the effectiveness of the optimizations depends strongly on the programming style employed in the program to be optimized. As a rule, the control flow optimization of a program written by an experienced CDL2 programmer results in a reduction of about 50% in both time and space, whereas programs written in a dogmatic top-down fashion (or generated in that style by other programs) have shown a reduction of 75% (that is, a factor of four) in time and space. In some experiments with extremely efficiency conscious styles of programming we found that the global optimization still saved about 15%.

The effect of the data flow optimization is between 10% and 25%, dependent upon the target machine.

8.2.5 Optimization results

The CDL2 compiler produces code that, although it sometimes is locally sub-optimal, has a surprisingly good overall efficiency. Experiments have confirmed that CDL2 is at least as efficient in use as other implementation languages like C and BCPL, and that the code produced is a factor of 5–10 faster than that produced by conventional compilers for problem-oriented languages like PASCAL.

Obviously, an effective global optimization needs a thorough analysis of the global flow of control and data. Therefore the optimizer is a slow component (which can be switched off if so desired). The time spent in analysis and optimization is usually, however, more than offset by the reduction in assembly-time for the target code produced! For this reason, optimization is routinely performed also in early phases of the development of a program. As a side benefit, the analysis performed is the basis also for a semantic check of the program (see chapter 9).

8.3 Adaptation

adaptation

Porting is the activity of getting the software transferred to another hardware (virtual or actual) while retaining correctness. *Adaptation* then is the activity of making the software work at a prescribed level of efficiency while retaining correctness.

The efficiency of programs may in practice not be sacrificed to any great extent for the portability. To make a program efficient, it has to be *adapted* to the new environment after it has been ported. Under adaptation we subsume the changes necessary in the program to make it run efficiently in this new environment. These changes concern the realization of basic algorithms and data structures in such a way that good use is made of the machine resources.

Thus, following the pure porting step, an adaptation phase has to be performed which has to deal with tuning and balancing

- the memory requirements of the program
- the CPU time and input/output time requirements.

The adaptation phase must be based on insight in the resource usage of the running program, preferably on statistical data gathered by e.g. a program profile. This is especially useful for recognizing those points where the program spends most of its execution time. These critical points are candidates for adaptation concerning speed.

The adaptation process, and indeed the practicality of doing an adaptation heavily depends on the programming language chosen to implement the program to be ported.

Adaptation can be quite a messy process, unless it is a planned activity for which sufficient handles have already been included in the design of the program.

8.4 Portability versus adaptability

To get a program running on more than one type of machine, different strategies — as discussed in Chapter 7 — can be used. To achieve not only portability but also adaptability we must use a language which

- offers the advantages of a higher-level language,
- is translatable to a small set of instructions of an abstract machine, to achieve portability,
- gives direct access to the target machine, to achieve adaptability.

Our proposed solution is the use of an *open-ended* language. Open-endedness means that the language has no (totally) fixed kernel, but that the programmer has to implement (or extend) a suitable kernel by borrowing from a target environment (e.g. by use of a macro mechanism). By this means, the language is *semantically extensible*.

Using such a language, the programmer is able to write the largest part of her program in terms of the implementation language, and a small part, containing machine dependencies in terms of the target language, using the extension mechanism. This part has to be chosen carefully for each problem to be solved, considering the potential set of target machines. When the program is being ported to a particular machine, the programmer in principle has full access to its features.

A perfect adaptation has therefore not been made impossible by the language system, though it might require extensive rewriting in all parts of the program negating the benefits of portability. We have, however, practical experience that this approach can give the same degree of portability as the approach using universal abstract machines (see section 7.4.2), at the same time giving the same freedom for adaptation as assembly language coding.

8.4.1 Language requirements

An open-ended language to be used for porting and adapting must offer solutions for the following problems:

1. For porting an application program, the language has to be realised on various machines. To ease the implementation on a new target machine, the fixed part of the language should be mapped on a small set of abstract machine instructions. These must be easily implementable on all concrete machines considered for porting to.

2. To make good use of the semantic extension mechanism, the programmer must have precise knowledge about the relevant aspects of the translation to machine code on the target machine. For example, the programmer has to know, how data structures are mapped onto the target machine, which parameter mechanism is used, how procedures are implemented etc. Therefore the implementation language has to be simple enough and of sufficient conceptual clarity.

3. The language must offer a clear and well-designed interface to the open-ended constructs, to allow

   - easy linkage of these parts to the rest of the programs,
   - easy exchangeability of these parts when porting to another machine,
   - easy replacement of parts written in the object language by such written in the target language and vice versa as a step in adaptation.

From these requirements we see the need to keep the implementation language of restricted, though adequate complexity. Complicated data structures and their operations on them prohibit the programmer from formulating data access and manipulation algorithms of his own. Also, they make porting much more difficult. On the other hand, the language should not be too simple-minded. It must, after all, be well suited for all but the most machine dependent parts of the programs. It therefore should offer good control structures and abstraction tools to allow programming in a well-structured and readable way.

It should be immediately obvious that programming in an open-ended language, with full freedom to add semantic primitives, is not something for the beginner; it is work for an experienced programmer, with a deep understanding understanding of the application area, the problems involved in assuring portability and the possible target environments.

8.4.2 CDL2 as a solution

CDL2 is a an open-ended language which meets all three requirements listed above:

1. All language constructs of the fixed part of CDL2 (procedure declarations and calls, control structures, data declarations etc.) are translated to a small set of abstract machine instructions with fairly low complexity. The abstract machine is realised by an algorithmic interface to a concrete code-generator. This interface is well defined [HOL80] and is incorporated both in the CDL2 compiler and the CDL2 LAB.

   The semantics of these instruction are very simple and make it easy to implement them on any new machine. On most machines, more than half of these instructions map onto single machine instructions. Also, because of the nearly total lack of a language kernel in CDL2, the run-time system is extremely small and on most machines consists of not more assembly instructions than fit on one sheet of paper.

2. Because of the simplicity of the abstract machine instructions the programmer can easily understand how constructs relevant for her are mapped from the CDL2 source program onto the target machine. This concerns mainly the translation of data and the passing of parameters to the macros.

uniform referent principle

3. Calls to both types of algorithms — procedures written entirely in CDL2 and macros written in the target language — have the same form, obeying the uniform referent principle [ROS70]. Also the heads of procedure and macro declarations have the same form.

Programming in CDL2 forces the programmer to define data structures in an algorithmic way: the language gives him only the means to map data structures onto a contiguous piece of storage. The programmer has to give structure to this by formulating appropriate access macros (see 3.6 and 3.7). At first sight this seems needlessly laborious. We know, however, that most machine dependencies are concentrated in the access to data and their manipulation, concerning questions of word size, addressing structure (words or bytes, perhaps even bits) and the machine instructions to manipulate data. This design is therefore very appropriate for a language used to write portable software.

So on the one side, all (or most) machine dependencies are concentrated in the macros, leaving the rest of the program machine independent; on the other side, the programmer has full freedom to access every bit and byte, to adapt his program to the target machine. The macro part has to be rewritten when porting to a new machine. How difficult this will be, is highly dependent on the discipline of the one who wrote the program originally. At the very least, reasonable documentation of the macros must be available, because a large CDL2 program usually contains about 50 to 100 macros, luckily a high percentage of these are similar data access macros. Furthermore, a set of modules containing "standard" macros is available for nearly each machine, for which a code generator exists (see appendix B). This is then both the strong and the weak point in the CDL2 approach: porting can be made extremely easy by a good choice of macro interface, but also nearly impossible by a bad design and description of the macro interface.

8.5 An example of adaptation

To demonstrate the possibilities for adapting a CDL2 program to a particular target machine, a test program proposed by the IFIP WG 2.4 [WIC77] was used, which is intended to measure the quality of system implementation languages and their compilers. This program computes the transitive closure of a boolean matrix[1].

The algorithm works as follows (procedure names are given in parentheses): In an input matrix, whose elements initially all are reset (`clear matrix`), a number of elements are set (`fill matrix`). The indices of these elements are chosen by a pseudo-random number generator (`gen random`). The input matrix then is copied to an output matrix (`copy matrix`), to which Warshall's Algorithm (`warshall`) is applied. This process is repeated a number of times and the execution time is measured. A control output at the end of the program prints the two matrices (`print matrix`) and the number of elements set in both of them (`count matrix bits`).

First the program was written in CDL2 with macros in ALGOL 60. This program was made to be correct, rather than efficient. Therefore one matrix element was mapped on one machine word, even though it only can take on the values 0 or 1. After the program was correct, it was "ported" from ALGOL 60 to IBM/370 Assembler, by replacing the ALGOL 60 macros by equivalent assembler instructions. Still, one element of the matrix was mapped on one word. Then a series of adaptation steps was performed, trying to

[1] The $m \times m$ input matrix represents a graph of m nodes. A nonzero element (i, j) in the matrix means, there is an arc leading from node i to node j. In the transitive closure of that matrix, a nonzero element (i, j) means, that there is a way — of one or more arcs — from node i to node j.

- decrease the size of storage necessary to keep one matrix element (from words to half-words, bytes or bits); i.e. save space
- make good use of the instruction repertoire offered by the IBM/370 (storage to storage instructions working for up to 256 bytes, instructions to clear or copy parts of the working storage efficiently); i.e. save time
- keep values which are referenced often, in a variable instead of calculating them for each reference (on the IBM, CDL2 lists are addressed via a descriptor and a base register, thus an address calculation has to be performed each time a list element is accessed); i.e. save time

During the whole adaptation phase, we paid attention that

- the program was kept as near as possible to the original ALGOL 60 program (e.g. equivalent procedures, same kind and number of parameters);
- as few changes as possible were applied to the part of the program that has been written in the higher-level CDL2-constructs, as these had been shown to be correct, i.e. we concentrated on macros;
- the program should still work for different matrix sizes, i.e. only one constant definition should be modified for another matrix size.

To find those parts of the program spending most of the execution time, execution profiles were made. The profiling utility used, counts how often every procedure was executed and calculates the percentage of execution time spent in it. These dynamic data point out where further adaptation is profitable. A summary of the profile data and the execution time measurements for all adaptation steps is given in 8.5.3.

8.5.1 The porting step

The first version of the program was written with macros in ALGOL 60. After a short debugging phase, the program was ported to assembly by rewriting the bodies of all macros in /360 assembly language. The rest of the program was left unchanged.

The change-over from ALGOL 60 to assembler already resulted in a gain of 40% in time, which is largely due to avoiding doubled index calculation, one explicit in the program itself and the other implicit in the ALGOL 60 system; and to the ALGOL 60 parameter handling.

This ported program was taken as the standard which the adapted programs have been compared with.

8.5.2 The adaptation step

After the program had been ported to assembler it was adapted in five steps. The first two steps give existing macros more efficient bodies, the third and the fourth use semantic extension to introduce three new abstract instructions. The last step again introduces more efficient realizations of these instructions.

The first adaptation simplifies address calculations by moving a constant part to an initialization routine executed only once. This simple adaptation decreases the number of instructions necessary to access one matrix element and results in an execution time reduction of 12%.

The second adaptation reduces the matrix size by mapping one matrix element on a byte. This also simplifies address calculation, as the IBM is a byte oriented machine. There are furthermore single instructions available for testing, setting and resetting a byte in storage. In this way we

optimized both space and time requirements of the program: execution time was decreased by 19% and the size of the matrix storage by 75%.

A program profile made after this adaptation shows that the program spends 80% of its time in the procedure `warshall` and there 73% in the "or"ing of two rows of the matrix. Happily enough, on the IBM we can make use of the instruction "`OC`" (Or Character) which "or"s two operands of up to 256 bytes in main storage. Thus, using the semantic extension mechanism of CDL2 we replace the loop-wise "or" by a macro using this instruction. This decreases execution time by 75%.

A new execution profile shows that the relation between the percentages of execution time needed in the various procedures, changed dramatically: the "or"-ing of rows now only uses 8% of the total execution time, whereas `copy matrix` now uses 25% and `clear matrix` 17% (7.5% and 4.9% respectively before this adaptation was applied). The next adaptation step should therefore be concerned with these two procedures.

The introduction of two new macros to copy and to reset a matrix decreases execution by 50%. They use the instruction "`MVCL`" (MoVe Characters Long), which allows copying and clearing a matrix without a loop.

A new execution profile now shows that 52% of the time is spent in `warshall` (13% in the "or"ing of rows) and no measurable amount of time in `copy matrix` and `clear matrix`.

The next step should obviously be concerned with saving space, as it seems difficult to reduce execution time further. We therefore realise one element of the matrix in one bit, reducing the size of the matrix storage by a factor of 8. This was done by rewriting three matrix access macros, totalling about 30 lines.

An execution profile of the resulting program shows that 52% of the total execution time now is spent in `warshall` and nearly all of rest printing the matrix and generating the random numbers.

8.5.3 Gains from adaptation

The profiling utility of the CDL2 compiler proved a useful tool showing clearly where adaptations could be applied for greatest benefit. The following table gives the relative execution times of selected procedures, as percentage of the total execution time of the program:

procedure	ASM1	ASM2	ASM3	ASM4	ASM5	ASM6
warshall	80.2	80.4	80.5	28.8	51.8	52.1
oring of rows	73.7	73.8	73.8	8.1	12.8	7.3
clear matrix	5.0	4.8	4.9	16.7	0.2	0.0
copy matrix	7.7	7.5	7.5	25.5	0.0	0.0
fill matrix	0.2	0.1	0.1	0.5	1.4	1.1
gen random	0.3	0.4	0.5	1.8	2.4	3.3
print matrix	4.7	4.9	4.7	17.0	35.5	33.8
count matrix bits	1.3	1.2	1.2	3.9	7.8	9.2

The original program (ASM1) and the first two adaptation steps (ASM2 and ASM3) show very nearly the same relative times. This is to be expected, as the adaptations performed in these steps both have a global effect. The step producing ASM4 uses the CDL2 macro mechanism to introduce a new abstract instruction for the oring of rows. This changes the distribution quite a lot, as this adaptation has a very localized effect. A similar effect is to be seen in the next adaptation step (ASM5) where also two new abstract instructions were defined for `clear matrix` and `copy matrix`. The last adaptation (ASM6) left the distribution mainly unchanged as it again had a global effect.

The improvements in time and space reached by each adaptation are presented in the table below. The columns "in secs." and "in bytes" give the absolute values; the columns "%" are relative to the ASM 1 version. The execution times given are for 10 times repeated execution on an IBM/370-158 under MVS.

	execution time		data size		code size	
	in secs.	%	in bytes	%	in bytes	%
CDL/ALGOL 60	23.12	175	19600	100		-
CDL/ASM1	13.22	100	19600	100	2200	100
CDL/ASM2	11.56	87	19600	100	2128	97
CDL/ASM3	9.39	70	19600	100	2152	97
CDL/ASM4	2.42	17	5040	26	2112	96
CDL/ASM5	1.14	9	5040	26	1890	85
CDL/ASM6	1.10	8	840	4	1984	90

To allow a comparison, data gathered from realizations of the same test program in FORTRAN, ALGOL 60, SIMULA and IBM/370 Assembly on a IBM/370-158 are presented here:

	compile options used	exec time	bytes for matrix	bytes of code
ALGOL 60	opt=0	21.66	19600	
ALGOL 60	opt=1	14.24	19600	
SIMULA 67		21.23	19600	
FORTRAN word matr.			19600	
WATFIV		60.36		3576
IBM G		18.73		3680
IBM H	opt=0	18.83		3300
IBM H	opt=1	12.47		2866
IBM H	opt=2	4.17		3258
FORTRAN byte matr.			4900	
IBM G		18.97		3676
IBM H	opt=0	13.86		3100
IBM H	opt=1	10.98		2806
IBM H	opt=2	5.23		3020
best CDL2		1.10	840	1984
assembler		0.49	630	722

- Comments on the ALGOL 60 figures: The ALGOL 60 compiler used compiles very fast and generates medium quality code. Specification of the option `OPT=1` lets the compiler perform some optimizations in `FOR`-loops (removal of repeated address calculation) which result in a shorter execution time. The SIMULA 67 compiler generates about the same code as the ALGOL 60 compiler without this optimization.

- Comments on the FORTRAN figures: The WATFIV compiler is a student compiler meant for very fast compilation. Compilation time is indeed very short but the generated code is quite suboptimal. The FORTRAN G compiler generates medium quality code, which is a bit less optimal than that produced by the ALGOL 60 compiler. The FORTRAN H compiler without optimization generates about the same code as FORTRAN G, but by switching on different optimizations, the code becomes much better in time and a bit better in size. The option `OPT=1` results in an optimized register allocation; the option `OPT=2` additionally performs a thorough flow-of-control analysis which removes common subexpressions and index calculations out of the innermost loops.

- Comments on the assembler figures: Finally the algorithm was written in IBM/370 assembler language, in competition between a number of experienced programmers making use just of about all applicable coding tricks. The code of the resulting program is very short and the execution time is half of that of the best adapted CDL2 version.

The adaptation example in CDL2 and the results of the implementation of the algorithm in other languages, clearly show both the merits and the shortcomings of the presented method for obtaining portability and adaptability. The comparison of the figures for code length and execution time shows that a good automatic optimization as applied by the FORTRAN H compiler, results in an efficient program. But we also can see the limits of the classical optimization strategies when compared to the optimization by adaptation:

- Even the best optimization programs are not able to understand the semantics of the program thoroughly enough to take full advantage of very special machine instructions. The CDL2 programmer can easily make use of such machine properties in defining a new macro, i.e. a new abstract machine instruction.
- Packing the matrix one bit per element in most general purpose high-level languages is inefficient and very tedious work. Unless, of course, the programmer goes beyond the framework of his language by introducing assembly subroutines. In CDL2 it was half an hours work to perform this last adaptation step by changing the semantics of 5 macros while retaining the interface. Experience has shown this to be a predictable and safe operation.

On the other hand, the CDL2 approach has disadvantages too:

- Since portability and adaptability are achieved by the same mechanism, namely the macros, it is possible to make an adapted program unportable by tailoring the macro set used so specifically to one machine that a later re-writing is made impractical.
- The programmer has to know not only the implementation language but also the target language.
- The absence of the conventional loop control structures, and the unknown semantics of the user macros make it quite difficult to implement machine-sensitive optimizations in the compiler.

Chapter 9

Static semantic checks

In this chapter we discuss one of the more interesting aspects of the CDL2 technology, viz. the semantic checks performed by the compiler. This chapter is a rewritten and expanded version of [FEU78].

9.1 Introduction

By a *static semantic check* we mean the investigation of the plausibility of a syntactically correct program, based on an analysis of the flow-of-control and flow-of-data in the program, verifying those semantic properties of the program that can be verified statically, i.e. by inspecting the program without actually executing it.

This check may (classically) answer such questions as:

- do all applied identifiers have a defining occurrence?
- are, in all assignations, the types of source and destination compatible?
- is the value referred to by a reference compatible with the use made of it?

but for not so classical languages it may also determine:

- do all variables have defined values upon being used?
- are various parts of the program free from side effects upon one another?
- is the (concurrent) program guaranteed to be free of deadlocks?

and other questions pertaining to the execution of the program but (under specific conditions) answerable at compile time.

Such a static semantic check attempts to catch semantic errors which would otherwise be detected only dynamically, with the intention of

- shortening the test phase and the whole implementation process
- increasing the reliability of the product, and
- improving maintainability.

In the extreme case, static semantic checking should eliminate all incorrect programs, but we will argue that even a partial check can already be of enormous value.

In this chapter, we will be concerned with the feasibility of static semantic checking, and with those aspects of the design of a language that affect the effectiveness and the efficiency of such a check.

In particular, we will discuss the question how such a check can be made for an open-ended language, i.e. a language which allows in its programs the application of algorithms, objects and types defined outside the framework of the language itself. The techniques described here are not new, but both their actual feasibility and the particular design choices made in the open-ended implementation language CDL2 may be of interest.

9.2 Open-ended languages as SIL's

Programming in a language without a fixed kernel poses some problems, requiring sophisticated users. The usage of open-ended languages seems to be restricted to systems implementation lan-
SIL guages (*SIL*s), where they are of help in portability and adaptability: since the language kernel is not fixed, it can be chosen to suit the application and with one eye on portability. Also, open-endedness furnishes a systematic solution for the need expressed by machine oriented programmers to "have access to every bit and every instruction of their machine" in a SIL. Any general macro processor (e.g. STAGE2, ML/1) can be used as an open-ended language. Examples of partially open-ended languages there are many: in fact, library facilities and provisions for machine code procedures can be and have been used for semantic extension, adding features to a language that cannot usefully be expressed in the language itself (e.g. FORTRAN + assembly language patches).

MARY[CON77] is a partially open-ended language because of its facility for the inclusion of arbitrary instructions from the target machine; this facility is used in the "implementation prelude" to provide machine-orientation and thus enhance efficiency, and also gives the programmer access to the interrupt system, status bits and other odds and ends. The open-endedness of MARY was considered unsafe and undesirable by its authors, but has clearly been found unavoidable.

CDL2 is an extreme example of an open-ended language, because its kernel language is practically empty: all operations and data access primitives have to be borrowed form the target language by means of macros and can then be composed by built-in control structures and abstracted by procedures.

9.3 Semantic checks

We are concerned with the investigation by the compiler of the plausibility of syntactically correct programs, taking into account only those properties of the program which can be analyzed statically, i.e. without actually executing it.

Such a check is by no means the same as a proof of the correctness of the program, which would necessitate checking the program against some specification. In most cases, no suitable specification is available. The program can however be checked against itself: it can be checked for the absence of contradictions and patent impossibilities, by exploiting the redundancy present in programs. What then constitutes useful redundancy?

9.3.1 Useful redundancy

The prime example of redundancy in programming languages is the presence of mandatory declarations, including type specifications.

Although such declarations are technically not necessary (declarations by default as in FORTRAN or dynamic typing as APL can be implemented just as well) they provide for redundancy, allowing:

- the detection of most spelling errors in identifiers which, in languages with default declarations, might lead to surprising program behaviour whose cause may be hard to find by testing
- the static attribution of a type to objects, allowing the prohibition of assignations to unsuitable variables and of the applications of unsuitable operations.

That is, declarations introduce useful redundancy. Redundancy is not useful when it cannot be checked, (like e.g. comments), when it is not mandatory (like the optional **goto** symbol in ALGOL 68) or when it serves merely to check whether the programmer at one place in his program can repeat what he has said at another, as e.g. the destination list in an assigned goto in FORTRAN.

Redundancy is useful when it serves to specify abstractly the *intention* of the programmer, in such a way that it can be checked against his concrete actions elsewhere in the program.

In designing a programming language, useful redundancy should be introduced systematically. As an example, we will discuss the redundancies introduced into CDL2.

In the next sections we will discuss some limitations and problems in static checking.

9.3.2 Incompleteness of a static semantic check

A static semantic check can in general not be complete: the compiler knows only the static properties of the program and will therefore have to make worst-case assumptions ("if something can go wrong, it will").

In the first place, the knowledge of the flow-of-control may be insufficient. As an example, consider the following piece of program:

if $(p * p)$ **mod** $4 = 2$ **then**
  begin var a : **real**; $\{uninitialized!\}$
  $print(a)$ **end**

It is not reasonable to demand from the compiler that it knows that the square of an integer number modulo four can assume only the values 0 or 1 (did *you* know it?), and that therefore the condition always yields false. The semantic check should report a potential error, even though the clever programmer might have known that the error cannot occur.

In competently written programs, such cleverness should be infrequent, and since the behaviour of programs depends critically on their input, we will have to assume that all paths through a program are taken, and therefore have to report all potential errors. We will take the point of view that programs which are not demonstrably correct are in error. This may be unjust, especially for some tricky programs. But from an engineering standpoint it makes good sense, since a program which is not obviously correct may be a source of pitfalls in maintenance. A serious program, like Caesar's wife, should be above suspicion.

Apart from the lack of knowledge about the dynamic flow of control, another reason for incompleteness of static semantic checks may be the lack of knowledge about the identity of variables.

In order to answer e.g. the question "does each variable have a defined value upon being used", the compiler has to know what variables are assigned to. We will term a variable x an *alias* for a variable y if an assignment to y affects the value of x, as in the ALGOL 68 program

alias

begin real x;

procedure $f(y)$; $y := 0$;
procedure p; $f(x)$;
procedure $t(z)$; z;
$t(p)$;
$print(x)$
end

Whether questions about identity of variables are statically decidable depends on the presence in the language of such alias-makers as the FORTRAN `EQUIVALENCE` statement, the PASCAL pointer assignment or the ALGOL 68 identity declarations of the form

ref amode $x = y$

To make matters worse, aliasing may be dynamic as in

ref amode $x = y1[i]$

making it under circumstances impossible to decide statically whether the array $y1$ has been properly initialized. The systematic avoidance of any dangerous aliasing has been the primary design goal of the language EUCLID [LAM77]. In [LAN73] it is discussed in the context of ALGOL 60 how the presence or absence of certain language properties may affect the static decidability of specific properties of programs; in particular this article shows the necessity of full specification of formal parameters.

9.3.3 Incompleteness caused by open-endedness

Open-endedness of the language is another cause of incompleteness of the static semantic checks. Fundamentally, the semantics of borrowed algorithms is outside the scope of the check, and any knowledge about the effect of such algorithms has to be introduced explicitly by a specification which is the responsibility of the programmer. Without such a specification, it would be impossible to write meaningful programs. The correctness of the specification itself cannot be decided within the framework of the language, but has to be taken on trust.

As an example, the specification of a library routine

proc (**real**) **real** sin = **fromlib**(`"ASIN"`)

allows a modicum of security in calling that routine, provided the specification adequately and correctly captures the relevant properties of the corresponding library routine. It is definitely more revealing than just

```
EXTERNAL FUNCTION ASIN
```

or even

```
EXTERNAL ASIN
```

In designing an open-ended language, the necessary redundancy has to be carefully tailored in, in order to minimize the unavoidable risks.

Specifications of a borrowed algorithm may include:

- its logical effect, given in the form of logical relationships or algebraic axioms (not pursued in this paper)
- the type of its result

- the number, order and types of its parameters
- its calling mode (e.g. an algorithm borrowed form PL/1 may have to be called differently than one borrowed form LISP)
- the presence or absence of side-effects.

The obvious way to include such specifications into a language is to demand the writing of a heading for each algorithm to be borrowed.

Whether the semantics of the algorithm borrowed fits the heading is lastly the responsibility of the programmer writing the algorithm, who will have to program in a disciplined fashion.

Open-endedness should be used to introduce a small number of primitives, whose relevant properties are specified as precisely as possible, and *not* for the promiscuous spread of code inserts all over the program, demanded by ill-informed assembly programmers as a precondition for reluctantly writing their programs in a systems implementation language.

9.4 Static semantic checking in CDL2: a case study

As an illustration of the ideas and problems treated in the previous sections, we will discuss the semantic check implemented for CDL2. We will first review those of its properties relevant for the semantic investigation, in order to show how decisions taken in the design of the language influence the feasibility of static semantic checks.

9.4.1 Program Structure

Apart from constructs for modularization and information hiding, which are not discussed here, CDL2 programs consist of a collection of declarations, followed by a sequence of calls. The objects declared are:

- *procedures* with parameters and local variables
- *macros* that serve as a means for semantic extension by borrowing from a target language
- *variables;* named *constants;* and fixed size *arrays.*

The macros and procedures together will be termed the *algorithms.*

At the end of the program there are one or more *root calls*, which serve as start calls for the program. In addition, *prelude calls* may be defined, which are performed before the first root call and serve for initializing. *Postlude calls*, which finalize the program, are called after the last root call. Preludes and postludes are not essential for a CDL2-program, but facilitate such tasks as the opening and closing of files, initializing of stack pointers etc. — routines of a low level of abstraction, which otherwise would have to be passed through the whole hierarchy to the root.

9.4.2 Procedure structure

All procedures yield a Boolean result which is used to guide the flow of control. The body of a procedure consists of a number of *alternatives*, interconnected with the "McCarthy *or*" (i.e. not evaluating the second operand if the first yields *true*). An alternative consists of a number of calls of algorithms (the *members*) interconnected with the "McCarthy *and*" (i.e. not evaluating the second operand if the first yields *false*). alternative member

The *grouping* of a number of alternatives by enclosing them between brackets, a *repetition* operator and *leave* operators complete the control structures. The resulting algorithm structure could flow graph

hardly be more concise. It may be modeled by a binary graph [FEU75]. Furthermore, due to the absence of jumps, it has the properties claimed by Dijkstra [DAH72] for structured programming. The flow graphs are "reducible" [ALL70, LAM77] and easily analyzable and optimizable.

9.4.3 Specifications

The basis for the semantic checking possibilities in CDL2 are the concise specifications of attributes of each algorithm with regards to its result, its global effect and its parameters.

9.4.4 Specification of the result

The body of an algorithm may be structured such that the algorithm always returns *true* (it always *succeeds*), or it may be such that it can also return *false* (it may *fail*). The programmer specifies for each algorithm whether it (according to her) may fail — this introduces redundancy. For a macro, such a specification is essential, since the desired information cannot be deduced automatically. For a procedure, this specification is redundant. The consistency of the calls and the procedure specifications throughout the program is checked in terms of the specifications of the algorithms.

9.4.5 Specification of the global effect

environment

effect
defect

Let us define as the *environment* of the program the set of all global variables, arrays and input/output files. A change to this environment is termed an *activity*. Any algorithm may perform an activity upon success, a so-called *effect*, or upon failure, which we then call a *defect*. As a consequence of the original idea of considering all procedures as parsing procedures, we consider defects as unwanted side-effects. In CDL2 defects are forbidden and effects must be specified explicitly.

In connection with the result attribute, we distinguish four classes of algorithms:

test

test: failure possible, no effect

predicate

predicate: failure possible, effect

function

function: failure impossible, no effect

action

action: failure impossible, effect

9.4.6 Specification of parameters

parameter
copy

restore

affix

In CDL2, *parameters* are passed via a copy/restore-mechanism, i.e. on each call, local variables are set up, which may be initialized with the value of the corresponding actual parameter (*copy*) and whose value may be passed to the corresponding actual parameter after elaboration of the procedure (*restore*). The restoring takes place only after successful elaboration of the algorithm. Upon failure, no restoring is done. It must be specified whether the parameters are copied on entry, restored upon success, or both. Furthermore, a procedure or macro may have *local* variables. Parameters and local variables together are termed *affixes*.

Each affix has one of the following four types according to its specification:

inherited: copied, not restored

derived: not copied, restored upon success

transient: copied, restored upon success

free: not copied, not restored (local variable).

It should be noted that in CDL2 only simple variables and constants can be used as actual parameters: neither arrays nor procedures can be passed as parameters. These limitations ensure that the alias problem for simple variables is decidable.

9.4.7 Visibility rules

There are just two scope levels in CDL2 on which variables may be declared. The *program level* consists of the (global) variables, constants and arrays, whose extent is the program. Restrictions on visibility are imposed by a partition of the programs into *layers* and *sections* (see chapter 4). Variables and arrays can only be accessed in the section in which they are declared, and they cannot be passed through an interface.

On the *algorithm level*, affixes are declared whose scope is the algorithm. Their extent is the incarnation of the procedure or macro that they are local to. There is no nesting of procedure declarations, there is no passing of procedures as parameters, there is no block structure within procedures.

9.5 Local semantic check

The local semantic check deals with one procedure at a time, checking its body against its heading, i.e. checking the programmer's actions against his intentions.

The body of the procedure is represented by its *control graph* (see sections 2.3 and 9.5.1). For every affix and every node of the graph an *abstract value* is calculated denoting properties of the values the affix may take at execution time upon reaching the node (see 9.5.2). The environment and changes to that environment are modeled by one extra pseudo affix with type *global*. Using these terms, we may formulate certain conditions on the use of affix values, which are to be checked (see 9.5.3).

9.5.1 The control graph

The nodes of the control graph are the members of the body of the procedure, i.e. calls for algorithms. If the algorithm called may succeed of fail, two edges leave the member, on labelled t and one labelled f (the *true-edge* and the *false-edge*). If the algorithm called cannot fail, it is only left by a true-edge. All members of one alternative are interconnected sequentially by their true-edges, while all false-edges point to the first member of the next alternative. The true-edge of the last member of each alternative points to the *true exit*, while the false-edges of all members of the last alternative point to the *false exit*. This structure may be modified by the use of operators for repetition and leave and by grouping.

Each member corresponds to a node in the graph. The node corresponding to the member reached upon entering the rule, is called the *entrance*. The *true-successor* of a member m is the node pointed to by m's true-edge, its *false-successor* is the node pointed to by its false-edge.

9.5.2 Abstract values

The *abstract value* of an affix denotes properties of the set of values the affix may take at execution time. The properties we are interested in, are:

- is the value *undefined*?
- is the value obtained by copying the actual parameter before elaboration of the rule (value *inherited* from caller)?

- is the value obtained by some call in the body of the algorithm (value *derived* from called), and if so, by which call?

Since some paths through the program graph may join, more than one of the cases above may in general occur at a node. Thus the abstract values are modeled by sets which are constructed by union from the following elementary abstract values:

$\{undef,\ inherited,\ der_m \mid m \text{ member of the procedure }\}$

We say a value is *assigned* to an affix a by some member m if m has that affix at a derived or transient position. A value is assigned to the pseudo affix modeling the environment if m has some global variable at a derived or transient position or if m is a call for an action or predicate (thus, all global variables are lumped together).

An affix is *used* by a member m if m has the affix at an inherited or transient position. An affix is used by the true exit if it has the type *derived* or *transient*. The pseudo affix is used by every node. The calculation of the abstract values AV may then proceed iteratively as follows, using well-known techniques (see e.g. [COU77]).

1. For all nodes n and for all affixes a: $AV(a,n) := \emptyset$.
2. For all affixes a with type free or derived: $AV(a, entrance) := \{undef\}$

   For all affixes a with type inherited, transient and global:

   $AV(a, entrance) := \{inherited\}$.
3. We now have to walk through the graph starting at the entrance and performing calculations of the abstract values at each node. In the case of loops, parts of the graph will have to be considered more than once; the walk ends, if in a step no abstract value is changed. Termination of the algorithm is guaranteed by the fact that abstract values are only changed by the addition of elements.

   Upon passing a member m the following calculations have to be done for every affix a:

   $AV(a, falsesucc(m)) := AV(a,m) \cup AV(a, falsesucc(m))$

   a is not assigned to by m

   $AV(a, truesucc(m)) := AV(a,m) \cup AV(a, truesucc(m))$

   a is assigned to by m

   $AV(a, truesucc(m)) := der_m \cup AV(a, truesucc(m))$

9.5.3 Conditions on the use of abstract values

Using the abstract values calculated above and some additional information obtainable from the graph, the following conditions are checked:

- no undefined value may be used neither by assigning it to the input parameter of a called algorithm nor by restoring it upon success. If a node n uses an affix a, then

  $\{undef\} \cap AV(a,n) \stackrel{!}{=} \emptyset$

  The sign $\stackrel{!}{=}$ is to be read as "must be", and $\stackrel{!}{\neq}$ as "must not be".

- every value that is not undefined should be used somewhere, otherwise its computation was superfluous.

  We may model this by constructing for each affix a *possibly used abstract value* PAV, which is the union of all abstract values of the affix omitting $\{undef\}$. Then we can construct the *used abstract value* UAV as the union of all abstract values which are used.

$PAV(a) := \cup_n(AV(a,n)) \setminus \{undef\}$

$UAV(a) := \cup_n(AV(a,n)) \mid n \text{ uses } a$

$PAV(a) \stackrel{!}{=} UAV(a)$

- no value, that has been created by an assignment in one alternative, may be passed to any alternative following it (*independence of alternatives* within a procedure body).

  $AV(a,n) \cap \{der_m \mid m \text{ is in a previous alternative}\} \stackrel{!}{=} \emptyset$

- a transient affix must obtain a value somewhere. For all affixes a with type *transient*:

  $AV(a, trueexit) \stackrel{!}{\neq} \{inherited\}$

- no activity may be performed upon failure (defect). For the pseudo affix *global*: defect

  $AV(global, falseexit) \stackrel{!}{=} \{inherited\}$

- if the procedure has one of the types `ACTION` or `PREDICATE`, it must have an effect. For the pseudo affix *global*: effect

  $AV(global, trueexit) \stackrel{!}{\neq} \{inherited\}$

- if the procedure has one of the types `FUNCTION` or `TEST`, it must not have an effect.

  $AV(global, trueexit) \stackrel{!}{=} \{inherited\}$

9.6 Global semantic check

Due to the large amount of time and space which would be necessary for a complete analysis, the control over the use of global variables will not be made as tight as the control over affixes. It is checked, whether undefined values of global variables may be used somewhere. Notice that this check is concerned with simple variables only, since access to composed variables (e.g. arrays or structures) does not belong to CDL2 but is borrowed, and can therefore not be checked.

9.6.1 Control over initialization

Some languages (e.g. BCPL) force the programmer to initialize all global variables when declaring them. For CDL2, this does not seem to be the right way, since the variables may then only be initialized by constants or by previously defined variables.

Firstly, this is sensitive to the specific order of the declarations, which we would like to avoid. Secondly, even though for some kinds of variables the static initialization may work (initialize with zero or nil), for others they make no sense (how to initialize a global variable that is to hold some attribute of the current element of a huge data structure?).

Even if in such a case the programmer initializes the variable with an "impossible" value and checks for that, he has to perform run-time tests in order to find out if there exists some path in his program leading to the use of this "impossible" value. The compiler can carry a big part of this work on its shoulders, by testing statically whether a variable may be used without initialization — provided the language does not enforce static initialization in those cases where this makes no sense.

9.6.2 Applicability to CDL2

CDL2 has some properties that simplify a global check:

- the impossibility of exporting variables from one section to another and the parameter mechanism by copy/restore decrease the number of global variables to be treated at any time.
- most of the variables — at least those used throughout the program — are initialized in the prelude part and thus need not be considered in the main part of the program.
- the structure of CDL2 programs as a hierarchy of procedures gives rise to a rather efficient "Bottom-Up" analysis hardly applicable to languages with a more complicated structure.
- due to the copy/restore parameter mechanism and the absence of formal procedures, the alias problem is for simple variables decidable, so that the flow of data can easily be analyzed.

The global check could be extended to keep track of not only the defined and undefined values, but also of the range of a (defined) value. The mechanism should not be greatly different (see [COU77, KIL73]).

9.6.3 The method

For every procedure, it is possible to calculate the set of variables that must be initialized before calling it. Let us call this set the *use* of the procedure. It is also possible to calculate the set of variables that are initialized by the procedure: this is the set of global variables that obtain values on all possible paths through its body to the success exit, but which are not contained in its use. This set is called the *yield* of the procedure.

If we calculate these two sets for all procedures, including a quasi-procedure constructed from the prelude, root and postlude-calls of the program, all variables which are in the use of the prelude are used somewhere without initialization.

We can improve this algorithm by performing the calculation just for the first prelude call and then disregarding the variables which are in the yield of that prelude call. Repeating these steps for all prelude, root and postlude-calls, we get a decreasing number of variables for each step. Since our experience shows that most of the global variables are initialized in the prelude-part, the analysis of roots, which contain most of the procedures, does not have to consider many variables.

9.6.4 Separate compilation

Within the framework of our checks, it is especially interesting if some exported algorithm uses variables, which have to be initialized by either another exported algorithm or by some prelude or root call. The conditions that have to be fulfilled between modules are simple as long as no mutual intermodule communication is allowed, e.g. if the modules form a hierarchy. Having completely unrestricted communication between modules, the conditions obtain such a complexity that they may hardly be checked.

As long as we compile the importing and exporting modules together, it is possible to check globally that no uninitialized variables are used.

When separately compiling these two modules we have a problem — in compiling the defining module we do not know the order of the calls; in compiling the applying module we do not know the restrictions on the order of calls.

9.7 Semantic check results

In using implementation languages, the presence of a static semantic check can catch a great many errors which would otherwise be found only by run-time tests.

Of the semantic checks for CDL2 described here, the local check has been in use since early 1976. The global check is in use in the CDL2 LAB, where the whole program is accessible.

After initial resistance from our (quite conservative) users, and improvements to its human engineering, the semantic checker has come to be accepted as an extremely useful production tool. One reason why each CDL2 program is now routinely checked, is the speed at which the check is performed. The local check needs about 16 seconds of CPU time for a 5000 lines program on an IBM/370-158. This contrasts favourably to the 0.3 to 0.5 seconds per FORTRAN statement mentioned for a similar (but weaker) checker implemented on a CDC 6400 by Fosdic and Osterweil [FOS76], which in that form seems hardly applicable to large programs.

The methods described are of value to implementation languages in general. Some properties seem to be essential for the application of the checks:

- the copy/restore parameter mechanism and the absence of procedures as parameters are necessary (even though not sufficient) for solving the alias-problem and simplifying the checks on data flow
- tight visibility control and a simplified block structure enhance the perspicuity of the program and reduce the complexity of the calculations involved in the checks
- in designing an open-ended implementation language, useful redundancy should carefully be introduced.

Chapter 10

The CDL2 LAB

10.1 Introduction

The CDL2 LAB is a program development environment centered around the language CDL2. It is intended for the development of software by a team of programmers and its realization on diverse computers by cross-compilation techniques.

In this chapter three aspects of the system are highlighted

- the user interface to the program database
- the facilities for partial compilation and recompilation
- the control of overlays

This chapter is a rewritten version of [BAY81]. Further information on the CDL2 LAB can be found in [EPS88a] and [EPS88b].

10.1.1 Conventional programming environments

The CDL2 compiler has a number of features which are unusual for a compiler. In particular it includes a Static Semantics Checker (described in Chapter 9) and a Global Optimizer (described in Chapter 8) as well as some facilities for cross referencing, profiling and separate compilation that, in conjunction with a good editor, make it a useful production tool in a timesharing environment.

In using the compiler a number of shortcomings of the conventional programming environment become evident:

- There is no support for design and specification of software systems in current operating systems, apart from an editor and possibly some word processing software. These systems are not of much use until the construction phase.
- Editors have only limited cognizance of the structure of programs, so that they support disastrous changes as readily as orderly development.
- Conventional file systems are not very helpful in keeping track of the components of a large program, in various stages of development at the hands of a number of programmers.
- The command language interfaces of different operating systems are widely different in level of abstraction and comfort, influencing not only the ease of working but also the working style and efficiency. A unified user interface should enhance productivity on different computers.

- On some systems, the designers appear to have gone out of their way to make life miserable for program developers, who have to struggle continuously with fixed-size partitions, job control parameters spelled out in nauseating detail, hardware- and configuration dependencies as well as inflexibility of utilities. A user interface tailored to the needs of programmers is sorely needed.
- The cost of recompilations, frequently needed after trivial changes is quite high. Using separate compilation is rather cumbersome because of the manual bookkeeping. Separate compilation greatly reduces the potential for optimization and generally precludes production of quality code.
- The extraction of subsystems (passes and overlays) as well as the production of variants and the maintenance of multiple versions usually implies a lot of tedious organizational and clerical task, since it is not supported in any way.

The CDL2 LAB, designed and implemented in 1976-1980 with support by the Bundesministerium für Forschung und Technologie (German Federal Ministry for Research and Technology) and SIEMENS AG, overcomes these shortcomings. It is a relatively early example of an integrated programming environment.

10.1.2 Components and functions of the CDL2 LAB

Figure 10.1 attempts to give an overview of the components of the CDL2 LAB.

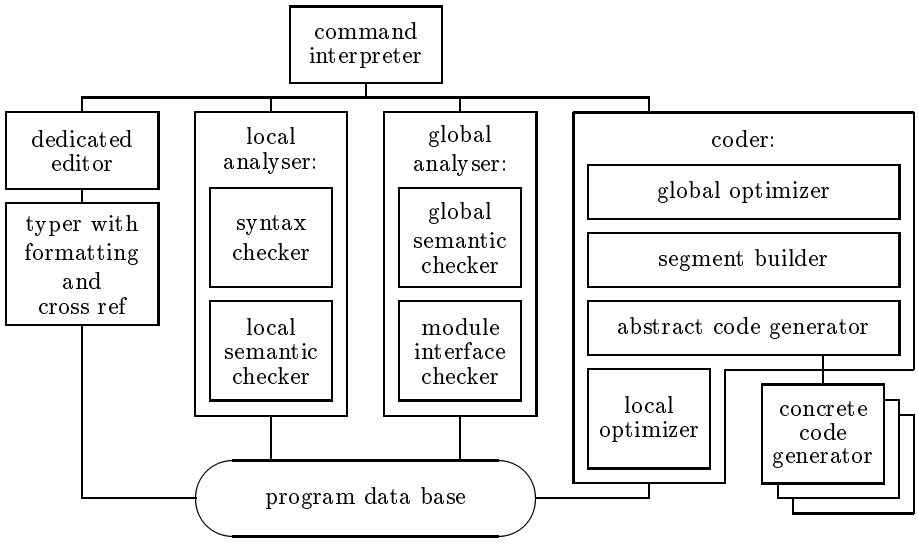


Figure 10.1: CDL2 LAB components

These components serve to support the various stages of the design and development process:

- design and specification: dedicated editor, interface checker
- construction: dedicated editor, analyser, typer
- coding: optimizers, segment builder
- test and debug: concrete code generators, trace, back-trace
- tuning and adaptation: profiling, segment builder

10.2 Language dedication

In developing software on a general purpose system, one is usually confronted with different languages corresponding to different levels of activity:

- the command language which is the basic tool of interaction with the system,
- the editor command language for manipulating source files,
- the programming language chosen to solve the given problem,
- the debugging language(s) to be used during the test phase as input to debugging tools (if available).

10.2.1 Conformity of levels

As opposed to systems supporting several programming languages, the CDL2 LAB is a single language environment: software developed within this system can only be written in CDL2. The same language serves as basis of the command language:

- composite commands are written like procedure bodies, using the CDL2 control structures for sequencing, alternatives, and repetition;
- keywords and source text fragments of the programming language are accepted as parameters to the commands, and are used for retrieval and manipulation of the source;
- source input does not require commands: any sentence of the programming language forms a valid command and is used for source input or modification; there is no distinction between input mode and command mode.

The conformity introduced by using one language for all purposes has a number of advantages:

- there is no need to learn a special command language and all user input is based on the same syntax;
- logging, filing, editing, compiling and debugging facilities are merged into a uniform development process, which is controlled by one single, high-level language;
- in extending the user interface, the user does not loose the security provided by the programming language.

10.2.2 Commands

command

The invocation of a command is just a call on a CDL2 procedure. The syntax of command parameters, however, is extended to permit selectors, CDL2 source text fragments, numbers and special symbols as parameters. Command procedures are generic: different procedures may have the same name, provided they differ in the type or number of parameters.

For example, three different procedures `type`, one with a selector parameter, one with a numerical parameter and one with an asterisk as parameter, are identified generically by the respective commands:

```
type MODULE reader ACTION (read special, read number)
type 5
type *
```

Source sentences and source fragments enclosed between slashes may be used as parameters. Complete source sentences may be entered as they are, without any special commands or additional symbols, and are then treated as implicit enter commands. The following commands are equivalent:

```
enter /CONST zero=0./
CONST zero=0.
```

The control structures of the command language are those of CDL2. A command is either a single call or an (enclosed) group. A command may therefore be composed of alternatives, named groups, calls, and control operators.

Let us give two examples; a simple typing loop:

```
next, type, *
```

A global modification:

```
next MODULE CALL make PAR empty,
delete,
rename CALL make empty, *
```

The second example replaces each call of the form `make+xxx+empty` within the given module by a call of `make empty+xxx`.

10.2.3 Abridged commands

All commands can be shortened by leaving out the last letters. It is sufficient to give a unique prefix, one letter in the case of the most frequent commands. Some examples of shortest prefixes:

```
focus    f
list     l
type     t
delete   d
next     n
previous p
top      to
last     la
code     c
analyse  a
insert   i
```

As an example, the `list`-command may be rendered as any of

```
list   lis    li   l
```

The types occurring in commands may be similarly abridged:

```
MODULE    M
SECTION   S
PREDICATE PRE
PROGRAM   PRO
OBJECT    O
ACTION    AC
FUNCTION  FU
TEST      TE
AFFIX     AF
MEMBER    ME
```

and so on.

For expository reasons, we will not use abridged commands in this chapter.

10.3 Software data base

The information available in the data base of the CDL2 LAB forms a hierarchy according to authorship and language constructs. The data base deals with the source texts and interfaces as well as annotations to the text.

It remembers a number of attributes of the components such as stage of development, time of last update, access right and authorship.

Input, retrieval and manipulation of information is supported by a general selection mechanism described in the following sections and by options, examples of which will occur in later sections. The selection mechanism constitutes a data base navigation and retrieval language which is tailored to the needs of software development.

10.3.1 Tree structure

All information is kept in a tree, reaching from the root `LABORATORY` down to the smallest syntactic units. The top levels provide a hierarchy of ownership of modules. Each of the lower levels in the tree corresponds to some construct of the programming language. The structure is shown in figure 10.2.

Levels have as *type* a language construct (`MODULE`, `SECTION`, etc.), and subtrees are identified by their type and by their name.

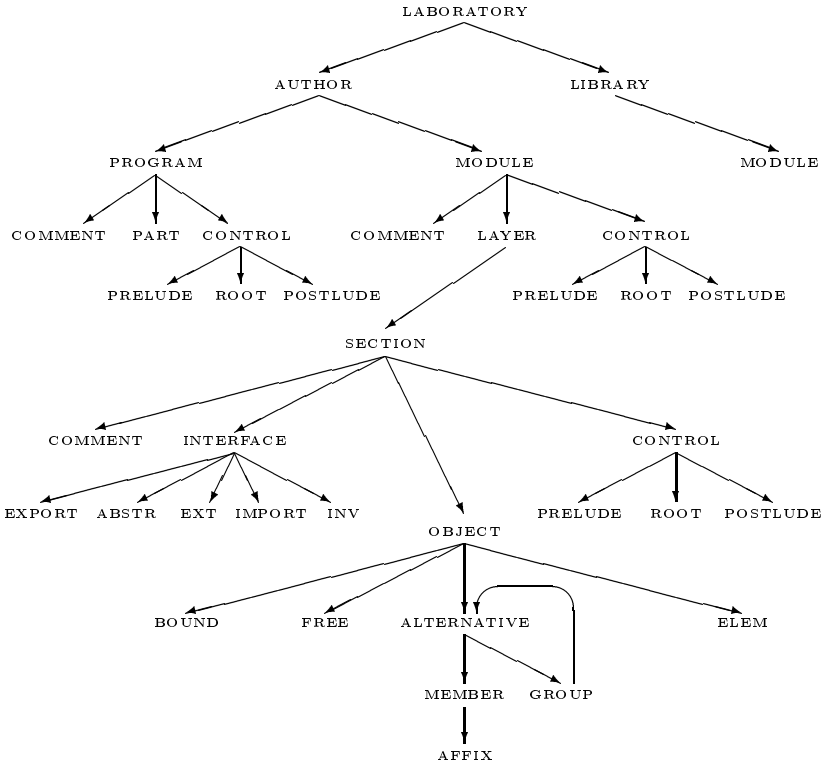


Figure 10.2: Structure of the data base

10.3.2 Focus of interest

focus

While constructing the program, the programmer's interest always centers around a certain part of the software — a specific module, section, interface, object, etc. This *focus* of interest is modeled inside the data base to reflect the programmers interest.

The current focus serves as an implicit parameter to all commands, so that in most cases only the operation wanted has to be indicated. For example,

```
list
```

lists the current focus on the screen.

The focus can be moved up or down the tree or shifted on one same level by means of the focus command, e.g.

```
focus VAR ptext
```

Notice that `focus` and `ptext` are identifiers whereas `VAR` is a type and thus written in capital letters.

10.3.3 Subtree selection

The CDL2 LAB serves to support software development, not general information retrieval. The selection mechanism therefore is defined entirely in terms of the language and the software. Selection expressions contain constructs of the program but not other symbols such as quantifiers, separators, range delimiters etc.

Program constructs are not just arbitrary strings occurring somewhere in the software, but are *named units* corresponding to language constructs, represented as subtrees in the data base hierarchy. For this the programming language CDL2 is a help, because all entities are named.

By combining the above principles — focus of interest and naming of entities — subtree selection is simple. The selection starts at the current focus and evaluates the *selector* given as command parameter. Selectors consist of concatenations of type name pairs.

```
focus MODULE symbols
type
```

moves the current focus to `MODULE symbols` and types it. It has the same effect as

```
type MODULE symbols
```

In a more elaborate example

```
focus  MODULE symbols SECTION buffer
delete EXPORT open symbol
```

has the same effect as

```
delete MODULE symbols SECTION buffer EXPORT open symbol
```

For the inspection or modification of the source it is often necessary to retrieve all entities of a specific type within a certain range. Selection of single subtrees can easily be generalized to selection of multiple subtrees, by omitting identifiers or by entering identifiers containing *wild-card characters* (_), indicating all-quantification. The command

```
type MODULE symbols SECTION OBJECT CALL open symbol
```

retrieves all calls of `open symbol` in objects belonging to sections in the module `symbols`.

```
type MODULE symbols SECTION ALG _open_
```

might cause the following to be typed:

```
ACTION open symbol + > symbol:
    ...
ACTION open buffer:
    ...
TEST is buffer open:
    ...
ACTION open hash + > val:
    ...
```

In the selectors only the levels relevant in the specific situation have to be given. For example, if the programmer wants to know all applications of the constant `trace channel` in his modules, irrespective of the layer and section, he may simply ask

```
type MODULE _ CALL PAR trace channel
```

and will get the answer exactly in the terms of the question — no layers, sections, or objects will be given, but just the modules and the calls having that parameter, in the form

```
MODULE scanner
CALL outtext PAR trace channel
CALL outchar PAR trace channel
MODULE symbols
CALL close channel PAR trace channel
```

10.3.4 Extending the focus

Similar to subtree selections, there is the possibility to look around the current focus of interest — *zooming in* to look at a detail or *panning* to take a wider view, like a camera. Since the focus of interest corresponds to a subtree in the information structure, there is always a sequence of extensions that corresponds to the supertrees of that tree (see figure 10.3).

zooming in

panning

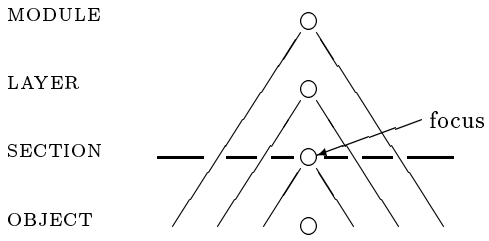


Figure 10.3: Focus extension

The different extensions of a focus can simply be identified by their type, therefore the name may be omitted. Example:

```
focus SECTION identifiers
list  MODULE SECTION
code  MODULE
```

Extension of a focus and selection of a subtree have the same syntactic form, but have a different interpretation, depending on the context, e.g.:

```
focus MODULE scanner
list  SECTION

focus ACTION open buffer
list  SECTION
```

In the first case, the selector `SECTION` is *below* the focus, so it is a subtree selector; since the identifier is omitted, all-quantification is implied — all sections of the module `scanner` are listed.

In the second case, the selector `SECTION` is *above* the focus, so it leads to an extension of that focus; the identifier is omitted, because it is redundant — the section containing the action `open buffer` is listed.

Since there is no need for different extensions of a focus within one command, only the first selector in a selection may be an extension. An extension can be combined with subsequent subtree selections.

```
focus ACTION open buffer
type  SECTION VAR
```

Here the selector `SECTION` extends the object-focus to the section and `VAR` selects all variables of the section: type all variables defined in the section which surrounds the action `open buffer`.

10.3.5 Ordering the tree

So far, no specific order has been implied for the tree-structure of information. Selection of subtrees, ranging over subtrees and focus extensions do not depend on any order of the tree: subtrees either are uniquely identified by their name and type, or all-quantification is assumed, which can be done in any order.

However, in its textual representation software is ordered, and the order of modules in a program, of sections in a module, or of objects in a section is often meaningful to the programmer.

The tree-structure of information therefore is ordered; the components of each subtree are arranged in their textual order as visualized in figure 10.4.

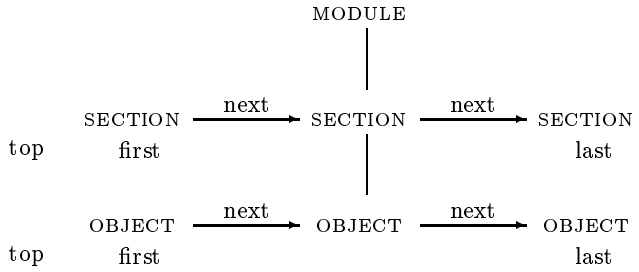


Figure 10.4: Tree order

The CDL2 LAB contains appropriate commands to move to the next, previous, top, first, or last entity in any ordered sequence in the tree. It is necessary to move to the top entity to insert a new first one, an imaginary bottom entity is reached, if the last one is deleted.

10.4 Editing

Editing is a highly personal activity: nobody willingly gives up the editor he is used to, and a new editor is accepted only with the greatest reluctance. A programming environment should not impose one unique way of editing (whether syntax-directed, screen oriented or line oriented) and should allow a user to retain his private habits.

Within the CDL2 LAB, two editors are built in: A *structure-editor*, operating directly on the program database and a simple but powerful *screen editor*.

Furthermore, all editors from the host environment can easily be invoked.

10.4.1 The structure editor

The structure editor is a syntax-driven editor, operating on the current focus, and can deal with both large and small entities. It can delete, insert, rename or replace an entity by an entity of the same type, and works hand in glove with the selection and quantification mechanism. Some examples:

```
delete MODULE CALL trace output
```

deletes all occurrences of a call on `trace output` from the current module.

```
focus OBJECT BOUND x, insert BOUND />y./
```

inserts an input parameter `y` after the (bound) parameter `x` of the current object.

```
focus OBJECT ALTERNATIVE, n, n, d
```

focuses on the third alternative of the current object and deletes it.

Editing activities can be repeated using CDL2 control structures, to perform a global modification on selected parts of a program unit:

```
next MODULE CALL open outfile,
insert /MEMBER set append mode, set variable format./, *
```

inserts the two calls `set append mode` and `set variable format` after each call of `open file` in the module currently focused on.

Syntactic units can be replaced:

```
replace TEST wants trace /TEST wants trace :+./
```

replaces the selected algorithm `wants trace` by a new one which always yields true.

```
rename + vax bottom 1v4
```

renames the current focus to `vax bottom 1v4`.

The structure editor offers a consistent way to edit both small and large entities, with a powerful facility for search and (global) modification. Like all syntax-driven editors however, it is difficult to use when making structural transformations which involve moving pieces of text from one position to another and modifying the nesting structure of constructs. Also it is hard to make a modified copy of some object. For such purposes, syntax-directed editing is not so suited.

10.4.2 The screen editor

The screen editor is based on the idea of *replay*: any construction can be shown on the screen (by the `type`-command) and then read back from the screen, possibly after some modification, at the same place (to replace an existing object) or at another place. replay

The screen acts as a note pad, which can be scrolled back and forth over many pages, in which texts can be noted down and from which texts can be copied at will.

This replay facility turns out to be the ideal complement of the structure editor, having complementary strengths and weaknesses. In fact this difference seems to be a question of granularity: the screen editor is most useful for dealing with small objects one at a time, whereas the structure editor is better suited for more global modifications.

The replay facility is also applicable to commands and other input issued by the programmer, so that (possibly modified) commands and command sequences may be replayed at will.

10.4.3 The host editor

The `edit` command of the CDL2 LAB can use any editor of the host operating system to edit the selected or focused syntactical unit, e.g.

```
focus MODULE scanner SECTION character io, edit
```

types the section `character io` into a file on the host system, calls the host system editor, which allows the modification of this file, and then reads back the modified file.

This allows for complicated textual changes, which cannot easily be performed by structural or screen editing, and gives users the possibility to use their favorite, indispensable text editor. It should be realised, however, that the syntactical correctness is only checked when reading back the modified file and that disastrous changes are not prevented in any way.

10.5 Development support

10.5.1 Support principles

The development of software is to some extent influenced by the development tools, the more so if they are interactive, comprehensive and are shared by the different programmers. Support given by the CDL2 LAB to the development process is guided by three principles:

- interactive, but undistracted individual work
- tolerant but steady enforcement of the CDL2 philosophy
- coordinated but independent multi-user work

The interactive development of programs in the CDL2 LAB comprises not only editing, but intends to cover the whole cycle of analysis and error correction: debugging activities, documentation, administration, and retrieval. Such a wide application range of interactions in a unified environment opens up new possibilities, but also gives rise to the danger that the programmer perceives the system as a straitjacket or is overwhelmed by information.

Undistracted interaction with a dedicated editor implies that only those errors are shown upon input, that are obvious mistypings and violate the context free syntax. Incomplete program parts are tolerated so that the order of input and correction is insignificant.

Using the more conventional CDL2-compiler, the programmer was driven to carry out all the good intentions of CDL-philosophy at the same time: he had to think how to achieve suitable

program structure, had to write detailed interfaces and correct import specifications, had to find suitable refinements, and, last but not least, had to care for proper data flow and effect specifications.

The CDL2 LAB is more tolerant: it allows the programmer's interest to center around one goal at a time by ignoring deviations from another, which is not yet relevant. A phase model is imposed on the development which guarantees that the necessary steps are taken in due time and that finally everything is checked. Errors are shown on demand only and related to the phase the unit in question is in.

Independent development of CDL2 modules may be done with the separate compilation facility of the classical compiler as well. But in the CDL2 LAB, independence is emphasized by individual working environments and by distinct authorship, which is invested with access rights and responsibilities.

Features for incremental intermodular analysis together with the communication facilities between members of programming teams enable a coordination of the individual work throughout the whole implementation process.

Interaction, tolerance and coordination are new qualities introduced into the development process which cannot be provided by an isolated compiler. They are possible because the compiler and the database are combined into a comprehensive environment.

10.5.2 Source units

The language CDL2 provides concepts for the modular and hierarchical decomposition of software: programs are decomposed into modules, which are made up of sections, and are structured into layers. These concepts play a definite and distinct role in the development.

Programs (or groups of programs) are the units of problem solving, responsibility and documentation to the outside world. A program is structured, designed and constructed by a group of authors.

Modules are the units of program construction, and possibly of design. A module may be designed and will be constructed by an individual author who is responsible for its interface and function. It is a functional unit, providing algorithms and constants to other modules. Its internal operations are not visible to the outside.

Sections are not visible to the outside, but serve to provide operational units, viz. procedures and objects.

In the design and the construction of CDL2 units three aspects may be distinguished:

- structural (modules, layers, sections)
- functional (section and module interfaces)
- operational (procedures with their associated objects)

Structural design means that the layer, module, and section structure is fixed; functional design comprises the specification of interfaces; operational design may be done informally or by partial construction of procedures, operational construction means full construction of procedures within sections. In the CDL2 LAB, design and construction phases may interleave, and the depth of design and of construction may vary for the different units of a program. As a consequence, different methods may be applied in their development.

According to the stage of development static checks are performed on the software as appropriate for each unit; especially a unit may be checked while other units in its environment are still missing, inconsistent or incomplete. The different aspects of construction may be checked largely independent of each other:

- structural analysis of decomposition
- functional analysis of interfaces
- operational analysis of objects

Operational analysis comprises static semantic checks for plausibility of the information flow and for consistency of effect specifications.

10.5.3 Development methods

The method suitable for the development of a program largely depends on the nature and size of the problem and on the organization of the programming team. Additionally the methods used "in the large" for the design of programs and modules are usually not appropriate "in the small" for the design of sections. Therefore different development methods are supported by the CDL2 LAB; to illustrate them let us consider the following cases.

A chief programmer or a design team has designed a program and has specified the module interfaces. Now, the construction of the modules is delegated to the different members of the team. Not a single object of the program has been constructed yet, but already an analysis of interfaces may be performed to assure that the specified modules fit together to form a consistent program. The programmers constructing the module bodies can rely on a concise specification of the interfaces that are to be realised and of those that may be used.

A programmer has to develop a module; she first partitions a problem into layers and sections and defines their interfaces. Then she wants to construct the sections one after the other. During construction of a section she can ask the CDL2 LAB whether it is already consistent or complete and whether conflicts arise with other sections. The method applied in both cases may be called *skeleton programming.*

Another author constructs a basic module to be used in several programs or at a later time, when the global design of the containing program will be finished. The form and extent of the export interface is fixed by himself and may change and grow on demand during construction. Certain modules are operationally complete, while others are still being designed. This method of development may be called *pilot programming.*

Of course, ordinary Top-Down or Bottom-Up programming or a mixture of both is also possible in the CDL2 LAB and is usually applied within modules or at least within sections. Programming bottom up, the defined objects may be checked for operational correctness in an early stage. Top-down defined objects may be checked as soon as all objects applied are at least specified.

These methods are supported by the fact that the components of the CDL2 LAB tolerate incomplete programs wherever this is possible. Additionally, the CDL2 LAB helps with the writing of interfaces, and with adapting of interfaces, as it becomes necessary after any restructuring of modules and layers:

- an incomplete module is recognized as such, but the parts already constructed are checked completely as far as specifications of missing objects are provided;
- objects or whole sections can be easily moved from one place or layer to another;
- objects can be renamed together with all their applications without affecting objects with the same name in other contexts;
- missing or unresolved invocations are tolerated but are reported as warnings; invocation interfaces can be straightened on demand, so that only the necessary objects are invoked;
- the providing interfaces of sections can be automatically reorganized into extension and abstraction parts (as far as the strong hierarchy of layers allows), so that a redesign of the layer structure is facilitated;

- unused specifications can be deleted from the source.

10.5.4 Development stages

During development cycles (design, construct, test, reconstruct) each source unit may be characterized by its *development stage* which may be

- modified (recently edited and changed)
- compatible (the interface fits into the environment, but the unit may still contain internal ambiguities or conflicts; i.e. it is not yet consistent)
- consistent (compatible with the outside, and free of internal ambiguities and conflicts)
- complete (consistent and fully constructed; precondition for coding)
- coded (complete and code generated)

Compatibility, consistency and completeness should be discussed in more detail according to the structural, functional and operational aspect of development.

The structural aspect is handled by the editor: the context-free syntax check guarantees structural consistency. Structural completeness is of course not decidable and structural compatibility makes no sense.

The functional aspect concerns the interfaces of the unit and its relationship to the environment thus contributing to the compatibility. A section is (functionally) compatible with the surrounding module, if its interfaces do not give rise to ambiguities or conflicts within the containing module. An incompatible section spoils the module, which becomes inconsistent. A module cannot be incompatible as long as it is not bound to a program. A bound module (or program part) is incompatible, if its export/import interface gives rise to conflicts in the program. Again, incompatible parts contribute to the inconsistency of the program. Functional completeness is defined for an environment and means, that all functions used by some constituent are defined in the environment.

The operational aspect primarily concerns sections, since all operations are defined within them, but is passed to modules and programs. A section is operationally consistent if its objects are built up on each other without ambiguities or conflicts and if the data flow and effect specifications are plausible. It is operationally complete, if in addition all object bodies have been constructed.

Working with the CDL2 LAB, the programmer will usually not distinguish between structural, functional and operational errors, but may simply ask for the development stage reached for the respective unit.

```
focus MODULE scanner
list inconsistent SECTION
type incomplete OBJECT
```

In this example, the inconsistent sections of the module `scanner` are listed. Then all incomplete objects of that module are typed out. Here we use the options `inconsistent` and `incomplete`. In the following example, all sections are listed which contribute to inconsistent interfaces within the module `syntax`. Then all interface conflicts within the respective program part are typed out.

```
list incompatible MODULE syntax SECTION
type incompatible PROGRAM p PART syntax
```

10.5.5 Error annotations

Offences against consistency and completeness may be of different severity. Therefore, errors, warnings and hints are distinguished. Errors are statically recognized properties indicating that the program cannot function as intended, e.g. uninitialized variables. They inhibit translation when not corrected. Warnings and hints are slight infringements agains CDL-philosophy or implausibilities (e.g. units of program which are never executed, automatically repaired units, etc.). They give rise to the suspicion of errors — possibly at other places — and therefore the author is invited although not forced to correct them.

Errors, warnings and hints are communicated to the programmer on demand only. No immediate presentation of errors is given, and no ad-hoc correction is forced. When finding an erroneous situation, the CDL2 LAB reacts in a tolerant way: it attaches an annotation to the erroneous source construct which can be displayed at a later time. Erroneous or, more specifically, incompatible, inconsistent or incomplete units may be selected for the respective source unit for inspection.

```
type error inconsistent MODULE SECTION OBJECT
next hint  MODULE
next warning incomplete MODULE syntax SECTION
```

In this example in all sections of the current module, all objects containing inconsistencies are typed together with the corresponding annotations. Then the next module containing (at least) a hint annotation is selected and after that in the module `syntax` the first section is selected which contains an incompleteness warning.

10.5.6 Notes

note annotation

The annotation mechanism used by the CDL2 LAB for error presentation is also available to the programmer. User written annotations, so called *notes*, may be attached as labels to any source construct. Errors, warnings, hints and notes are handled the same way by the CDL2 LAB.

```
note inoperative ACTION switch channel
focus SECTION
next note inoperative SECTION OBJECT
type labels SECTION
```

Here an action is selected and noted as `inoperative`. It may be selected by the following command which searches the first object carrying such a note. Then all labeled units of the respective section are typed out together with their labels.

10.5.7 Incremental analysis

During test and reconstruction of a program, reanalysis of modules and programs is frequently needed. But the changes made in a single correction step are often rather small and local. By means of the information stored in the database about the development state of the different units, it is possible to restrict the analysis to the program units actually modified. Reanalysis of a module is restricted to those sections which have either been modified or are influenced by these modifications; reanalysis of a program is restricted to those sections which have been modified since the last analysis. Sections are taken as increments of the analysis for two reasons:

- the sections and not the procedures are functional units and therefore several procedures of one section are usually changed together
- the interface structure held in the database helps to determine the path along which a modification influences other parts of the program.

Three qualities of CDL2 are exploited to achieve incremental behaviour:

- locality of declarations (within sections) and information flow along strict interfaces
- one level of declarations (no block structure)
- object-orientation of interfaces.

10.6 Compilation schemes

While for program development the speed and ease of compilation is important, for the production version of a program the quality of the code produced is crucial. Obviously, the two goals are quite different and cannot be achieved by a single compilation scheme: fast recompilation is made possible by separate compilation of modules; high quality code, be it in space or in time, requires global optimizations across the boundaries of modules.

10.6.1 Modules and programs

Separate compilation of modules puts limitations on the quality of the code produced:

- Intermodular communication and control, i.e. access to constants and calls of procedures, may be degraded.
- modules may provide more features than are needed in a specific application, and therefore have more code than necessary
- modular decomposition of algorithms is often rather detrimental to the locality of the running code; modules are conceived as logical units, and not as execution phases or localities.

Clearly, these limitations can only be overcome if programs are optimized across the boundaries of modules.

The CDL2 LAB provides for both separate compilation of modules and integral compilation of programs as well as for any intermediate form.

10.6.2 Separate module compilation

Let `formatter` be some program, composed of modules as shown in figure 10.5.

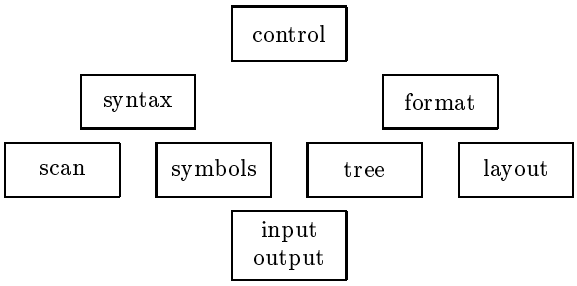


Figure 10.5: Module structure

All modules may be compiled separately in some order and the program compiled afterwards to provide the instantiation and control:

```
code MODULE input output.
code MODULE symbols

        ⋮

code MODULE control
code PROGRAM formatter
```

Each module is now compiled and optimized separately. Since optimization cannot overcome module boundaries, the composite program is far from optimal. On the other hand it is easy to modify and recode only one module, without the need to recode any of the other modules.

10.6.3 Integral program compilation

For integral program compilation the program itself is translated; its modules for as far as they have not already been compiled separately, are integrated selectively, viz. only code that is actually necessary is included.

```
code PROGRAM formatter
```

Integral program compilation composes the modules and the program control into one single code segment. Boundaries between modules are dissolved, so that communication and control are fast and local; algorithms and data that cannot be reached or accessed by the program are stripped from the code, so that alternatives excluded by static tests (`trace wanted`, `check wanted`, `dump wanted`, ...) and unused variants (`for ibm`, `for siemens`, `for z80`, ...) disappear entirely.

Procedures are coded in a bottom-up, true-false order throughout the program, so that they can be expected to exhibit reasonable execution locality.

Using integral program compilation instead of separate compilation of modules, reductions in the size of code were experienced ranging from 1.5:1 to 4:1, depending on the degree and kind of modularization.

10.6.4 Compiling subsystems

Often, some parts of a program are complete and stable, so that they could be integrated, while other parts are still under development, so that they should be kept apart. Or, a program may be ready for integration, except for some variant parts that should be compiled on their own. Sometimes, a group of modules may even be developed independent of any specific program, so that their code could be integrated as a separate program part into more than one program.

Any subsystem of modules may be coded integrally. For this purpose, it is often useful to group a number of modules together into one subsystem by introducing an artificial module, which contains no code but only reexports the exports from those modules. Any code that "hangs from" the export interface of this artificial module will be integrated.

A group of modules $\mathtt{m}_l$, ... , $\mathtt{m}_k$ that contribute in this fashion to some module `c` may be integrated together with `c`: the contributing modules are left uncoded until the module `c` has been coded.

```
code PROGRAM p PART c
```

Note, that a module may contribute code to different subsystems; the respective algorithms are integrated where needed, possibly with duplications. To prevent duplication, variables and lists are coded only once, namely when coding the program proper.

As an example, the sample program `formatter` might be compiled as four subsystems. The module `input output` is compiled first to prevent duplications of its code.

```
code MODULE input output
code PART syntax
code PART format
code PROGRAM formatter
```

The resulting segment structure may be depicted as in figure 10.6. segmentation

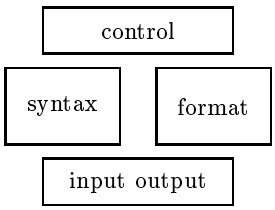


Figure 10.6: Segment structure

10.6.5 Constructing overlays

High quality code, both in space and in time, as is achieved by integral program compilation, is not sufficient in all cases: for large systems or for small computers, different phases or components of a program may have to be overlaid in storage. To determine a good overlay structure and to get a compiler to produce it is traditionally hard work and absorbs a good deal of the development effort.

In the CDL2 LAB the construction and overlaying of segments is specially supported:

- arbitrary program parts, such as module groups (components, phases or localities) can be identified by their export interfaces,
- on demand, the segment builder of the CDL2 LAB computes minimal directed overlays that indicate the possible overlays and their size, and
- program parts can be compiled integrally across the boundaries of modules, with automatic inclusion of overlay calls.

Suppose for example that the module `input output` of the `formatter` comprises terminal I/O and file I/O. If the `syntax` part does not need file output, the `format` part does not need terminal input and the two phases are largely disjoint in time, then by integrating the `input output` module in each of those parts

```
code PART syntax
code PART format
code PROGRAM formatter
```

the overlay structure shown in figure 10.7 is reached.

Notice that although the total code size of the `formatter` is larger than in the previous example, the present form allows a higher degree of overlaying and shows better execution locality.

10.7 Authors and groups

Software development usually means the production of a program or program system by several programmers; thus, any system that claims its ability to support the kind of multiprogramming involved in that process, has to meet a number of requirements:

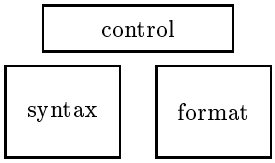


Figure 10.7: Overlay structure

- it has to reflect the existence of users and of groups of users, that share in the development of software
- it has to show the division of work inside a group during the different phases of development and has to carry out the consequences of the division — such as responsibilities and access rights of users with respect to the different parts of the software
- it has to support coordination between separate programming activities
- but it also has to remain adequate for private, one-person implementation jobs

Various models have been proposed for the organization of software development, such as chief programmer teams, project libraries, and others. In the CDL2 LAB we chose to support a fairly flexible organization which may be applied to specific models. It is oriented towards programmers, called authors, and groups centered around libraries, not towards projects or components, but this may only be a question of terminology.

author
group

Groups are not defined explicitly, but are formed implicitly by those authors that have access to common group libraries. Groups are not exclusive: an author may be a member of several groups. An author may have one or more *ranges* to work in.

The grouping of authors into groups will reflect the structure of the respective software.

Programs may be decomposed into separate modules; the division of work within a group then follows the module structure of the program(s). That is, a group member is responsible for a module as a whole; concurrent development activities will never affect the same module. The responsibility for a module may, however, be transferred between different authors, if both agree.

Module development as well as program analysis and integration take place within the author ranges; thus an author range may either be used for module development, or for program integration, or may be used for both tasks. While the development and testing of components might be done by the individual authors, the integration of the whole system might be performed in a separate author range, which is well protected against component updates.

The coordination of separate development and integration processes is done via shared library ranges. Authors logically forming a group all have the right to access and update at least one library, in which they may put (locally consistent) versions of their modules; conversely, other authors may get such versions into their private ranges for integration into programs. (Authors may have update access to several libraries, thus implicitly belonging to several groups at a time.)

In addition to the update right, link and copy access to libraries give the possibility to organize complex group structures with specific access rights. Authors having link access to a library may use modules from it but not change them; modules transferred in the copy mode to an author's range may be updated, but may not be put back into the library. Similarly, an author's update access to a library is limited to those modules he is responsible for; that is, once an author has put a version of a module in a library, any co-author in the group may get it — in link or even in copy mode — but not put it back, unless the first author deletes his version in the library.

The CDL2 LAB itself is the range of all authors and libraries; within this range, authors, libraries and connections between authors and libraries may be established or deleted.

Unauthorized access to `AUTHOR` ranges and the `LAB` range is prevented via passwords and libraries are protected by the access restrictions mentioned.

10.8 Conclusion

The CDL2 LAB is an integrated software development tool for professional users. It is not a toy to be used without instruction; it is not a self explanatory menu-driven triviality. In the hands of an experienced user it allows tremendous productivity both in development and in production situations. Its backbone is the marriage between solid database and compiler construction techniques, motivated by a clear perception of the needs of the Software Engineer. As such it is an ideal workhorse for a high-technology software house or a computer science department. It is used as such by a number of companies and universities, mostly in Europe.

Chapter 11

Example: a navigational database

.

This final example will be concerned with the design and implementation of a small navigational database.

Its possibilities are so limited and consequently its structure is so simple that it might perhaps better be called a "datastool". Yet it shows a number of realistic traits and is suitable for generalization to a quite useful system.

11.1 Description of the system

The semantic universe we are talking about consists of *objects*, between which certain *relations* hold. We will introduce commands to establish relations and to delete them. In this process, objects may be added.

This semantic universe can be modeled as a directed labeled ordered graph, in which the objects are represented by the nodes (each node having a unique name, so that nodes with the same name must be the same node) and relations are represented by directed arcs (each arc having a name.)

Relations will be left-unique, i.e. all outgoing arcs of a node will differ in name, and arcs with the same name going out from one node must be the same arc. Names will, as usual be of form

$$letter\ \{\ letter \mid digit \mid -\ \}^*$$

using the dash as a visible space within the names.

Initially, the graph consists solely of one node, the *starting node* of the graph.

A session is a dialogue, consisting of commands in one direction (input) and information displays in the other direction, (output) in which the graph can be examined and modified. From one session to the next the graph may be preserved as a data file.

11.1.1 Navigation

We wish to provide commands for navigating over the graph structure of the database, following arcs by giving their name and in this way traversing various nodes. In this navigation it is very important that the user should at all points know where he is. Therefore we will, after each command, prompt the user by giving the name of the current node. The farther we are away from the starting node, the deeper we will indent this node, in this way emphasising the following of an arc and the return to a previous node.

Initially, the current node is the starting node and the indentation will be, e.g. twenty positions.

11.1.2 The data manipulation language

We will start by describing an interactive data manipulation language for examining and modifying the database. The *commands* of the database manipulation language will be in free format style (that is, spaces will serve to separate the elements of a command), with one command per line. We will distinguish the following commands, which all consist of a single character, possibly followed by one or two names:

`+ selector target`

The *add-command* consists of a plus-sign followed by two names, the name of the selector and the name of the target. It adds the relation `(current node, selector, target)` to the contents of the database, provided the current node does not yet have an outgoing arc named `selector`. In case the target name is one which has not appeared before, it is added to the universe of objects.

`=`

The *list-command* consists of a single equal-sign. It lists the names of all outgoing arcs of the current node (all "selectors").

`> selector`

The *follow-command* consists of a greater-sign followed by the name of the selector to be followed. If the current node has an outgoing arc of the name `selector` to some target node, the indentation is increased, by e.g. four spaces, and that target node is made the current node.

`<`

The *return-command* consists of a less-sign. It makes the previous node, i.e. the node from which we reached the current node, the current node and decreases the indentation. If there is no previous node the dialogue halts after writing the results of our work into a file and displaying the current storage utilization.

11.1.3 Example session

In order to illustrate both the intended user interface and the commands we will give an example session. The program identifies itself and then ask whether we want to load an existing database possibly saved in a previous session. Here, we decide not to load a database and the program inquires the name of the starting node.

```
Info System
Read old database? n
Start node
?
```

We type the name of the start node

```
? William
```

From now on, whenever the program wants to read input, it displays the name of the current node and prompts with a question mark on a new line for further input, e.g.

```
                  William
?
```

Just now the current node is still the start node. We now add an arc to this node by the command

```
? + wife Mary
```

The system responds with

```
                    William
```

and we continue with adding a second relation

```
? + daughter Julia
                    William
```

Note that the current node still is our start node. We have now obtained the following information in the database:

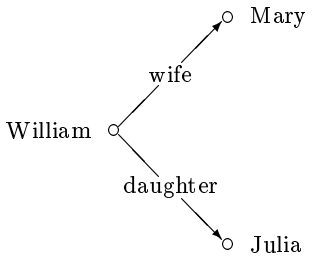


Figure 11.1: Situation after two add commands

We give the list-command in order to see what relations we have established and obtain:

```
? =
                        wife
                        daughter
                    William
```

Following the daughter-relation we change the current node

```
? > daughter
                        Julia
```

and continue adding two relations

```
? + husband John
                        Julia
? + daughter Cassandra
                        Julia
```

If we attempt to add a second relation named `daughter`, this will be reported as an error, e.g.

```
? + daughter Alexandra
Selector not unique, ignored
                        Julia
```

But we can add

```
? + father William
                        Julia
```

If we follow the father-relation

```
? > father
```

we arrive again at the node `William`, but now at a deeper level.

```
                          William
```

We return by successive commands to the root

```
? <
                      Julia
? <
                  William
```

At the root, an additional return-command brings us out of the program

```
? <
Storage for text= 46
Number of work cells left= 98
```

reporting the amount of storage used by the added relations and writing all information entered so far to a file.

11.1.4 Structure in the large

We will not in any way derive or justify the structure in-the-large of the solution that now follows, but will just describe it.

The top layer contains a driver section that recognizes the commands and calls the appropriate semantic actions. The middle layer contains sections for the reading of names and the recognition of special symbols. Furthermore there will be a section administering the relations in the database and a section to perform the transput, including all messages of the user interface. In the lowest level we will distinguish sections for storing the texts of names, for handling characters and relations, for arithmetic and an interface to the operating system. The resulting structure can be depicted as:

11.2 Text of the program

We give the text of the database program (which is included on the distribution disk of the Tiny-CDL-system), annotated with some explanatory remarks.

11.2.1 The lowest layer

The lowest layer `utilities` contains the five sections `arithmetic`, `io interface`, `character reading`, `text handling` and `working storage`.

```
MODULE simple database.

LAYER utilities.
```

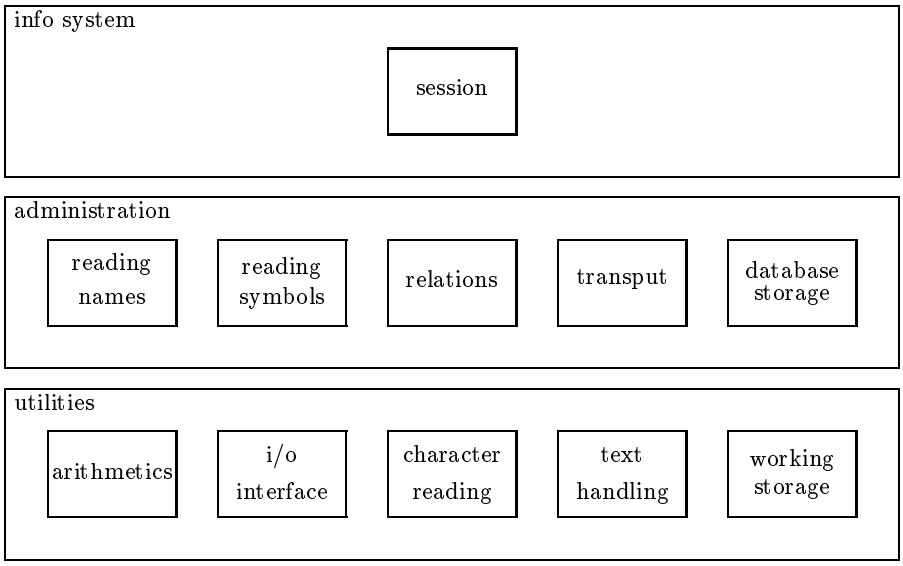


Figure 11.2: Structure of the database program

Arithmetic

This section contains a collection of macros that are used in practically every CDL2-program. The target language is PASCAL. For other target languages, the relevant CDL2 compiler manual can be consulted.

```
SECTION arithmetic.

  ABSTR decr, incr, is zero, make zero, zero, between.

  EXT
     make, equal, lseq, incr, add, make zero, decr,
     is zero, zero, one, invert, take abs, less,
     sub, div rem, pack char, unpack char,
     between, one.

  CONST zero=0, one=1.

  FUNCTION make +x>+>y=
     x ":=" y.

  FUNCTION invert +>x>=
     x ":=" "-" x.

  FUNCTION take abs +>x+xabs>=
     xabs ":= abs(" x ")".

  FUNCTION add +>x+>y+z>=
     z ":=" x "+" y.

  FUNCTION sub +>x+>y+z>=
     z ":=" x "-" y.
```

```
  FUNCTION incr +>x>=
     x ":=" x "+1".


  FUNCTION decr +>x>=
     x ":=" x "-1".


  FUNCTION mul +>x+>y+res>=
     res ":=" x "*" y.


  FUNCTION div +>x+>y+quot>=
     quot ":=" x " div " y.


  FUNCTION mod +>x+>y+rem>=
     rem ":=" x " mod " y.


  FUNCTION div rem +>x+>y+quot>+rem>:
     div+x+y+quot, mod+x+y+rem.


  FUNCTION add mul +>x+>y+>z+res>=
     res ":=" x "*" y "+" z.


  FUNCTION make zero +x>=
     x ":= 0".


  TEST equal +>x+>y=
     "if" x "=" y "then ".


  TEST less +>x+>y=
     "if" x "<" y "then ".


  TEST is zero +>x=
     "if" x "=0 then ".


  TEST lseq +>x+>y=
     "if" x "<=" y "then ".


  TEST between +>x+>y+>z=
     "if (" x "<=" y ") and (" y "<=" z ") then ".


  CONST max char=128.


  FUNCTION pack char +>w>+>c:
     add mul+w+max char+c+w.


  FUNCTION unpack char +>w>+c>:
     div rem+w+max char+w+c.

ENDSEC arithmetic.
```

The interface to the OS

The following section provides the interface to the Operating System. The choice of PASCAL as a target language causes some problems at this place, since PASCAL compilers are not all compatible in this respect.

```
SECTION io interface.

  ABSTR
     put newline, writeln, write, put char,
     writeln outfile, switch to infile,
     close infile, switch to outfile, readln,
     inchar file, outchar file, close outfile,
     get newline.

  EXT
     get char, write, put char, writeln, put newline,
     put int, switch to infile, close infile,
     switch to outfile, inchar file, outchar file,
     writeln outfile, eof char, readln,
     switch to term in, switch to term out,
     if infile reading, get newline, close outfile.

  IMPORT inchar, write, writeln, inchar file.

  INV
     add, decr, div rem, equal, less, lseq, make,
     first digit, space char, zero.

  CONST eof char=4.

  #-------------------------------------------------#
  # Terminal IO                                     #
  #-------------------------------------------------#
  ACTION inchar +char>.

  ACTION readln=
     "readln".

  ACTION outchar +>char=
     "write(chr(char))".

  ACTION write *msg.

  ACTION writeln *msg.

  ACTION newline=
     "writeln".

  #-------------------------------------------------#
  # File IO                                         #
  #-------------------------------------------------#
  ACTION switch to infile *file name=
     "reset (infile," file name ");"
      inchan ":=" file in.
```

```
ACTION inchar file +c>.

ACTION readln infile=
   "readln(infile)".

ACTION close infile=
   "close(infile)".

ACTION switch to outfile *file name=
   "rewrite (outfile," file name ");" outchan ":="
   file out.

ACTION outchar file +>c=:
   "write(outfile,chr(c))".

ACTION writeln outfile +>c=
   "writeln(outfile,chr(c))".

ACTION newline outfile=
   "writeln(outfile)".

ACTION close outfile=
   outchan ":=" term out.

#-------------------------------------------------#
# General IO routines                             #
#-------------------------------------------------#
VAR inchan, outchan.

CONST term in=0, file in=1, term out=0, file out=1.

ACTION switch to term in:
   make+inchan+term in.

TEST if infile reading:
   equal+inchan+file in.

ACTION switch to term out:
   make+inchan+term out.

ACTION get char +char>:
   equal+inchan+file in, inchar file+char;
   inchar+char.

ACTION put char +>char:
   equal+outchan+file out, outchar file+char;
   outchar+char.

ACTION get newline:
   equal+inchan+file in, readln infile;
   readln.

ACTION put newline:
   equal+outchan+file out, newline outfile;
   newline.
```

```
   #------------------------------------------------#
   # Special print routines                         #
   #------------------------------------------------#
   ACTION put int +>x:
      put intfo+x+zero.

   ACTION put intfo +>int+>length -c:
      less+int+base, put spaces+length, out dig+int;
      div rem+int+base+int+c, decr+length,
           put intfo+int+length, out dig+c.

   ACTION put spaces +>l:
      lseq+l+zero;
      put char+space char, decr+l, *.

   ACTION out dig +>d:
      add+d+first digit+d, put char+d.

   CONST base=10.

   PRELUDE switch to term in.

ENDSEC io interface.
```

The reading of characters

The following section provides the buffered reading and recognition of characters.

```
SECTION character reading.

   ABSTR
      is, plus char, equal char, less char,
      greater char, if yes, is letter, is digit,
      next char, end line char, space char,
      start at new line, start input, skip layout,
      dash char.

   EXT
      chars in word, end text char, end line char,
      space char, start at new line, start input,
      skip layout, first digit.

   INV
      between, equal, lseq, make, eof char, get char,
      get newline, if infile reading, put char,
      switch to term in, write.

   #------------------------------------------------#
   # no significant char can have the code 0.       #
   # A S C I I                                      #
   #------------------------------------------------#
   CONST
      plus char="ord('+')", equal char="ord('=')",
      greater char="ord('>')", letter y char="ord('y')",
      less char="ord('<')", first letter="ord('a')",
      last letter="ord('z')", first digit="ord('0')",
      last digit="ord('9')", dash char="ord(' ')", end text char=1,
      end line char=10, space char="ord(' ')",
      bold letter y char="ord('Y')", chars in word=1.
```

```
#-------------------------------------------------#
# the first nonconsumed character:                #
#-------------------------------------------------#
VAR charbuf.

ACTION next char:
   may prompt for more, get char+charbuf,
     (equal+charbuf+eof char, if infile reading,
     switch to term in, may prompt, * next char;
     +).

ACTION start at new line:
   equal+charbuf+end line char, next char;
   next char, *.

ACTION start input:
   may prompt, next char, skip layout.

ACTION may prompt for more:
   equal+charbuf+end line char,
     (if infile reading;
     write+"? "); .

ACTION may prompt:
   if infile reading;
   write+"? ".

ACTION skip layout:
   equal+charbuf+space char, next char, *;
   +.

PREDICATE is +>c:
   equal+charbuf+c, next char.

PREDICATE is letter +c>:
   between+first letter+charbuf+last letter,
   make+c+charbuf, next char.

PREDICATE is digit +c>:
   between+first digit+charbuf+last digit,
   make+c+charbuf, next char.

PREDICATE if yes:
   is+letter y char;
   is+bold letter y char.

ENDSEC character reading.
```

Handling of text

The following section implements a symbol table, employing a binary tree. Equal symbols are mapped onto equal keys, and different symbols onto different keys.

```
SECTION text handling.
```

```
ABSTR add to text, enter text, write text,
   get prop, put prop.

INV
   add, equal, incr, invert, is zero, less, lseq,
   make, make zero, one, pack char, take abs,
   unpack char, zero, put char, put int,
   put newline, write, writeln, chars in word,
   end text char.

#-------------------------------------------------#
# The text stack is a binary tree, realised in the #
# list 'text'.                                    #
# One text entry has the following structure:     #
#             |-----------------------|           #
#             |      left      -------|---> text  #
#             |-----------------------|           #
#             |      right     -------|---> text  #
#             |-----------------------|           #
#             |      prop      -------|---> work  #
#             |-----------------------|           #
# ptext ----> |   first repr          |           #
#             |-----------------------|           #
#                 .                               #
#                 .                               #
#                 .                               #
#             |-----------------------|           #
#             |    last repr          |           #
#             |-----------------------|           #
#-------------------------------------------------#
CONST max text=500, text cell length=3.

LIST text(one: max text).

FUNCTION get left +>ptr+left>=
   left ":=" text "[" ptr "-3" "]".

ACTION put left +>ptr+>left=
   text "[" ptr "-3" "]" ":=" left.

FUNCTION get right +>ptr+right>=
   right ":=" text "[" ptr "-2" "]".

ACTION put right +>ptr+>right=
   text "[" ptr "-2" "]" ":=" right.

FUNCTION get prop +>ptr+prop>=
   prop ":=" text "[" ptr "-1" "]".

ACTION put prop +>ptr+>prop=
   text "[" ptr "-1" "]" ":=" prop.

FUNCTION get repr +>ptr+repr>=
   repr ":=" text "[" ptr "]".

ACTION put repr +>ptr+>repr=
   text "[" ptr "]" ":=" repr.
```

```
VAR word, byte count.

ACTION add to text +>char:
   equal+chars in word+byte count, stack text word,
     make+word+char, make+byte count+one;
   pack char+word+char, incr+byte count.

ACTION stack text word:
   lseq+ptext+max text, put repr+ptext+word,
   incr+ptext; writeln+"Text table overflow", ?.

ACTION init text:
   make+ptext+one, reserve admin space.

ACTION finalize text:
   equal+byte count+chars in word, invert+word,
   stack text word; add to text+end text char, *.

VAR ptext, new.

ACTION reserve admin space:
   add+ptext+text cell length+ptext,
     (less+ptext+max text,
       put left+ptext+zero, put right+ptext+zero,
       put prop+ptext+zero, clear repr,
       make+new+ptext;
     writeln+"Text table overflow", ?).

ACTION clear repr:
   make zero+byte count, make zero+word.

ACTION enter text +>p>+key> -old-prev:
   finalize text,
     (is zero+p, make+p+new, make+key+new, reserve admin space;
     make+old+p,
       (try next entry: make+prev+old,
         (lex before+new+old, get left+old+old,
           (is zero+old, put left+prev+new, make+key+new,
             reserve admin space;
           * try next entry);
         lex before+old+new, get right+old+old,
           (is zero+old, put right+prev+new, make+key+new,
             reserve admin space;
           * try next entry);
         make+ptext+new, make+key+old, clear repr))).

TEST lex before +>p1+>p2 -w1-w2-a1-a2:
   get repr+p1+w1, get repr+p2+w2, take abs+w1+a1, take abs+w2+a2,
     (less+a1+a2;
     equal+a1+a2,
       (less+w1+zero, -;
       less+w2+zero;
       incr+p1, incr+p2, * lex before)).

ACTION write text +>t -w:
   get repr+t+w,
     (lseq+w+zero, invert+w, write word+w;
     write word+w, incr+t, * write text).
```

```
  ACTION write word +>w -c:
     is zero+w;
     unpack char+w+c, write word+w,
       (equal+c+end text char;
       put char+c).

  ACTION report text:
     write+"Storage for text= ", put int+ptext, put newline.

  PRELUDE init text.

  POSTLUDE report text.

ENDSEC text handling.
```

Storage of lists

This section provides a rather general form of linear lists, that can be used as an associative memory (as in this example).

```
SECTION working storage.

  ABSTR find2, insert2, detach2.

  INV
     add, decr, equal, is zero, lseq, make, make zero, one,
     put int, put newline, write, writeln.

  #-------------------------------------------------#
  # The working storage contains zero or more singly #
  # linked lists of pairs, originally combined into  #
  # a free-list. One cell of the storage has the     #
  # following structure:                             #
  #               |---------------|                  #
  #     index --->|   link    ----|----> work        #
  #               |---------------|                  #
  #               |   head    ----|----> text        #
  #               |---------------|                  #
  #               |   tail    ----|----> text        #
  #               |---------------|                  #
  # pointers may be zero.                            #
  #-------------------------------------------------#
  CONST nmb of cells=100, cell size=3, max work=300.

  LIST work(one: max work).

  ACTION put triple +>ptr+>l+>h+>t=
     work "[" ptr "]" ":=" l ";" work "[" ptr "+1" "]"
          ":=" h ";"
     work "[" ptr "+2" "]" ":=" t.

  FUNCTION get triple +>ptr+l>+h>+t>=:
     l ":=" work "[" ptr "];" h ":=" work
     "[" ptr "+1];"
     t ":=" work "[" ptr "+2]".
```

```
    ACTION get link +>ptr+l>=
       l ":=" work "[" ptr "]".

    ACTION put link +>ptr+>l=
       work "[" ptr "]" ":=" l.

    VAR pfree, nfree.

    ACTION create free list -p-q:
       make zero+q, make+p+one,
         (lseq+p+max work, put link+p+q, make+q+p,
         add+p+cell size+p, *;
         make+pfree+q, make+nfree+nmb of cells).

    TEST find2 +>p+>hd+tl> -hp:
       is zero+p, -;
       get triple+p+p+hp+tl,
         (equal+hp+hd;
         * find2).

    ACTION insert2 +>p>+>hd+>tl -next:
       is zero+nfree, writeln+"Workspace overflow", ?;
       get link+pfree+next, put triple+pfree+p+hd+tl,
       make+p+pfree,
         make+pfree+next, decr+nfree.

    TEST detach2 +>p>+hd>+tl>:
       is zero+p, -;
       get triple+p+p+hd+tl.

    ACTION report:
       write+"Number of work cells left= ", put int+nfree,
       put newline.

    PRELUDE create free list.

    POSTLUDE report.

  ENDSEC working storage.

ENDLAY utilities.
```

11.2.2 The functional layer

In this layer the sections `reading names` and `reading special symbols` deal with the syntax of the language of the database, whereas the sections `relations` and `transput` deal with the semantics.

The section `reading and saving the database` server to keep the database on a file between sessions.

```
LAYER administration.
```

The reading of names

This section recognizes the names of nodes and selectors and stores them into the symbol table.

```
SECTION reading names.

  ABSTR is name.

  INV
     make zero, dash char, is, is digit, is letter,
     skip layout,
     add to text, enter text.

  VAR pname.

  PREDICATE is name +key> -c:
     is letter+c, add to text+c,
       (is letter+c, add to text+c, *;
       is digit+c, add to text+c, *;
       is+dash char, add to text+dash char, *;
       enter text+pname+key, skip layout).

  ACTION init names:
     make zero+pname.

  PRELUDE init names.

ENDSEC reading names.
```

Recognising delimiters

In this section the reading and recognition of various delimiters is defined. Together with the previous section, it constitutes a simple form of lexical analyser.

```
SECTION reading special symbols.

  ABSTR is greater symbol, is equal symbol, is less symbol,
        is plus symbol.

  INV equal char, greater char, is, less char, plus char,
        skip layout.

  PREDICATE is greater symbol:
     is+greater char, skip layout.

  PREDICATE is equal symbol:
     is+equal char.

  PREDICATE is less symbol:
     is+less char.

  PREDICATE is plus symbol:
     is+plus char, skip layout.

ENDSEC reading special symbols.
```

Dealing with relations

This section provides a number of semantic actions, with which relations in the database can be established, queried and used for navigation.

```
SECTION relations.

  ABSTR write selectors, add relation, has target.

  INV
     writeln, get prop, put prop, detach2, find2, insert2,
     down, up, write name.

  ACTION write selectors +>node -rel-sel-targ:
     get prop+node+rel, up,
       (detach2+rel+sel+targ, write name+sel, *;
       down).

  ACTION add relation +>node+>sel+>targ -rel-old:
     get prop+node+rel,
       (find2+rel+sel+old,
       writeln+"Selector not unique, ignored";
       insert2+rel+sel+targ, put prop+node+rel).

  TEST has target +>node+>sel+targ> -rel:
     get prop+node+rel, find2+rel+sel+targ.

ENDSEC relations.
```

Screen layout

In this section the elements of the screen layout are introduced. In particular, the indentation with which prompts will appear on the screen is administered.

```
SECTION transput.

  ABSTR up, down, write name.

  EXT up, down, write name.

  INV decr, incr, is zero, make zero, put newline, write,
  write text.

  VAR indentation.

  ACTION indent +>p:
     is zero+p, write+"                    ";
     decr+p, write+"    ", *.

  ACTION set margin:
     make zero+indentation.

  ACTION up:
     incr+indentation.
```

```
  ACTION down:
     decr+indentation.

  ACTION write name +>x:
     indent+indentation, write text+x, put newline.

  PRELUDE set margin.

ENDSEC transput.
```

11.2.3 Reading and saving the database

The database obviously needs a mechanism for storing the results of a session until the start of a next session. It would of course be possible to keep and update the contents of the database on a file, but is also sufficient to keep it in memory during a session but to read it from file at the start of the session and write it to a file at the end of the session.

Of course the contents of the database could be dumped in some numeric form but the following scheme is more attractive. At the end of the session, the current contents of the database is written onto a file in the form of a sequence of commands which, when executed, reconstruct the current contents of the database.

Programming this is by itself a nice exercise in recursion. We then will further modify the input so that it takes place initially from a file and changes to the terminal upon meeting the end of that file.

The technique described is useful in many different situations and surprisingly easy to implement.

```
SECTION database storage.

  ABSTR save database, start database.

  INV
     zero, close outfile, put char, put newline,
     switch to infile, space char, plus char,
     switch to outfile, greater char, less char,
     start input, get prop, put prop, write text, detach2.

  ACTION start database:
     switch to infile+"database", start input.

  ACTION save database +>root:
     switch to outfile+"database", write text+root, put newline,
       dump subgraph+root, close outfile.

  ACTION dump subgraph +>node -prop-rel-target:
     get prop+node+prop, put prop+node+zero,
       (detach2+prop+rel+target, dump add command+rel+target,
         dump follow command+rel, dump subgraph+target,
         dump return command, *;
       +).

  ACTION dump add command +>rel+>target:
     dump+plus char, write text+rel, dump+space char,
       write text+target, put newline.
```

```
    ACTION dump follow command +>rel:
       dump+greater char, write text+rel, put newline.

    ACTION dump return command:
       dump+less char, put newline.

    ACTION dump +>x:
       put char+x.

  ENDSEC database storage.

ENDLAY administration.
```

11.2.4 The heart of the matter

The last layer, consisting of only one section, serves as a driver for the database. It goes into a discussion with the user of the database while (recursively) navigating over it.

```
LAYER info system.

  SECTION session.

    INV
       write, writeln, if yes, start at new line,
       start input, is name, is equal symbol,
       is greater symbol, is less symbol,
       is plus symbol, add relation, has target,
       write selectors, down, up, write name,
       save database, start database.

    PREDICATE list command +>home:
       is equal symbol, write selectors+home.

    PREDICATE follow command +>home -sel-targ:
       is greater symbol,
         (is name+sel,
           (has target+home+sel+targ, up, dialogue+targ,
           down;
           writeln+"Unknown selector name");
         writeln+"Missing selector name").

    PREDICATE add command +>home -sel-targ:
       is plus symbol,
         (is name+sel,
           (is name+targ, add relation+home+sel+targ;
           writeln+"Missing node name");
         writeln+"Missing selector name").

    PREDICATE return command:
       is less symbol.

    ACTION invalid command:
       writeln+"Invalid command",
       writeln+"Commands are:",
         writeln+" = list command",
         writeln+" + add command     selector node",
         writeln+" > follow command  selector",
         writeln+" < return command".
```

```
    #-------------------------------------------------#
    # heart of info system:                           #
    #-------------------------------------------------#
    ACTION sign on:
       writeln+"Info System".


    ACTION dialogue +>home:
       write name+home, start at new line,
         (list command+home, * dialogue;
         add command+home, * dialogue;
         follow command+home, * dialogue;
         return command;
         invalid command, * dialogue).


    ACTION create start node +start>:
       write+"Read old database", start input,
         (if yes, start database,
           (is name+start;
           writeln+"No node name in old database", ?);
         writeln+"Start node", start at new line,
           (is name+start;
           writeln+"Missing node name", * create start node)).


    ACTION discussion -home:
       sign on, create start node+home, dialogue+home,
         save database+home.


    ROOT discussion.


  ENDSEC session.

ENDLAY info system.
```

11.2.5 The program control

Finally, the program control establishes the order of the various preludes, the root and the postludes.

```
PRELUDE
   io interface, text handling, working storage,
   reading names,
   transput.


ROOT session.


POSTLUDE text handling, working storage.


ENDMOD simple database.
```

This completes the description of the database.

11.3 Possible extensions

It is not hard to suggest a number of sensible extensions to the database just described that make it a lot more interesting and possibly even useful.

Obviously a delete command is missing, with a syntax like

```
- selector
```

to delete the relationship with that selector from the current node. It is not difficult to add, but it is not immediately clear what to do if, through the deletion of a relation, some object becomes unreachable. The easiest solution is to simply leave the object in the text table, since there is no point in explicitly removing it.

Similarly, it might be useful to add a command for jumping immediately to a node with a specific name, e.g. in the form

```
! name
```

meaning: jump immediately to the node of that name (note that, because of the reading strategy followed, once the command has been read a node of that name exists, even if there is no arc leading to or from it.)

Further extensions are of course possible, bringing the database nearer to a real life system:

- it is possible to impose on each node a type, like integer, real, text or table, so that different forms of information can be kept in the database.
- it is possible to impose an a-priori structure on the database by fixing the number of fields and the names of the selectors a node can have. One attractive technique is to equip the database with a description of its contents in the form of a context-free grammar.
- we can also generalize this database for simultaneous access by many people, using some form of lockouts for safe simultaneous update.

The small database described is a good starting point for experiments in the construction of all kinds of software, not only databases — this example has already been cannibalized by a number of people to make their own compilers and information systems.

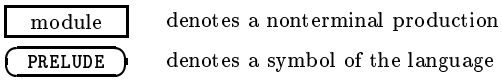
The reader is invited to exercise his own phantasy.

Appendix A

Syntax of CDL2

In this appendix we give an overview of the syntax of CDL2. (As described in [DEH76]). We make use of syntax diagrams which should be self-explanatory. The following conventions are used:

conventions



For the sequence keyword–name–period the following shorthand is used:

⟨*any*⟩ sentence

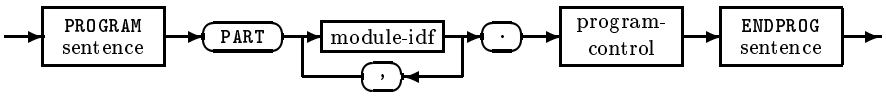
⟨*any*⟩ is to be replaced by the respective construct of the language.

We will give some remarks about semantics interspersed between the syntax rules.

Both the CDL2 compiler and the CDL2 LAB use the same concrete syntax, apart from some slight differences in the overall structure of programs due the fact that the compiler is intended for sequential input whereas in the CDL2 LAB the database has a tree-like structure. Below the level of modules, the syntax is identical.

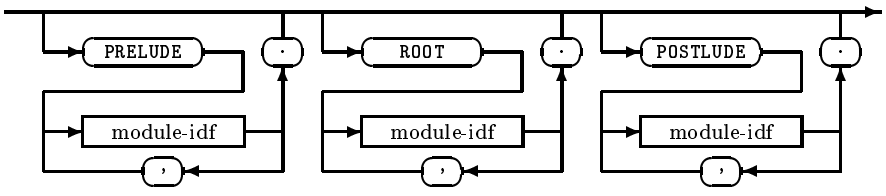
For the CDL2 LAB:

program



The PART-list gives the names of the modules out of which the program is composed.

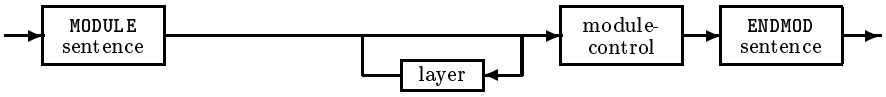
program-control



The `pcontrol` contains module names.

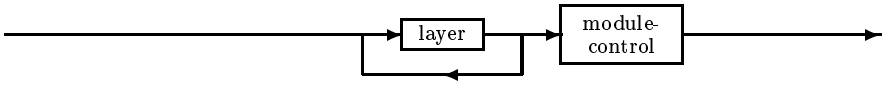
Separate from the program, the CDL2 LAB administers a number of modules, each with the following syntax:

module



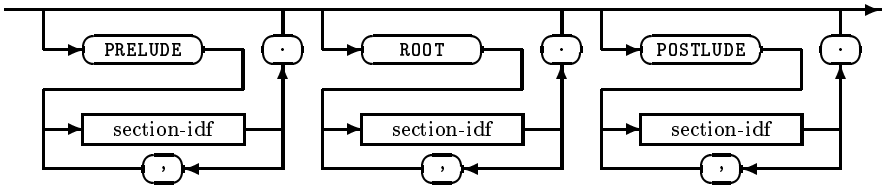
The CDL2 compiler without separate compilation allows programs consisting of one module only, with the slightly simplified syntax

program



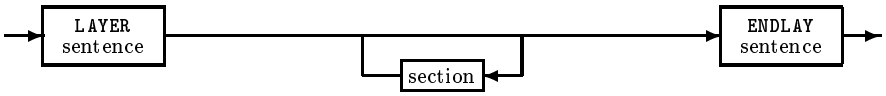
From now on the syntax is the same for compiler and CDL2 LAB:

module-control



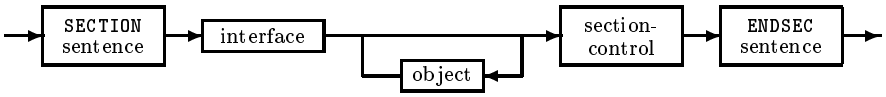
The module control contains section names.

layer



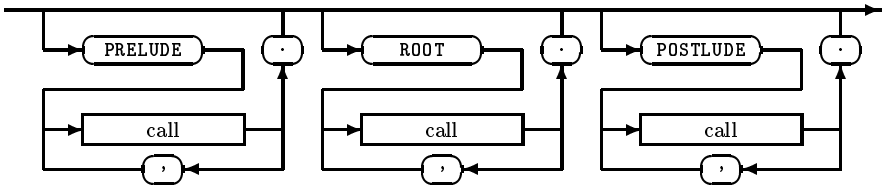
The layers should be given in bottom-up order.

section

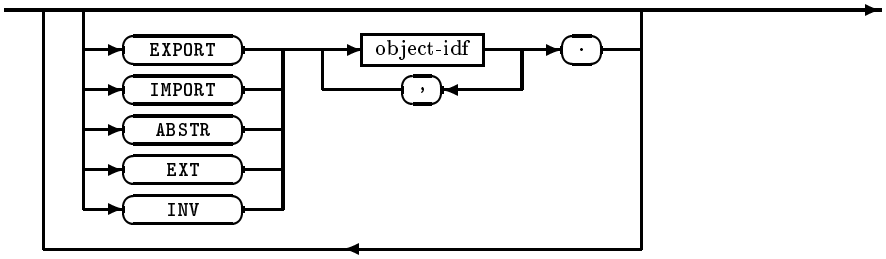


The order of sections within a layer is free.

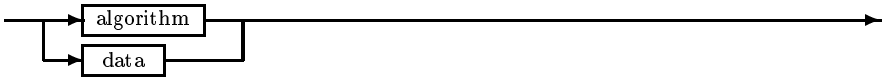
section-control



interface



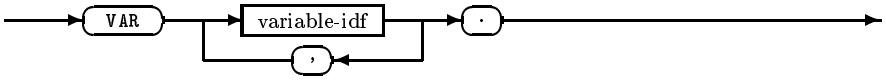
object



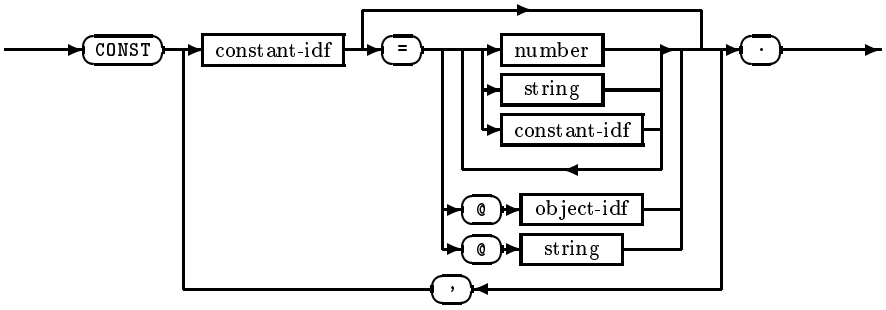
The order of objects in a section is free.

data

variable



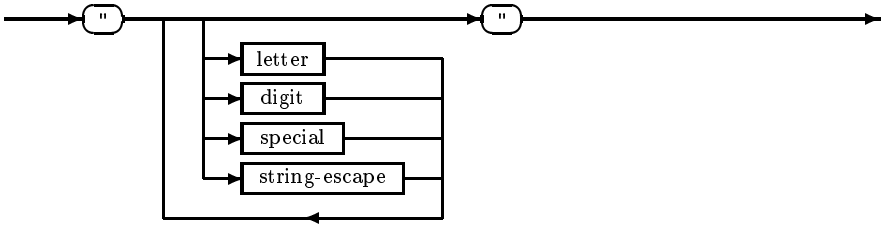
constant



The elements in a constant declaration have the same form as those that occur in a macro specification. In fact a constant can be seen as a parameterless macro, whose meaning depend on the target environment.

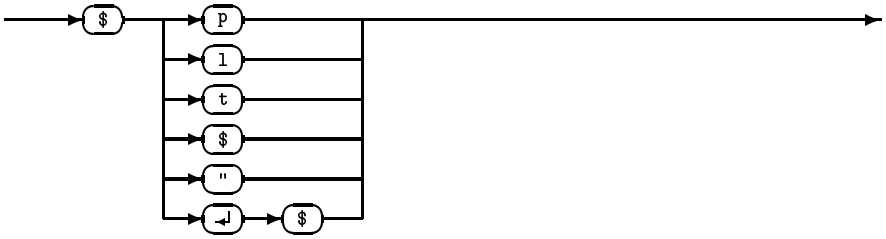
A string is taken literally, i.e. transliterated to the sequence of its string elements. A name is translated to an equivalent name in the target code, in a manner described in the manual for the particular CDL2 code generator. A number is translated to its equivalent in the target code (e.g. some hexadecimal literal). Layout is not transliterated. A `@` followed by an object name or string is translated to the address of the object; this construct is not supported by all code generators towards high-level languages.

string



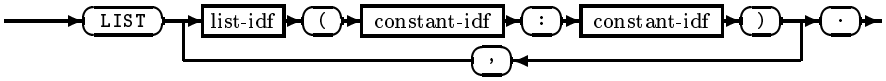
The dollar is used as the escape character. The following escape sequences exist:

string-escape



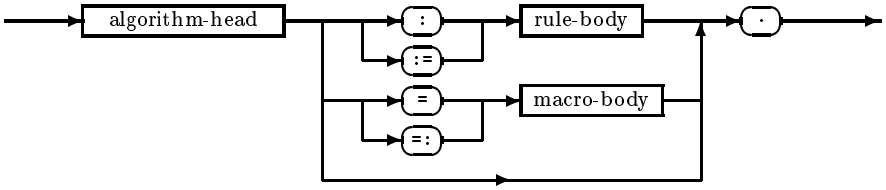
The `$p, $l, $t` are translated to new page, new line, or tab characters respectively. A `$$` and `$"` is used to enter a dollar or quote character. To spread a string over several lines can be done with `$` followed by any layout characters and another `$`.

list

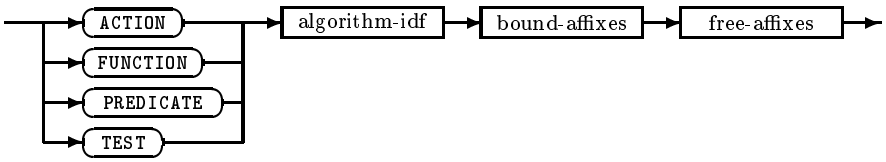


The bounds have to be constants (not numbers!).

algorithm

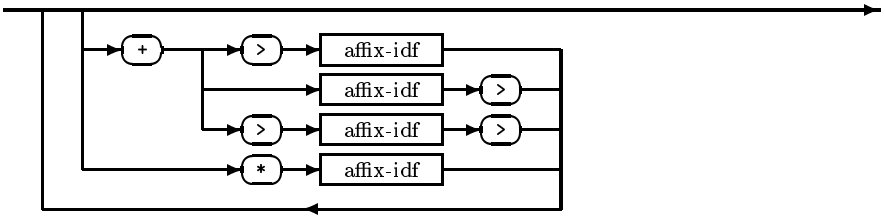


algorithm-head



The headings of macros and procedures have the same form. In particular also a macro can have local parameters, which are then administered by the CDL2 compiler.

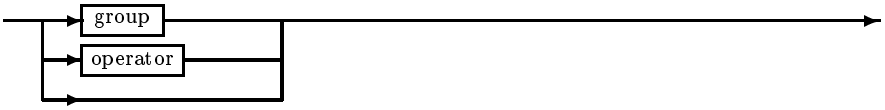
bound-affixes



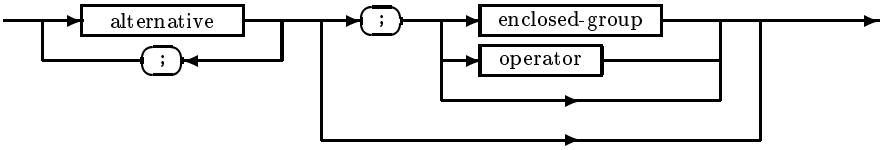
The fourth form of formal parameters serves only for passing texts.

free-affixes

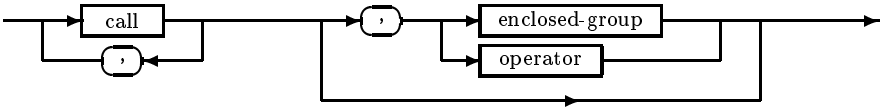
rule-body



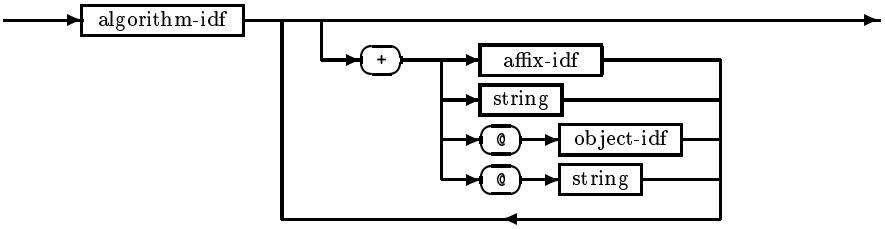
group



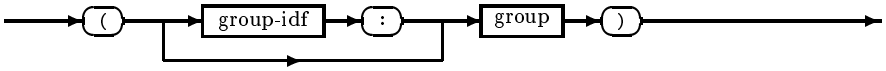
alternative



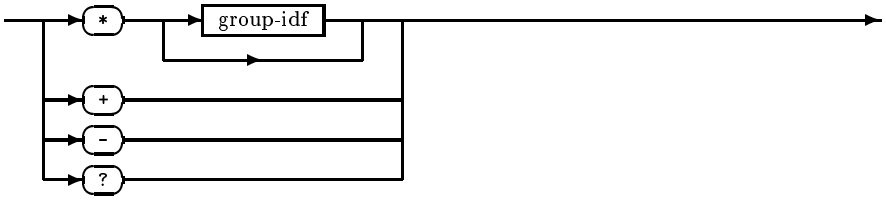
call



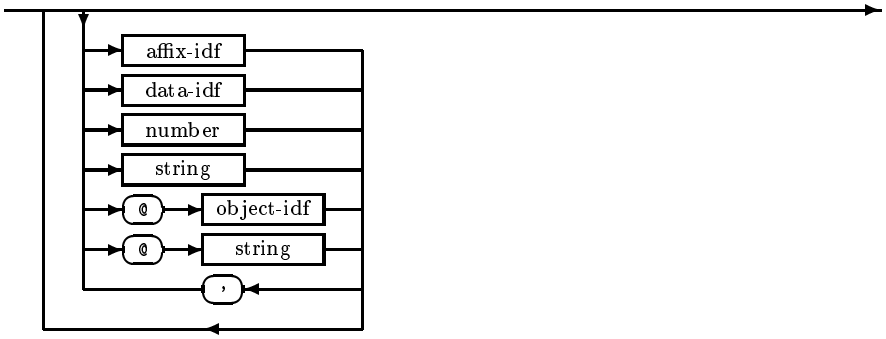
enclosed-group



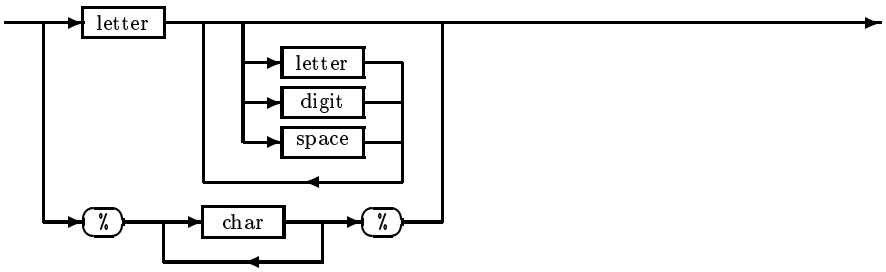
operator



macro-body

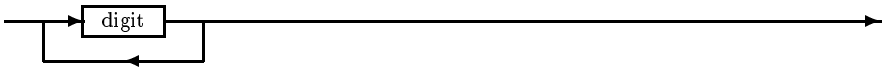


⟨*any*⟩-idf



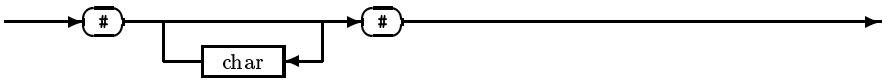
Names may contain spaces to enhance readability. They are ignored by the compiler.

number



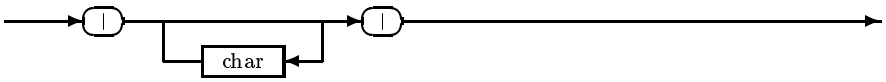
Finally we have two constructs that can appear anywhere in the text

comment



A comment does not influence the meaning of the program.

pragmat



A pragmat is a special comment, which is (intended as) meaningful to the compiler, and influences its behaviour.

Finally we point out the lexical convention that constructions, such as comments, pragmats and strings, that begin and end with the same token are not allowed to extend over one line. This simplifies error recuperation.

This is all the syntax there is — CDL2 is so simple.

Appendix B

Standard macros

The following set of macros is delivered which each code generator towards assembly language. On most computers there are other standard algorithms for input/output and other operating system functions. But some of these are not completely machine independent.

Because high-level languages like PASCAL do not support addressing of general data types, the CDL2 address operator is not supported for these target languages. Therefore the access macros for lists and strings cannot be given in this generality.

Assignment and simple arithmetic operations

```
FUNCTION make + x> + >y.
FUNCTION incr + >x>.
FUNCTION decr + >x>.
FUNCTION neg + >x>.
FUNCTION set zero + x>.
FUNCTION abs + >x>.
```

Arithmetic comparisons and tests

```
TEST equal + >x + >y.
TEST less + >x + >y.
TEST lseq + >x + >y.
TEST is zero + >x.
```

Arithmetic operations (2 address style)

```
FUNCTION add ab + >x> + >y.
FUNCTION sub ab + >x> + >y.
FUNCTION mul ab + >x> + >y.
FUNCTION div ab + >x> + >y.
```

Arithmetic operations (3 address style)

```
FUNCTION add + >x + >y + z>.
FUNCTION sub + >x + >y + z>.
FUNCTION mul + >x + >y + res>.
FUNCTION div + >x>y + z>.
```

```
FUNCTION divrem + >div + >dor + quot> + rem>.
FUNCTION mod + >div + >dor + rem>.
```

Testing and setting Boolean values

```
FUNCTION set off + x>.
FUNCTION set on + x>.
TEST     is off + >x.
```

Logical operations

```
FUNCTION and ab + >x> + >y.
FUNCTION or ab + >x> + >y.
FUNCTION xor ab + >x> + >y.
FUNCTION not ab + >x>.

FUNCTION and + >x + >y + z>.
FUNCTION or + >x + >y + z>.
FUNCTION xor + >x + >y + z>.
FUNCTION not + >x + y>.
```

Bit field and mask operations

```
FUNCTION include + >x> + >mask.
FUNCTION exclude + >x> + >mask.
TEST     contains + >x + >mask.

FUNCTION include bit + >x> + >pos.
FUNCTION exclude bit + >x> + >pos.
TEST     bit included + >x + >pos.

FUNCTION shift right + >x> + >bits.
FUNCTION shift 1 right + >x>.
FUNCTION shift left + >x> + >bits.
FUNCTION shift 1 left + >x>.
```

List access operations

```
FUNCTION access list + >list + >index + buff> + off>.

ACTION   store buff hw + >buff + >off + >hw.
FUNCTION load buff hw + >adr + >off + hw>.
ACTION   store buff by + >buff + >off + >by.
FUNCTION load buff by + >buff + >off + by>.
```

String access operations

```
FUNCTION access string *string + handle>.
TEST has string next char *string + >handle> + char>.
```

Bibliography

[ASU86] A. V. Aho, R. Sethi, J. D. Ullman, *COMPILERS — Principles, Techniques and Tools*, Addison-Wesley publ. co., 1986

[ALL70] F. E. Allen, *Control Flow Analysis*, SIGPLAN Notices 5/7, July 1970

[BAU74] F. L. Bauer(Ed.), J. Eickel (Eds.), *Compiler Construction, An Advanced Course*, Lecture Notes in Computer Science 21, Springer Verlag, Berlin 1976

[BAY81] M. Bayer, B. Böhringer, J.-P. Dehottay, H. Feuerhahn, J. Jasper, C. H. A. Koster, U. Schmiedecke, *Software Development in the CDL2 Laboratory*, in: [HUN81]

[BOE81] B. Böhringer, H. Feuerhahn, *Separate and Integral Compilation of Subsystems*, in: A. J. W. Duijvestijn, P. C. Lockemann, editors, *Trends in Information Processing Systems*, pp. 50–64, Lecture Notes in Computer Sciences 123, Springer Verlag, 1981

[BOE82] B. Böhringer, H. Feuerhahn, *Static Semantic Checks of Global Variables in a Procedural Language*, in *Programmiersprachen und Programmentwicklung*, pp. 165–176, Informatik Fachberichte 53, Springer Verlag, 1982

[BOO76] H. J. Boom, E. de Jong, *A critical comparison of several implementations of programming languages*, Report IW 58/76, Mathematisch Centrum Amsterdam, 1976

[COL74] S. S. Coleman, P. C. Poole, W. M. Waite, *The Mobile Programming System Janus*, in: Software Practice and Experience, Vol 4, 5-23 (1974)

[CON58] M. E. Conway, *Proposal for an UNCOL*, Comm. ACM, Vol. 1 No. 10, 1958

[CON77] R. Conradi and P. Holager, *Mary Textbook*, RUNIT 1977

[COU77] P. Cousot, R. Cousot, *Static Verification of Dynamic Type Properties of Variables*, in: Proceedings 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, St. Monica, CA, Jan. 1977

[CUN76] K. Cinnis, M. Simonet and J. Voiron, *Methodologie d'Ecriture de Compilateur, une experience du langage ALGOL68*, Dissertation, IMAG Grenoble, 1976

[DAH72] O. J. Dahl, E. W. Dijkstra, C. A. R. Hoare, *Structured Programming*, Academic Press, London, 1972

[DEH76] J.-P. Dehottay, H. Feuerhahn, C. H. A. Koster, H.-M. Stahl: *Syntaktische Beschreibung von CDL2*; Technische Universität Berlin, Forschungsgruppe Softwaretechnik, Sept. 1976

[DRE75] F. DeRemer and H. Kron: *Programming-in-the-Large versus Programming-in-the-Small*; Proceedings of the International Conference on Reliable Software; SIGPLAN Notices Vol. 10 No. 6, June 1975

[EAR70] J. Earley, H. Sturgis, *A Formalism for Translator Interactions*, Comm. ACM, Vol. 13 No. 10, 1970

[EPS88a] epsilon, *CDL2 LAB Reference Manual*, Release 2.1, August 1988

[EPS88b] epsilon, *CDL2 LAB User's Guide*, Release 2.1, August 1988

[FEU75] H. Feuerhahn, *A Binary Control Structure and its Relationship to Grammars and Side-Effects*, In Proceedings of the 4th annual GI Meeting, Lecture Notes in Computer Science 26, Springer-Verlag, Berlin 1975

[FEU75a] H. Feuerhahn, *Semantische Überprüfung und Optimierung von CDL-Programmen*, TU Berlin, FB 20, Diplomarbeit, June 1975

[FEU78] H. Feuerhahn, C. H. A. Koster, *Static Semantic Checks in an Open-Ended Language*; in: [HIB78]

[FEU88] H. Feuerhahn, *Data-Flow Driven Resource Allocation in a Retargetable Microcode Compiler*, in *Proceedings 21st Microprogramming Workshop*, San Diego, CA, 1988

[FOS76] L. D. Fosdick, L. J. Osterweil, *Data Flow Analysis in Software Reliability*, ACM Computing Surveys 8/3, Sept. 1976

[GOO78] G. Goos and U. Kastens: *Programming Languages and the Design of Modular Programs*; in: [HIB78]

[GRU73] D. Grune, L. G. L. T. Meertens, J. C. van Vliet, *Grammar-Handling Tools applied to ALGOL 68*, IW/73, Mathematisch Centrum, Amsterdam, 1973

[GRU82] D. Grune, *On the design of ALEPH*, CWI Tract 13, Centre for Mathematics and Computer Science, Amsterdam 1982

[HAN74] P. Brinch Hansen: *Concurrent PASCAL, a Programming Language for Operating System Design*; Information Science Technical Report No. 10, California Institute of Technology, April, 1974

[HIB78] P. G. Hibbard and S. A. Schuman (Eds.) *Constructing Quality Software*; North-Holland Publishing Co., 1978

[HOL80] P. Holager, *Revised Code Generator Interface for the CDL2 Compiler*, Report No 13, Informatics Department, University of Nijmegen, 1980

[HUN81] H. Hünke, (ed.): *Software Engineering Environments*; North-Holland Publishing Co., 1981

[JAE79] S. Jähnichen, *'Exception Handling' in sequentiellen Programmen*, Dissertation, Technical University of Berlin, 1979

[JAE??] S. J̈hnichen article

[KIL73] G. A. Killdal, *A Unified Approach to Global Program Optimization*, ACM Symposium on Principles of Programming Languages, Boston, October 1973

[KIP82] I. M. Kipps: *Experience with Porting Techniques on a COBOL 74 Compiler*; in: Proceedings of the SIGPLAN 82 Symposium on Compiler Construction, Boston June 1982

[KOS65] C. H. A. Koster, *On the construction of ALGOL-procedures for generating, analysing and translating sentences in natural languages*, Report MR72, Mathematical Centre, Amsterdam 1965

[KOS71a] C. H. A. Koster, *A Compiler Compiler*, Report MR127/71, Mathematical Centre, Amsterdam, 1971

[KOS71b] C. H. A. Koster, *Affix Grammars*, in: J. E. L. Peck(Ed.), ALGOL 68 Implementation, North-Holland Publishing Co., Amsterdam 1971

[KOS72] C. H. A. Koster, *Towards a Machine-Independent ALGOL 68 Translator*, Report MR 129/72, Mathematical Centre, Amsterdam 1972

[KOS74a] C. H. A-Koster, *Portable Compilers and the UNCOL Problem*, in: Proceedings of a Conference on Machine-Oriented Higher Level Languages, North-Holland Publishing Co., 1974

[KOS74b] C. H. A. Koster, *Using the CDL Compiler Compiler*, in [BAU74]

[KOS76] C. H. A. Koster, *Visibility and Types*, in: Proceedings of an ACM Conference on Data Abstraction, SIGPLAN Notices vol 8 no 2, March 1976

[KOS87] C. H. A. Koster, *Top-Down Programming with Elan*, Ellis Horwood Publ. Cy, Chichester, 1987

[KRA85] B. Krämer, *Interactive Graphical Specification in a Syntax-Directed Environment: The SEGRAS-Lab Experience*, GRASPIN Technical Paper GMD 25/2, Gesellschaft für Mathematik und Datenverarbeitung, Birlinghoven, August 1985

[LAM77] B. W. Lampson, J. J. Horning, R. L. London, J. G. Mitchel, and G. L. Popek, *Report On The Programming Language Euclid*, SIGPLAN Notices vol 12, no 2, February 1977

[LAN73] H. Langmaack, *On Correct procedure parameter transmission in higher programming Languages*, Acta Informatica 2, 1973

[MEE62] L. G. L. Th.A-Meertens, C. H. A. Koster, *An Affix Grammar for a subset of English*, presented at Euratom Colloquium, Amsterdam 1962.

[OTT??] Otter, Dissertation

[PAR72] D. L. Parnas: *On the Criteria used in Decomposing Systems into Modules*; CACM Vol. 15 No. 12, December 1972

[POO73] P. C. Poole and W. M. Waite: *Portability and Adaptability*; in: Advanced Course on Software Engineering; Lecture Notes in Economics and Mathematical Systems 81, Springer 1973

[ROB75] L. Robinson, K. N. Levitt, P. G. Neumann, A. R. Saxena, *On attaining reliable software for a secure operating system*; SIGPLAN Notices, vol. 10 no 6 June 75

[RIC69] M. Richards, *BCPL: A Tool for Compiler Writing and System Programming*, Proc. AFIPS. SJCC 34 (1969) 557-566

[ROS70] D. A. Ross, *Uniform Referents, an Essential Property for a Software Engineering Language*, in: Software engineering, Academic Press 1970

[ROS??] D. A. Ross, *Structured Analysis and Design*

[SAB76] M. A. Sabin, *Portability — Some Experiences with FORTRAN*, in Software Practice and Experience, Vol 6, 393-396 1976

[SCH86] T. J. Schwartz, R. B. Dewar, E. Dubinsky, E. Schonberg, *Programming with Sets: An Introduction to SETL*, Springer Verlag, 1986

[SOM76] J. Somogyi, *Minicomputer Software Design and Implementation Based on the Use of a Systems Programming Language*, in: J. R. Bell and C. G. Bell, eds., Minicomputer Software, North-Holland Publishing Co., Amsterdam, 1976

[SOM79] J. Somogyi, *About portability of operating systems*, First János Neumann Congress, Szeged, 1979

[STA80] H. M. Stahl, *Portability and Efficiency in Using an Open-Ended Language*, in Angewandte Informatik 1980

[TAN??] A. Tanenbaum

[TEU89] I. Teufer, H. Schmitt, *The Abstract Syntax Definition Language for GRASPIN's Lisp Prototypes: Reference Manual*, GRASPIN technical paper EPS 7/1, Gesellschaft für Mathematik und Datenverarbeitung, Birlinghoven, April 1989

[WIC77] B. A. Wichman, *Test X1: Transitive Closure*, in: IFIP WG 2.4 Test Programs for System Implementation Languages, 1977

[WIR80] N. Wirth: MODULA-2, ETH Zürich, Institut für Informatik, report 36, March 1980

[WIJ69] A. van Wijngaarden(Ed.), B. J. Mailloux, J. E. L. Peck, C. H. A. Koster, *Report on the Algorithmic Language ALGOL68*, Numerische Mathematik 14, Springer Verlag, 1969.

[WIJ75] A. van Wijngaarden et al.(Eds.), *Revised Report on the ALgorithmic Language ALGOL68*, Acta Informatica 5, pp. 1–236, 1975; also SIGPLAN Notices 12(5), pp. 5–70, 1977

[WUL73] W. A. Wulf and M. Shaw: *Global variables considered harmful*, SIGPLAN Notices, vol. 8 no 2 Feb. 1973.

Index